

MVIS — Machine Vision Inspection System

Enterprise Architecture Document

Field	Value
Version	2.0
Status	Production Design — Implementation Active
Last Updated	2026-08-02
Classification	Internal — Architecture Review Board
Authors	MVIS Engineering

Table of Contents

- [1. Executive Summary](#)
- [2. Design Principles](#)
- [3. System Overview](#)
- [4. Architecture Diagram](#)
- [5. Service Placement](#)
- [6. Data Flows](#)
- [7. Redis Streams Pipeline](#)
- [8. Database Architecture](#)
- [9. Storage Architecture](#)
- [10. Synchronization Engine](#)
- [11. Network & Security Architecture](#)
- [12. Observability Architecture](#)
- [13. Deployment Strategy](#)
- [14. Implementation Phases](#)
- [15. Service Level Objectives](#)
- [16. Capacity Planning](#)
- [17. Session State Machine](#)
- [18. Failure Matrix](#)
- [19. Disaster Recovery](#)
- [20. Runbooks](#)
- [21. Architecture Decision Records](#)
- [22. Future Scaling Roadmap](#)
- [23. Risks and Mitigations](#)
- [24. Performance Benchmarks](#)
- [25. Production Readiness Checklist](#)
- [26. Appendices](#)

1. Executive Summary

1.1 Purpose

MVIS (Machine Vision Inspection System) is a distributed, GPU-accelerated railway inspection platform. A train passes through a fixed camera array at a maintenance depot; the system captures frames from twelve cameras simultaneously, routes them through a GPU inference pipeline on an NVIDIA DGX Spark, and produces a structured inspection report for every train visit. The report identifies defects, anomalies, and relevant OCR-extracted markings — providing a timestamped, auditable record for railway maintenance engineers.

1.2 Architecture Philosophy

The architecture is built around four immovable constraints:

1. **Frames never leave the DGX during a live session.** At 50 FPS × 12 cameras, real-time frame transmission over any network is infeasible. All inference runs on-device.
2. **The DGX must operate completely independently from the VPS.** Loss of network connectivity must not interrupt capture or inference. The VPS is the management plane; it is never in the hot path.
3. **Storage is decoupled from compute.** Frame objects are written to an S3-compatible store (MinIO on DGX during development; DO Spaces in production). Services hold paths, never raw bytes.
4. **The system must be auditable.** Every frame, every detection, every report, and every operator action must be traceable end-to-end via a correlation identifier.

1.3 Goals

Goal	Metric
Frame capture	50 FPS × 12 cameras = 600 frames/second sustained
Inference latency	YOLO + OCR complete within 100ms per frame
Pipeline throughput	≥ 540 frames/second (10% tolerance for backlog spikes)
Report generation	< 5 minutes after session end
Sync latency	Results visible in dashboard within 10 seconds of inference
System uptime	99.5% during operational hours (06:00–22:00 IST)
Data durability	Zero loss of inspection results after session commit to DGX PostgreSQL

1.4 Scalability Strategy

The architecture is designed in three tiers of scale:

- **Tier 1 (current):** Single DGX Spark, single VPS, single inspection station. Supports 1–10 train inspections per day.
- **Tier 2 (6–12 months):** Multiple DGX nodes per station, or multiple stations each with their own DGX. VPS is the central aggregation point. Supports 50–200 inspections per day.
- **Tier 3 (12–24 months):** Regional deployment with AGX Orin edge nodes for lightweight capture and a central DGX cluster for heavy inference. Supports national railway rollout.

Every schema, API, and configuration decision in this document is made with Tier 2 in mind and Tier 3 as a deliberate future target.

1.5 Document Scope

This document covers the complete system architecture: infrastructure, service design, data flows, database schema, event pipeline, storage, synchronization, security, observability, deployment, failure modes, disaster recovery, operational runbooks, and scaling roadmap. It is intended to serve as the single source of truth for engineering, operations, and customer-facing architecture reviews.

2. Design Principles

The following principles govern every architectural decision in MVIS. Any proposal that violates a principle must include an explicit justification and a mitigation plan.

P1 — Edge-First Inference

GPU inference runs at the edge (DGX), not in the cloud. Network topology and bandwidth constraints make real-time cloud inference infeasible for high-FPS multi-camera workloads. The VPS is a coordination and presentation layer only.

P2 — Event-Driven Pipeline

All inter-service communication within the inference pipeline uses Redis Streams (async, durable, backpressured). Services never call each other directly; they publish to and consume from named streams. This enables independent scaling, independent restartability, and zero coupling between inference stages.

P3 — Stateless Services Where Possible

Pipeline services (preprocessing, YOLO, OCR, correlation, worker) are stateless with respect to session data. All session state lives in Redis Streams (in-flight) and PostgreSQL (committed). Services can crash and restart without loss of data, provided Redis persistence is enabled.

P4 — Separation of Concerns

The ingestion layer (camsrv), inference pipeline, storage layer, synchronization layer, and presentation layer are strictly separated. No service crosses layer boundaries except through well-defined interfaces (Redis streams, PostgreSQL, REST API).

P5 — Loose Coupling, High Cohesion

Services are coupled only through the Redis stream message schema. Changing YOLO model logic does not require changing the correlation service. Changing the storage backend does not require changing the pipeline. Services are internally cohesive: each does exactly one job.

P6 — At-Least-Once Delivery with Idempotent Consumers

Every pipeline consumer implements XACK after processing and XAUTOCLAIM on startup to reclaim stale pending messages. Every database insert uses `ON CONFLICT DO NOTHING` for idempotency. The combination guarantees at-least-once processing with no duplicate rows.

P7 — Local Disk as Write Buffer

Raw frames are written to local NVMe first, then uploaded to MinIO asynchronously. The capture hot path never blocks on network or object storage I/O. This decouples frame capture reliability from storage availability.

P8 — Storage Abstraction via S3 Compatibility

MinIO during development; DO Spaces in production. Both expose the S3 API. Services use the boto3/minio-py SDK exclusively. Migration is a three-environment-variable change with zero code changes.

P9 — Defense in Depth

Security is layered: network firewall, Tailscale mesh VPN, TLS on all external traffic, JWT authentication on all APIs, per-service MinIO credentials with least-privilege policies, audit logging on all state-modifying operations. No single point of security failure.

P10 — Observability as a First-Class Citizen

Every service exposes `/metrics` (Prometheus), `/healthz` (liveness), and `/readyz` (readiness). Every log line is structured JSON containing `trace_id`, `session_id`, and `frame_id` where applicable. Distributed traces span the full pipeline from camsrv to the report storage write.

P11 — Graceful Degradation

The system is designed to continue operating in a degraded state rather than failing completely. If OCR fails, YOLO results are still preserved (with OCR fields null and a `partial_result` flag). If the VPS is unreachable, the DGX continues capturing and buffering indefinitely. No single service failure causes total system failure.

P12 — Auditability

Railway inspection reports may be used as legal documents in maintenance disputes. Every detection row carries a `model_version` , `trace_id` , and `captured_at` hardware timestamp. Every API call that modifies state is written to an audit log. Reports are digitally signed.

3. System Overview

3.1 Machines

Machine	Role	Tailscale IP	Public IP	Hostname
DGX Spark	GPU inference, frame capture, local storage	100.76.12.44	None	edgexpert-d8f3
VPS (DO BLR1)	Orchestration, API, dashboard, reporting	100.111.143.3	64.227.138.37	—
AGX Orin	Future edge node (offline currently)	100.76.123.107	None	mvis-desktop

3.2 Domain Architecture

Domain: `mvis.online` All public subdomains resolve to VPS (`64.227.138.37`). Traefik on VPS routes VPS-local services directly and proxies DGX services over Tailscale.

```
Browser → *.mvis.online → 64.227.138.37 (Cloudflare DNS-only, no proxy)
      → Traefik (VPS) → VPS service
      → Tailscale → DGX service
```

3.3 Deployment Platforms

Machine	Platform	Version
DGX	Coolify	Latest
VPS	Dokploy	v0.29.13

Both platforms manage Docker containers and handle TLS via Let's Encrypt. Traefik is the reverse proxy on both machines.

4. Architecture Diagram

```

graph TB
    subgraph CAMERAS["Camera Array (12x cameras, 50 FPS each)"]
        C1[Cam 00] & C2[Cam 01] & C3[...] & C4[Cam 11]
    end

    subgraph DGX["DGX Spark - GPU Inference Node (100.76.12.44)"]
        direction TB

        subgraph INGEST["Ingestion Layer"]
            CAMSVR["camsvr\n(session mgmt + capture)"]
            DASHBOARD["mvis-dashboard\n(operator UI) :9003"]
        end

        subgraph CAPTURE_STORE["Local NVMe Write Buffer"]
            NVME["/data/capture/{session}/{seq:08d}.jpg"]
        end

        subgraph PIPELINE["Pipeline Layer - Redis Streams"]
            direction LR
            S_RAW["Stream: raw_frames\nMAXLEN 10,000"]
            PREP["preprocessing\n(GPU: resize, denoise)"]
            S_PRE["Stream: preprocessed_frames\nMAXLEN 5,000"]
            YOLO["yolo\n(GPU: object detection)"]
            OCR["ocr\n(GPU: text recognition)"]
            ASYNC_UP["async-uploader\n(MinIO write)"]
            S_DET["Stream: detections\nMAXLEN 50,000"]
            S_OCR["Stream: ocr_results\nMAXLEN 20,000"]
            CORR["correlation\n(join buffer, 60s timeout)"]
            S_INSP["Stream: inspection_results\nMAXLEN 5,000"]
            PERSIST["result-persister\n(100ms batch INSERT)"]
            DLQ["DLQ handler\n(poison message sink)"]
        end

        subgraph STORAGE_DGX["Storage Layer"]
            PG_DGX["PostgreSQL\n(hot buffer)"]
            MINIO["MinIO\n(S3-compatible)\n:9010"]
        end

        subgraph SYNC["Sync Layer"]
            SYNC_ENG["sync-engine\n(state machine)"]
            FRAME_ARC["frame-archiver\n(replaces mc mirror)"]
            SESSION_MGR["session-manager\n(heartbeats + lifecycle)"]
        end
    end

    subgraph VPS["VPS - DigitalOcean BLR1 (100.111.143.3 / 64.227.138.37)"]
        direction TB
        TRAEFIK["Traefik v3\n(reverse proxy + TLS)"]
        PG_VPS["PostgreSQL\n(master + history)"]
        REDIS_VPS["Redis\n(job queue)"]
        BACKEND["vande Bharat-backend\n(napi.mvis.online)"]
        FRONTEND["vande Bharat-frontend\n(nfrontend.mvis.online)"]
        REPORT["report-generator"]
        PROM["Prometheus"]
        GRAFANA["Grafana\n(grafana.mvis.online)"]
        LOKI["Loki\n(log aggregation)"]
        HEARTBEAT["session-heartbeat-monitor"]
        HEADSCALE["Headscale\n(Tailscale control plane)"]
    end

    subgraph SPACES["Frame Storage - S3-Compatible"]
        NOW["NOW: MinIO on DGX\n(same machine)"]
        LATER["LATER: DO Spaces BLR1\n(direct DGX upload)"]
    end

    CAMERAS --> CAMSVR
    CAMSVR --> NVME
    CAMSVR --> S_RAW
    NVME --> ASYNC_UP
    S_RAW --> PREP
    PREP --> S_PRE
    S_PRE --> YOLO & OCR & ASYNC_UP
    YOLO --> S_DET
    OCR --> S_OCR
    S_DET --> CORR
    S_OCR --> CORR
    CORR --> S_INSP
    S_INSP --> PERSIST
    PERSIST --> PG_DGX
    S_RAW & S_DET & S_OCR & S_INSP --> DLQ

```

```

ASYNC_UP → MINIO

PG_DGX → SYNC_ENG
SYNC_ENG -- "Tailscale\nJSON batches only\nframes NEVER" → PG_VPS
SESSION_MGR -- "heartbeat\n30s" → HEARTBEAT
FRAME_ARC → SPACES

TRAEFIK → BACKEND & FRONTEND & GRAFANA
TRAEFIK -- "proxy via Tailscale" → DASHBOARD
BACKEND → PG_VPS & REDIS_VPS
REPORT → PG_VPS & MINIO
REDIS_VPS → REPORT
PROM → GRAFANA
LOKI → GRAFANA

FRONTEND -- "presigned URLs\ndirect fetch" → SPACES

style DGX fill:#1a1a2e,stroke:#4a90d9,color:#ffffff
style VPS fill:#0f3460,stroke:#4a90d9,color:#ffffff
style SPACES fill:#162447,stroke:#4a90d9,color:#ffffff
style CAMERAS fill:#1a0a0a,stroke:#d94a4a,color:#ffffff
    
```

4.1 Data Flow Summary

```

What travels over Tailscale (DGX → VPS)
✓ Structured results - JSON batches (< 5 MB/session)
✓ Session lifecycle events (start, heartbeat, end, complete)
✓ Sync confirmation receipts
X Raw frames - NEVER (72 GB/session at 120 MB/s)
X Preprocessed frames - NEVER
X Report PDFs - NEVER (uploaded direct from DGX to DO Spaces)
    
```

```

What travels from DGX to Storage (direct, no VPS in path)
✓ Frames → MinIO (local, zero network cost)
✓ Frames → DO Spaces (direct DGX upload via frame-archiver)
X VPS bandwidth is NEVER used for frame data
    
```

5. Service Placement

Service	Current Machine	Target Machine	Subdomain	Notes
<code>mvis-camsvr</code>	DGX	DGX	—	Must colocate with cameras and GPU pipeline
<code>mvis-dashboard</code>	DGX	DGX (proxied via VPS)	<code>cam.mvis.online</code>	Operator UI, live camera feed
<code>session-manager</code>	—	DGX (new)	—	Session lifecycle + heartbeat sender
<code>vandebharat-preprocessing</code>	DGX	DGX	—	GPU-bound
<code>vandebharat-yolo</code>	DGX	DGX	—	GPU-bound
<code>vandebharat-ocr</code>	DGX	DGX	—	GPU-bound
<code>vandebharat-correlation</code>	DGX	DGX	—	Reads GPU outputs in-stream
<code>result-persister</code>	DGX	DGX (split from worker)	—	Stateless DB writer
<code>async-uploader</code>	—	DGX (new)	—	Async frame → MinIO upload
<code>frame-archiver</code>	—	DGX (replaces mc mirror)	—	MinIO → DO Spaces with checksum
<code>sync-engine</code>	DGX	DGX	—	State-machine sync to VPS
<code>vandebharat-redis</code>	DGX	DGX	—	Pipeline backbone (AOF enabled)
<code>vandebharat-postgres</code>	DGX	DGX (hot) + VPS (master)	—	Two-DB pattern
<code>vandebharat-minio</code>	DGX	DGX	<code>minio.mvis.online</code>	S3-compatible frame store
<code>vandebharat-backend</code>	DGX	VPS	<code>api.mvis.online</code>	REST API — no GPU needed
<code>vandebharat-frontend</code>	DGX	VPS	<code>frontend.mvis.online</code>	Dashboard — no GPU needed
<code>vandebharat-report-generator</code>	DGX	VPS	—	No GPU needed
<code>mvis-grafana</code>	DGX	VPS	<code>grafana.mvis.online</code>	Monitoring UI
<code>mvis-prometheus</code>	DGX	VPS	—	Scrapes both machines via Tailscale
<code>loki</code>	—	VPS (new)	—	Log aggregation
<code>session-heartbeat-monitor</code>	—	VPS (new)	—	Detects stale sessions
<code>headscale</code>	—	VPS (new)	—	Self-hosted Tailscale control plane

6. Data Flows

6.1 Flow 1 — Active Train Session (Frame Ingestion)

```

sequenceDiagram
    participant OP as Operator
    participant DASH as mvis-dashboard
    participant CAMSVR as camsvr
    participant NVME as Local NVMe
    participant REDIS as Redis Streams
    participant PREP as preprocessing (GPU)
    participant YOLO as yolo (GPU)
    participant OCR as ocr (GPU)
    participant UP as async-uploader
    participant MINIO as MinIO
    participant CORR as correlation
    participant PERS as result-persister
    participant PGDGX as PostgreSQL (DGX)
    participant SYNC as sync-engine
    participant PGVPS as PostgreSQL (VPS)

    OP->>DASH: Click "Start Session"
    DASH->>CAMSVR: POST /sessions/start
    CAMSVR->>PGDGX: INSERT session (status=active)
    CAMSVR->>PGVPS: POST /api/sessions (via sync-engine)
    CAMSVR->>DASH: session_id

    loop Every frame (600 fps across 12 cameras)
        CAMSVR->>NVME: Write /data/capture/{session}/{seq:08d}_{cam}.jpg
        CAMSVR->>REDIS: XADD raw_frames {trace_id, frame_id, local_path, cam_id, seq, ts_hw}
    end

    loop Async (decoupled from capture)
        UP->>REDIS: XREADGROUP raw_frames async-upload-group
        UP->>MINIO: PUT /sessions/{id}/{seq:08d}_{cam}.jpg
        UP->>REDIS: XACK raw_frames
        UP->>NVME: Delete local file after confirmed MinIO write
    end

    loop Pipeline (GPU)
        PREP->>REDIS: XREADGROUP raw_frames preprocessing-group
        PREP->>PREP: resize, denoise, normalize
        PREP->>REDIS: XADD preprocessed_frames {trace_id, frame_id, ...}
        PREP->>REDIS: XACK raw_frames

        par YOLO and OCR in parallel
            YOLO->>REDIS: XREADGROUP preprocessed_frames yolo-group
            YOLO->>YOLO: Inference (MAXLEN: one msg per bbox)
            YOLO->>REDIS: XADD detections {trace_id, frame_id, label, conf, bbox, model_version}
            YOLO->>REDIS: XACK preprocessed_frames

            and
                OCR->>REDIS: XREADGROUP preprocessed_frames ocr-group
                OCR->>OCR: Text recognition
                OCR->>REDIS: XADD ocr_results {trace_id, frame_id, text, region, conf, model_version}
                OCR->>REDIS: XACK preprocessed_frames
            end

        CORR->>REDIS: XREADGROUP detections + ocr_results (join buffer, 60s timeout)
        CORR->>REDIS: XADD inspection_results {trace_id, frame_id, is_anomaly, detection_count, partial}
        PERS->>REDIS: XREADGROUP inspection_results
        PERS->>PGDGX: Batch INSERT (100ms window, up to 1000 rows)
        PERS->>REDIS: XACK inspection_results
    end

    loop Every 5s (background)
        SYNC->>PGDGX: SELECT unsynced rows WHERE session_id=?
        SYNC->>PGVPS: POST /api/sync/batch {batch_id, rows}
        PGVPS->>SYNC: {confirmed_ids: [...]}
        SYNC->>PGDGX: UPDATE synced=TRUE WHERE id IN confirmed_ids
    end

```

6.2 Flow 2 — Session End and Report Generation


```
sequenceDiagram
    participant OP as Operator
    participant CAMSVR as camsvr
    participant SESSMGR as session-manager
    participant PGDGX as PostgreSQL (DGX)
    participant SYNC as sync-engine
    participant PGVPS as PostgreSQL (VPS)
    participant BACKEND as vande Bharat-backend
    participant REDIS_VPS as Redis (VPS)
    participant REPORT as report-generator
    participant MINIO as MinIO / DO Spaces
    participant ARCHIVER as frame-archiver

    OP->>CAMSVR: Click "Stop Session" (or train clears sensors)
    CAMSVR->>SESSMGR: Signal SESSION_END
    SESSMGR->>PGDGX: UPDATE sessions SET status='draining'
    SESSMGR->>SYNC: Trigger final flush

    SYNC->>PGDGX: SELECT all unsynced rows WHERE session_id=?
    SYNC->>PGVPS: POST /api/sync/batch (final, marked is_final=true)
    PGVPS->>SYNC: {confirmed_ids: [...], all_committed: true}
    SYNC->>PGDGX: UPDATE synced=TRUE (confirmed_ids)
    SYNC->>BACKEND: POST /api/sessions/{id}/complete
    BACKEND->>PGVPS: UPDATE sessions SET status='complete'
    BACKEND->>REDIS_VPS: XADD report_jobs {session_id, priority, created_at}
    BACKEND->>SYNC: 200 OK
    SYNC->>PGDGX: UPDATE sessions SET status='synced'

    REPORT->>REDIS_VPS: XREADGROUP report_jobs
    REPORT->>PGVPS: SELECT detections, ocr_results, frames_metadata WHERE session_id=?
    REPORT->>MINIO: GET presigned URLs for anomaly frames (verify checksums)
    REPORT->>REPORT: Generate PDF (signed)
    REPORT->>MINIO: PUT /sessions/{id}/reports/{id}.pdf
    REPORT->>PGVPS: UPDATE reports SET status='complete', url=presigned_url
    REPORT->>REDIS_VPS: XACK report_jobs

    ARCHIVER->>MINIO: Mirror /sessions/{id}/ to DO Spaces (per-object, checksummed)
    ARCHIVER->>ARCHIVER: Verify each object checksum after upload
    ARCHIVER->>PGDGX: Mark session frames as archived

    Note over PGDGX: After archive confirmed, purge raw frames from DGX MinIO
    Note over PGDGX: Keep anomaly frames + report locally for 7 days
```

6.3 Flow 3 — Dashboard Frame Viewing

```
sequenceDiagram
    participant USER as Browser
    participant FRONTEND as vande Bharat-frontend
    participant BACKEND as vande Bharat-backend
    participant PGVPS as PostgreSQL (VPS)
    participant STORAGE as MinIO / DO Spaces

    USER->>FRONTEND: Open session detail page
    FRONTEND->>BACKEND: GET /api/sessions/{id}/frames?page=1&limit=100
    BACKEND->>PGVPS: SELECT minio_path, is_anomaly FROM frames_metadata WHERE session_id=? LIMIT 100
    BACKEND->>STORAGE: Generate presigned URLs (5-minute expiry)
    BACKEND->>FRONTEND: [{frame_id, presigned_url, is_anomaly, seq_num}]
    FRONTEND->>USER: Render frame grid with lazy loading
    USER->>STORAGE: Fetch frame images directly (VPS NOT in path)

    Note over BACKEND,STORAGE: VPS bandwidth: < 10 KB per page request
    Note over USER,STORAGE: Frame data: fetched directly from storage (GBs bypass VPS)
```

6.4 Flow 4 — DGX Offline Recovery

```
sequenceDiagram
    participant DGX as DGX (rebooted)
    participant PGDGX as PostgreSQL (DGX)
    participant SYNC as sync-engine
    participant SESSMGR as session-manager
    participant VPS as VPS Backend
    participant PGVPS as PostgreSQL (VPS)

    DGX->>DGX: Reboot complete, all containers restart
    DGX->>PGDGX: Connect (restored from WAL)
    SESSMGR->>PGDGX: SELECT sessions WHERE status IN ('active','draining','synced'=false)

    alt Interrupted session found
        SESSMGR->>VPS: POST /api/sessions/{id}/interrupted
        VPS->>PGVPS: UPDATE sessions SET status='interrupted'
        Note over SESSMGR: Operator reviews - decides resume or discard
    end

    SYNC->>PGDGX: SELECT all rows WHERE synced=FALSE
    SYNC->>VPS: POST /api/sync/batch (resume from watermark)
    Note over SYNC,VPS: Idempotent - ON CONFLICT DO NOTHING on VPS
    VPS->>SYNC: {confirmed_ids: [ ... ]}
    SYNC->>PGDGX: UPDATE synced=TRUE WHERE id IN confirmed_ids
```

7. Redis Streams Pipeline

7.1 Why Redis Streams (not Kafka, NATS, or RabbitMQ)

Property	Redis Streams	Kafka	NATS JetStream	RabbitMQ
Already in stack	Yes	No — new infra	No	No
Single-machine pipeline	Optimal	Overkill	Reasonable	Reasonable
Latency	Microseconds	Low ms	Microseconds	Low ms
Operational overhead	Near zero	High (JVM, ZK/KRaft)	Low	Medium
Consumer groups (fan-out)	Yes (XREADGROUP)	Yes (consumer groups)	Yes (push consumers)	Yes (exchanges)
At-least-once delivery	Yes (XACK/XPENDING)	Yes (offset commits)	Yes	Yes
Message persistence	Yes (AOF + RDB)	Yes (disk-based)	Yes	Yes (durable queues)
Dead letter queue	Manual (DLQ stream)	Built-in	Built-in	Built-in
Multi-DGX streams	Not designed for it	Excellent	Good	Good
RAM for 50k-msg buffer	~200 MB	N/A	~100 MB	~150 MB

Decision: Redis Streams is correct for Tier 1 (single DGX, single site). Re-evaluate at Tier 3 (10+ DGX nodes requiring cross-machine stream replication). At that point NATS JetStream is the preferred migration target — it has lower operational overhead than Kafka for this use case.

7.2 Stream Topology

Stream	MAXLEN	Purpose	Producer	Consumers
<code>raw_frames</code>	10,000	Capture → pipeline entry	camsvr	preprocessing-group, async-upload-group
<code>preprocessed_frames</code>	5,000	preprocessing → inference	preprocessing	yolo-group, ocr-group
<code>detections</code>	50,000	yolo → correlation	yolo	correlation-group
<code>ocr_results</code>	20,000	ocr → correlation	ocr	correlation-group
<code>inspection_results</code>	5,000	correlation → persistence	correlation	worker-group
<code>raw_frames:dlq</code>	1,000	Failed preprocessing msgs	DLQ handler	monitoring
<code>preprocessed_frames:dlq</code>	1,000	Failed YOLO/OCR msgs	DLQ handler	monitoring

MAXLEN rationale for detections: YOLO produces one stream message per bounding box, not per frame. At 600 FPS with an average of 5 detections per frame, the detections stream receives 3,000 messages/second. MAXLEN 50,000 provides a 16-second buffer — sufficient to absorb correlation processing lag.

7.3 Consumer Group Setup

```
# Run once at service startup - MKSTREAM creates stream if absent
redis-cli XGROUP CREATE raw_frames      preprocessing-group $ MKSTREAM
redis-cli XGROUP CREATE raw_frames      async-upload-group  $ MKSTREAM
redis-cli XGROUP CREATE preprocessed_frames yolo-group      $ MKSTREAM
redis-cli XGROUP CREATE preprocessed_frames ocr-group       $ MKSTREAM
redis-cli XGROUP CREATE detections      correlation-group   $ MKSTREAM
redis-cli XGROUP CREATE ocr_results      correlation-group   $ MKSTREAM
redis-cli XGROUP CREATE inspection_results worker-group      $ MKSTREAM
```

7.4 Message Schemas

Every message carries `trace_id` (assigned at frame capture, never changes) and `frame_id` (unique per frame). All consumers propagate both fields without modification.

```

# camsvr publishes to raw_frames
XADD raw_frames MAXLEN ~ 10000 *
    trace_id    {uuid4}      # assigned once at capture, follows frame forever
    frame_id    {ulid}       # ULID: time-sortable, unique per frame
    session_id  {uuid4}
    cam_id      {0-11}
    seq_num     {monotonic_int} # sequence number per camera per session
    local_path   /data/capture/{session}/{seq:08d}_{cam}.jpg
    ts_hw       {unix_ms}     # hardware/PTP timestamp where available

# preprocessing publishes to preprocessed_frames
XADD preprocessed_frames MAXLEN ~ 5000 *
    trace_id    {forwarded}
    frame_id    {forwarded}
    session_id  {forwarded}
    cam_id      {forwarded}
    seq_num     {forwarded}
    minio_path   /sessions/{id}/{seq:08d}_{cam}.jpg

# yolo publishes to detections (ONE message per bounding box)
XADD detections MAXLEN ~ 50000 *
    trace_id    {forwarded}
    frame_id    {forwarded}
    session_id  {forwarded}
    label       {str}
    confidence   {float}
    bbox        {x1,y1,x2,y2}
    model_version yolo8x-v2.1.0 # mandatory - for future re-inference

# ocr publishes to ocr_results (ONE message per text region)
XADD ocr_results MAXLEN ~ 20000 *
    trace_id    {forwarded}
    frame_id    {forwarded}
    session_id  {forwarded}
    text        {str}
    region       {x1,y1,x2,y2}
    confidence   {float}
    model_version paddleocr-v3.0.0

# correlation publishes to inspection_results (ONE message per frame)
XADD inspection_results MAXLEN ~ 5000 *
    trace_id    {forwarded}
    frame_id    {forwarded}
    session_id  {forwarded}
    is_anomaly   {0|1}
    detection_count {int}
    ocr_text_count {int}
    partial_result {0|1} # 1 if correlation timeout fired before both models returned
    minio_path    {path}

```

7.5 At-Least-Once Delivery and Reclaim

Every consumer implements XAUTOCLAIM on startup to reclaim messages abandoned by crashed consumers. The reclaim threshold is 30 seconds (longer than worst-case GPU restart time).

```

# In each consumer's startup sequence:
def startup_reclaim(redis, stream, group, consumer_name):
    """Reclaim messages idle for > 30 seconds after crash/restart."""
    cursor = "0-0"
    while True:
        next_cursor, messages = redis.xautoclaim(
            stream, group, consumer_name,
            min_idle_time=30_000, # 30 seconds in ms
            start_id=cursor,
            count=100,
        )
        for msg_id, data in messages:
            process_with_retry(msg_id, data)
            redis.xack(stream, group, msg_id)
        if next_cursor == "0-0":
            break
        cursor = next_cursor

```

7.6 Dead Letter Queue

Messages delivered more than 3 times without XACK are poison messages. They must not block the pipeline.

```
MAX_DELIVERY_COUNT = 3

def check_and_move_to_dlq(redis, stream, group):
    """Run every 10 seconds in each consumer."""
    pending = redis.xpending_range(stream, group, min="-", max="+", count=200)
    for entry in pending:
        if entry["times_delivered"] > MAX_DELIVERY_COUNT:
            msg_id = entry["message_id"]
            # Fetch the message data
            messages = redis.xrange(stream, min=msg_id, max=msg_id, count=1)
            if messages:
                _, data = messages[0]
                redis.xadd(
                    f"{stream}:dlq",
                    data | {"error": "max_deliveries_exceeded",
                           "original_stream": stream,
                           "original_id": msg_id},
                )
            redis.xack(stream, group, msg_id)
            log.error(f"Moved {msg_id} from {stream} to DLQ after {MAX_DELIVERY_COUNT} attempts",
                      extra={"trace_id": data.get("trace_id"), "frame_id": data.get("frame_id")})
```

7.7 Correlation Join Buffer

The correlation service merges detections and OCR results by `frame_id`. These arrive from two independent streams at different rates. A join buffer with a configurable timeout handles the asynchrony.

```
import threading
from collections import defaultdict
from datetime import datetime, timedelta

MAX_JOIN_WAIT_SECONDS = 60 # Must exceed worst-case GPU restart time

class CorrelationBuffer:
    def __init__(self, emit_fn, timeout_seconds=MAX_JOIN_WAIT_SECONDS):
        self._buffer = defaultdict(lambda: {
            "detections": [], "ocr": None,
            "received_at": datetime.utcnow(),
            "timeout_at": datetime.utcnow() + timedelta(seconds=timeout_seconds),
        })
        self._lock = threading.Lock()
        self._emit = emit_fn

    def on_detection(self, frame_id, detection):
        with self._lock:
            self._buffer[frame_id]["detections"].append(detection)
            self._maybe_emit(frame_id)

    def on_ocr(self, frame_id, ocr_result):
        with self._lock:
            self._buffer[frame_id]["ocr"] = ocr_result
            self._maybe_emit(frame_id)

    def _maybe_emit(self, frame_id):
        entry = self._buffer[frame_id]
        # Emit complete when both results have arrived
        if entry["detections"] and entry["ocr"] is not None:
            self._emit(frame_id, entry, partial=False)
            del self._buffer[frame_id]

    def flush_timeouts(self):
        """Call every 5 seconds from a background thread."""
        now = datetime.utcnow()
        with self._lock:
            expired = [fid for fid, e in self._buffer.items() if now > e["timeout_at"]]
            for frame_id in expired:
                entry = self._buffer.pop(frame_id)
                self._emit(frame_id, entry, partial=True) # emit with partial_result=1
```

7.8 Redis Configuration (Production)

```
# /etc/redis/redis.conf - mandatory production settings

# Persistence - mandatory for pipeline durability
appendonly yes
appendfsync everysec      # at-most-1-second data loss on crash
auto-aof-rewrite-percentage 100
auto-aof-rewrite-min-size 64mb

# Memory management - prevents OOM kill
maxmemory 8gb              # set to 40% of DGX available RAM
maxmemory-policy allkeys-lru # evict LRU keys when limit hit
# NOTE: with streams, this evicts TRIMMED stream entries first - safe

# RDB snapshots (secondary backup)
save 60 10000              # RDB every 60s if 10000+ changes

# Performance
tcp-backlog 511
hz 20
```

8. Database Architecture

8.1 Two-Database Pattern

Database	Machine	Purpose	Write Pattern	Retention
DGX PostgreSQL	DGX Spark	Hot buffer — active session	High-speed batch (100ms windows)	Cleared after confirmed sync
VPS PostgreSQL	VPS	Master — full history	Low-speed batch sync (5s intervals)	1 year (partitioned)

The DGX database is **not** the system of record — it is a high-throughput write buffer that survives network outages. The VPS database is the system of record. No row is deleted from DGX until VPS confirms it was committed.

8.2 DGX Schema — Hot Buffer

```

-- Use BIGSERIAL for local hot tables: sequential, fast B-tree inserts
-- Use UUIDs only where cross-device identity is needed

CREATE EXTENSION IF NOT EXISTS "pgcrypto";

CREATE TABLE sessions (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  train_id    VARCHAR(100) NOT NULL,
  device_id   VARCHAR(50) NOT NULL DEFAULT 'dgx-spark', -- multi-device ready
  station_id  VARCHAR(50) NOT NULL DEFAULT 'station-01',
  started_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  ended_at    TIMESTAMPTZ,
  status      VARCHAR(20) NOT NULL DEFAULT 'active'
              CHECK (status IN ('active', 'draining', 'processing', 'synced', 'interrupted')),
  synced_at   TIMESTAMPTZ,
  frame_count INT          DEFAULT 0,
  anomaly_count INT        DEFAULT 0
);

CREATE TABLE frames_metadata (
  id          BIGSERIAL PRIMARY KEY, -- fast local inserts
  global_id   UUID       NOT NULL DEFAULT gen_random_uuid(), -- stable cross-device ID
  trace_id    UUID       NOT NULL, -- correlation ID from camsvr
  session_id  UUID       NOT NULL REFERENCES sessions(id),
  camera_id   VARCHAR(20) NOT NULL,
  seq_num     BIGINT     NOT NULL, -- monotonic per camera per session
  captured_at TIMESTAMPTZ NOT NULL, -- hardware timestamp where available
  minio_path  TEXT,
  is_anomaly  BOOLEAN     NOT NULL DEFAULT FALSE,
  synced      BOOLEAN     NOT NULL DEFAULT FALSE,
  synced_at   TIMESTAMPTZ
);

CREATE TABLE detections (
  id          BIGSERIAL PRIMARY KEY,
  global_id   UUID       NOT NULL DEFAULT gen_random_uuid(),
  trace_id    UUID       NOT NULL,
  frame_global_id UUID NOT NULL, -- references frames_metadata.global_id
  session_id  UUID       NOT NULL,
  label       VARCHAR(100) NOT NULL,
  confidence  FLOAT      NOT NULL CHECK (confidence ≥ 0.0 AND confidence ≤ 1.0),
  bbox_x1     INT, bbox_y1 INT, bbox_x2 INT, bbox_y2 INT,
  model_version VARCHAR(50) NOT NULL, -- e.g. 'yolov8x-v2.1.0'
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  synced      BOOLEAN     NOT NULL DEFAULT FALSE
);

CREATE TABLE ocr_results (
  id          BIGSERIAL PRIMARY KEY,
  global_id   UUID       NOT NULL DEFAULT gen_random_uuid(),
  trace_id    UUID       NOT NULL,
  frame_global_id UUID NOT NULL,
  session_id  UUID       NOT NULL,
  extracted_text TEXT,
  region_x1   INT, region_y1 INT, region_x2 INT, region_y2 INT,
  confidence  FLOAT      CHECK (confidence ≥ 0.0 AND confidence ≤ 1.0),
  model_version VARCHAR(50) NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  synced      BOOLEAN     NOT NULL DEFAULT FALSE
);

-- Indexes - mandatory before any data is written
CREATE INDEX idx_frames_session ON frames_metadata(session_id);
CREATE INDEX idx_frames_trace ON frames_metadata(trace_id);
CREATE INDEX idx_frames_unsynced ON frames_metadata(session_id) WHERE synced = FALSE;
CREATE INDEX idx_detections_session ON detections(session_id);
CREATE INDEX idx_detections_frame ON detections(frame_global_id);
CREATE INDEX idx_detections_unsynced ON detections(session_id) WHERE synced = FALSE;
CREATE INDEX idx_ocr_session ON ocr_results(session_id);
CREATE INDEX idx_ocr_unsynced ON ocr_results(session_id) WHERE synced = FALSE;

-- PostgreSQL configuration for DGX hot buffer
-- In postgresql.conf:
-- synchronous_commit = off (DGX is a buffer; async commit doubles write throughput)
-- wal_level = replica (enables WAL archiving for PITR)
-- checkpoint_completion_target = 0.9
-- max_wal_size = 4GB

```

8.3 VPS Schema — Master History


```

-- Partitioned by captured_at for scalable archival and query performance

CREATE TABLE sessions (
    id                UUID                PRIMARY KEY,
    train_id          VARCHAR(100) NOT NULL,
    device_id         VARCHAR(50)  NOT NULL,
    station_id        VARCHAR(50)  NOT NULL,
    started_at        TIMESTAMPTZ NOT NULL,
    ended_at          TIMESTAMPTZ,
    status            VARCHAR(20)  NOT NULL
                        CHECK (status IN ('active', 'complete', 'interrupted', 'archived')),
    frame_count       INT,
    anomaly_count     INT,
    synced_at         TIMESTAMPTZ,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Partitioned by month - mandatory at scale
CREATE TABLE frames_metadata (
    global_id         UUID                NOT NULL,
    trace_id          UUID                NOT NULL,
    session_id        UUID                NOT NULL,
    device_id         VARCHAR(50)  NOT NULL,
    station_id        VARCHAR(50)  NOT NULL,
    camera_id         VARCHAR(20)  NOT NULL,
    seq_num           BIGINT            NOT NULL,
    captured_at       TIMESTAMPTZ NOT NULL,
    minio_path        TEXT,
    is_anomaly        BOOLEAN          NOT NULL DEFAULT FALSE
) PARTITION BY RANGE (captured_at);

CREATE TABLE detections (
    global_id         UUID                NOT NULL,
    trace_id          UUID                NOT NULL,
    frame_global_id   UUID                NOT NULL,
    session_id        UUID                NOT NULL,
    device_id         VARCHAR(50)  NOT NULL,
    label            VARCHAR(100) NOT NULL,
    confidence        FLOAT            NOT NULL,
    bbox_x1 INT, bbox_y1 INT, bbox_x2 INT, bbox_y2 INT,
    model_version     VARCHAR(50)  NOT NULL,
    created_at        TIMESTAMPTZ NOT NULL
) PARTITION BY RANGE (created_at);

CREATE TABLE ocr_results (
    global_id         UUID                NOT NULL,
    trace_id          UUID                NOT NULL,
    frame_global_id   UUID                NOT NULL,
    session_id        UUID                NOT NULL,
    device_id         VARCHAR(50)  NOT NULL,
    extracted_text     TEXT,
    region_x1 INT, region_y1 INT, region_x2 INT, region_y2 INT,
    confidence        FLOAT,
    model_version     VARCHAR(50)  NOT NULL,
    created_at        TIMESTAMPTZ NOT NULL
) PARTITION BY RANGE (created_at);

-- pg_partman manages monthly partitions automatically
-- SELECT partman.create_parent('public.frames_metadata', 'captured_at', 'native', 'monthly');

CREATE TABLE reports (
    id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id        UUID                NOT NULL REFERENCES sessions(id),
    generated_at       TIMESTAMPTZ,
    storage_path       TEXT,
    pdf_checksum       VARCHAR(64),          -- SHA-256 for integrity verification
    status            VARCHAR(20)  NOT NULL DEFAULT 'pending'
                        CHECK (status IN ('pending', 'generating', 'complete', 'failed')),
    failure_reason     TEXT
);

CREATE TABLE sync_log (
    id                BIGSERIAL          PRIMARY KEY,
    batch_id          UUID                NOT NULL UNIQUE,    -- idempotency key
    session_id        UUID                NOT NULL,
    source_device      VARCHAR(50)  NOT NULL,
    row_count         INT                NOT NULL,
    sequence_range     NUMRANGE,
    received_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    confirmed_ids      UUID[]            NOT NULL
);

```

```

);

CREATE TABLE devices (
  id          VARCHAR(50) PRIMARY KEY,
  name        VARCHAR(100) NOT NULL,
  tailscale_ip VARCHAR(50),
  type        VARCHAR(50) NOT NULL CHECK (type IN ('dgx', 'agx', 'jetson')),
  station_id  VARCHAR(50),
  last_heartbeat TIMESTAMPTZ,
  status      VARCHAR(20) NOT NULL DEFAULT 'offline'
              CHECK (status IN ('online', 'offline', 'degraded'))
);

CREATE TABLE audit_log (
  id          BIGSERIAL PRIMARY KEY,
  timestamp   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  user_id     VARCHAR(100),
  action      VARCHAR(200) NOT NULL,
  resource_type VARCHAR(100),
  resource_id  VARCHAR(200),
  result      VARCHAR(20) NOT NULL CHECK (result IN ('success', 'failure')),
  source_ip   INET,
  request_id  UUID
);

-- VPS Indexes
CREATE INDEX idx_vps_frames_session ON frames_metadata(session_id);
CREATE INDEX idx_vps_frames_trace ON frames_metadata(trace_id);
CREATE INDEX idx_vps_frames_anomaly ON frames_metadata(session_id) WHERE is_anomaly = TRUE;
CREATE INDEX idx_vps_det_session ON detections(session_id);
CREATE INDEX idx_vps_det_frame ON detections(frame_global_id);
CREATE INDEX idx_vps_det_label ON detections(label, model_version);
CREATE INDEX idx_vps_sessions_status ON sessions(status, started_at);
CREATE INDEX idx_vps_sessions_train ON sessions(train_id, started_at);
CREATE INDEX idx_vps_sessions_device ON sessions(device_id, station_id);
CREATE INDEX idx_sync_log_batch ON sync_log(batch_id);
CREATE INDEX idx_sync_log_session ON sync_log(session_id);

```

9. Storage Architecture

9.1 Object Naming Convention

Object keys use monotonic sequence numbers, not timestamps. Timestamps are metadata, not identifiers. This eliminates key collision at 50 FPS and enables byte-sortable ordering.

Bucket: vande Bharat

/sessions/{session_id}/raw/{cam_id}/{seq_num:08d}.jpg
 e.g. /sessions/abc123/raw/cam_03/00047832.jpg

/sessions/{session_id}/frames/{cam_id}/{seq_num:08d}.jpg
 e.g. /sessions/abc123/frames/cam_03/00047832.jpg

/sessions/{session_id}/anomalies/{cam_id}/{seq_num:08d}.jpg
 e.g. /sessions/abc123/anomalies/cam_07/00012041.jpg

/sessions/{session_id}/reports/{session_id}.pdf
 e.g. /sessions/abc123/reports/abc123.pdf

9.2 Write Path (Production)

```

CAPTURE (camsvr)
→ Write: /data/capture/{session}/{seq:08d}_{cam}.jpg (local NVMe - <1ms)
→ XADD raw_frames {local_path, ...} (Redis - microseconds)
- does NOT write to MinIO directly -

ASYNC UPLOADER (separate service, reads raw_frames stream)
→ XREADGROUP raw_frames async-upload-group
→ Read local file
→ Compute MD5 checksum
→ PUT to MinIO with x-amz-meta-checksum header
→ Verify checksum after write
→ Delete local file after confirmed MinIO write
→ XACK raw_frames

RESULT: camsvr is never blocked by storage I/O
MinIO unavailability does not cause frame drops
Local disk is the safety buffer

```

9.3 Checksum Verification

```

import hashlib
from io import BytesIO

def upload_frame_with_checksum(minio_client, bucket, object_name, local_path):
    with open(local_path, "rb") as f:
        data = f.read()

    checksum = hashlib.md5(data).hexdigest()

    minio_client.put_object(
        bucket, object_name,
        data=BytesIO(data),
        length=len(data),
        metadata={"x-amz-meta-md5": checksum},
        content_type="image/jpeg",
    )

    # Verify immediately after write
    obj = minio_client.stat_object(bucket, object_name)
    stored_checksum = obj.metadata.get("x-amz-meta-md5")
    if stored_checksum != checksum:
        minio_client.remove_object(bucket, object_name)
        raise StorageIntegrityError(f"Checksum mismatch for {object_name}: expected {checksum}, got {stored_checksum}")

    return checksum

def fetch_frame_with_verification(minio_client, bucket, object_name):
    obj = minio_client.get_object(bucket, object_name)
    data = obj.read()
    stored_checksum = obj.headers.get("x-amz-meta-md5")
    if stored_checksum:
        actual = hashlib.md5(data).hexdigest()
        if actual != stored_checksum:
            raise StorageIntegrityError(f"Corruption detected in {object_name}")
    return data

```

9.4 Bucket Policy (Production)

Reports use presigned URLs (not public). Raw frames and preprocessed frames use presigned URLs. No object in the bucket is publicly accessible.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Deny",
      "Principal": {"AWS": ["*"]},
      "Action": ["s3:*"],
      "Resource": ["arn:aws:s3:::vande Bharat/*"],
      "Condition": {
        "StringNotEquals": {
          "s3:authType": "REST-QUERY-STRING"
        }
      }
    }
  ]
}
```

All access is via presigned URLs with 5-minute expiry. The frontend requests fresh presigned URLs when navigating to a new page of frames.

9.5 DO Spaces Lifecycle Rules

```
{
  "Rules": [
    {
      "ID": "expire-raw-frames-90-days",
      "Filter": {"Prefix": "sessions/"},
      "Status": "Enabled",
      "Expiration": {"Days": 90},
      "Filter": {"Suffix": "/raw/"}
    },
    {
      "ID": "transition-session-data-1-year",
      "Filter": {"Prefix": "sessions/"},
      "Status": "Enabled",
      "Transitions": [{"Days": 365, "StorageClass": "GLACIER"}]
    }
  ]
}
```

9.6 Environment Configuration

```
# Phase 1 - MinIO on DGX (development / early production)
S3_ENDPOINT=http://100.76.12.44:9010
S3_ACCESS_KEY=${MINIO_SERVICE_KEY}      # Per-service key, not root
S3_SECRET_KEY=${MINIO_SERVICE_SECRET}
S3_BUCKET=vande Bharat
S3_REGION=us-east-1                      # MinIO default

# Phase 2 - DO Spaces (production)
S3_ENDPOINT=https://blr1.digitaloceanspaces.com
S3_ACCESS_KEY=${DO_SPACES_KEY}
S3_SECRET_KEY=${DO_SPACES_SECRET}
S3_BUCKET=vande Bharat
S3_REGION=blr1

# Zero code changes - same boto3/minio-py calls
```

10. Synchronization Engine

10.1 Design Rationale

The sync-engine is the most critical service in MVIS after camsvr. It is the only mechanism transferring inspection results from the DGX to the VPS. A naive polling loop is insufficient: it has no state machine, no idempotency, no confirmed-delete pattern, and no recovery path after partial failures.

The production sync-engine implements a formal state machine, idempotent batch delivery, and a confirmed-delete protocol that guarantees no data is lost from DGX until VPS has durably committed it.

10.2 State Machine

```
stateDiagram-v2
    [*] --> IDLE : Service start
    IDLE --> SYNCING : Session START signal received
    SYNCING --> SYNCING : 5s timer (batch available)
    SYNCING --> FLUSHING : Session END signal received
    FLUSHING --> CONFIRMING : All rows sent, VPS 200 OK
    CONFIRMING --> MIRRORING : VPS confirms all rows committed
    MIRRORING --> DONE : frame-archiver completes
    DONE --> IDLE : DGX cleanup complete
    SYNCING --> RETRYING : Network failure
    CONFIRMING --> RETRYING : Network failure
    RETRYING --> SYNCING : Network recovered
    RETRYING --> DEGRADED : Max retries exceeded (15 min)
    DEGRADED --> RETRYING : Operator reset / network recovered
    DEGRADED --> ERROR : Disk full or data corruption detected
    ERROR --> [*] : Manual operator intervention required
```

10.3 Batch Algorithm

```
class SyncEngine:
    def sync_batch(self, session_id: str) → SyncResult:
        # 1. Read unsynced rows ordered by sequence number
        rows = db.execute("""
            SELECT global_id, 'frames_metadata' AS table_name,
                   row_to_json(fm.*) AS data, seq_num AS sequence_num
            FROM frames_metadata fm
            WHERE session_id = %s AND synced = FALSE
            ORDER BY seq_num ASC LIMIT 1000
            UNION ALL
            SELECT global_id, 'detections', row_to_json(d.*), created_at::text::float
            FROM detections d
            WHERE session_id = %s AND synced = FALSE
            ORDER BY 4 ASC LIMIT 5000
        """, (session_id, session_id))

        if not rows:
            return SyncResult.NO_ROWS

        batch = {
            "batch_id": str(uuid4()),          # idempotency key
            "source_device": self.device_id,
            "session_id": session_id,
            "rows": [{ "id": r.global_id, "table": r.table_name, "data": r.data } for r in rows],
            "is_final": self._state == State.FLUSHING and not self._has_more(session_id),
        }

        for attempt in range(6):
            try:
                resp = self._vps_client.post("/api/sync/batch", json=batch, timeout=15)
                if resp.status_code == 200:
                    confirmed_ids = resp.json()["confirmed_ids"]
                    # Delete ONLY rows that VPS confirmed - never delete optimistically
                    db.execute(
                        "UPDATE frames_metadata SET synced=TRUE WHERE global_id = ANY(%s)",
                        (confirmed_ids,)
                    )
                    return SyncResult.SUCCESS
            except (NetworkError, Timeout) as e:
                wait = min(5 * (2 ** attempt), 300) # 5, 10, 20, 40, 80, 300s
                log.warning(f"Sync attempt {attempt+1} failed: {e}. Retrying in {wait}s",
                           extra={"session_id": session_id, "batch_id": batch["batch_id"]})
                time.sleep(wait)

        alert("sync_engine_batch_failed", session_id=session_id, attempts=6)
        return SyncResult.NETWORK_UNAVAILABLE
```

10.4 VPS Receiver Endpoint

```

@router.post("/api/sync/batch")
async def receive_sync_batch(batch: SyncBatch, db: AsyncSession,
                             x_client_cert: str = Header(None)):
    # mTLS: verify client certificate belongs to a registered DGX device
    device = verify_client_cert_and_get_device(x_client_cert)
    if not device:
        raise HTTPException(403, "Unrecognized device certificate")

    # Idempotency: if batch_id already processed, return same response
    existing = await db.scalar(
        select(SyncLog).where(SyncLog.batch_id == batch.batch_id)
    )
    if existing:
        return {"confirmed_ids": existing.confirmed_ids, "status": "already_processed"}

    confirmed_ids = []
    async with db.begin():
        for row in batch.rows:
            table = ALLOWED_SYNC_TABLES.get(row.table)
            if not table:
                continue
            stmt = insert(table).values(**row.data).on_conflict_do_nothing(index_elements=["global_id"])
            await db.execute(stmt)
            confirmed_ids.append(row.id)

        await db.execute(
            insert(SyncLog).values(
                batch_id=batch.batch_id,
                session_id=batch.session_id,
                source_device=device.id,
                row_count=len(batch.rows),
                confirmed_ids=confirmed_ids,
            )
        )

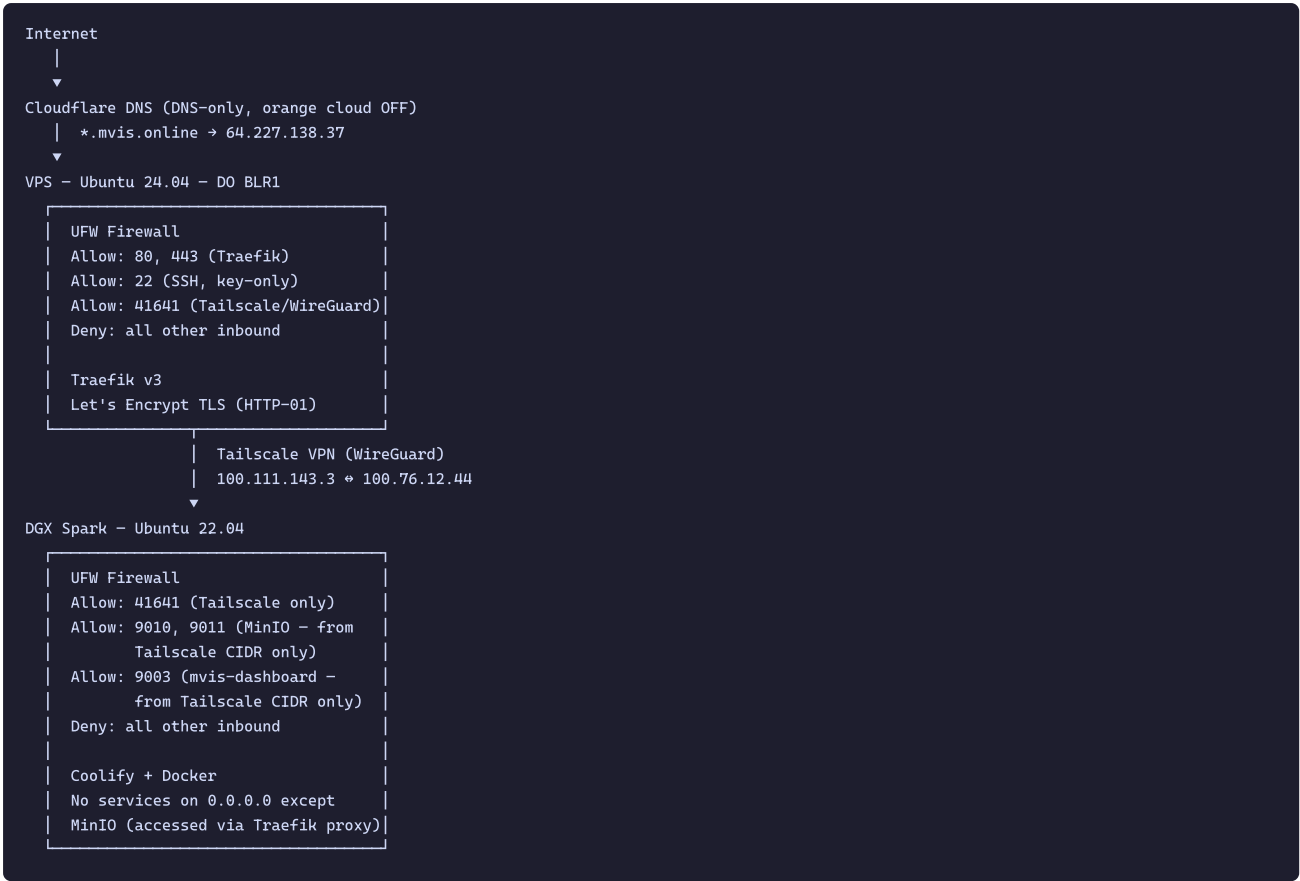
    # Audit log
    await write_audit_log(db, action="sync.batch.received", resource_id=batch.session_id,
                          result="success", source_device=device.id)

    return {"confirmed_ids": confirmed_ids, "status": "committed"}

```

11. Network & Security Architecture

11.1 Network Topology



11.2 TLS Strategy

Path	TLS	Certificate
Browser → VPS	TLS 1.3	Let's Encrypt via Traefik
VPS → DGX (Tailscale)	WireGuard encrypted	Tailscale MK
sync-engine → VPS backend	mTLS	Internal CA (device cert)
async-uploader → MinIO	TLS	Let's Encrypt (minio.mvis.online)
frame-archiver → DO Spaces	TLS 1.3	DO managed cert

11.3 Authentication & RBAC

Roles:

- OPERATOR → start/stop sessions, view live camera feed
- INSPECTOR → view completed reports, download PDFs
- ADMIN → configure devices, manage routes, view all data
- AUDITOR → read-only access to all historical data and audit logs
- SERVICE → machine-to-machine (sync-engine, report-generator)

JWT Configuration:

- Algorithm: RS256 (asymmetric - private key on auth server only)
- Access token: 15-minute expiry
- Refresh token: 7-day expiry, rotated on use
- Issuer: https://api.mvis.online
- Audience: mvis-platform

11.4 Per-Service MinIO Credentials

```
# Root credentials: stored in Coolify secrets, never used by application services
MINIO_ROOT_USER=admin
MINIO_ROOT_PASSWORD=<strong-root-password>

# Per-service access keys (least privilege):
mc admin user add dgx camsvr-svc ${CAMSVR_ACCESS_KEY}
mc admin policy create dgx camsvr-write-policy - << 'EOF'
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:PutObject"],
    "Resource": ["arn:aws:s3:::vande Bharat/sessions/*/raw/*",
                 "arn:aws:s3:::vande Bharat/sessions/*/frames/*"]
  }]
}
EOF
mc admin policy attach dgx camsvr-write-policy --user camsvr-svc

mc admin user add dgx report-gen-svc ${REPORT_GEN_KEY}
mc admin policy create dgx report-gen-policy - << 'EOF'
{
  "Version": "2012-10-17",
  "Statement": [
    {"Effect": "Allow", "Action": ["s3:GetObject"],
      "Resource": ["arn:aws:s3:::vande Bharat/sessions/*/anomalies/*"]},
    {"Effect": "Allow", "Action": ["s3:PutObject"],
      "Resource": ["arn:aws:s3:::vande Bharat/sessions/*/reports/*"]}
  ]
}
EOF
mc admin policy attach dgx report-gen-policy --user report-gen-svc
```

11.5 Secrets Management

Tool: Doppler (or equivalent secrets manager)
Syncs to Coolify (DGX) and Dokploy (VPS) environment injection

Rotation:

JWT signing keys: 90-day rotation
MinIO service keys: 180-day rotation
DB passwords: 90-day rotation
Sync service token: 30-day rotation

Storage rules:

- Secrets never in git (enforced by .gitignore + pre-commit hook)
- Secrets never in Docker image layers
- Secrets mounted as tmpfs volumes where possible
- Never logged - structured logger must mask fields containing 'key', 'secret', 'password', 'token'

11.6 Audit Logging

Every state-modifying API call is written to the `audit_log` table:

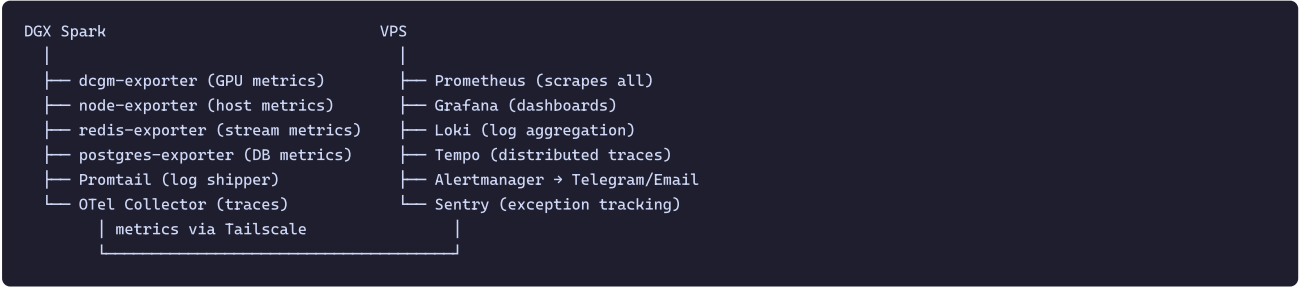
```
AUDITED_ACTIONS = [
  "session.start", "session.stop", "session.delete",
  "report.generate", "report.download",
  "device.register", "device.deregister",
  "user.login", "user.logout", "user.role_change",
  "sync.batch.received", "sync.session.complete",
  "route.add", "route.remove",
]
```

11.7 Public Subdomain Registry

Subdomain	Target	Auth	Notes
cam.mvis.online	DGX:9003 (via Traefik proxy)	App login (OPERATOR role)	Session UI
frontend.mvis.online	VPS backend	JWT (INSPECTOR+)	Results dashboard
api.mvis.online	VPS backend	JWT + RBAC	REST API
grafana.mvis.online	VPS Grafana	Grafana login	Monitoring
minio.mvis.online	DGX MinIO	Presigned URLs / mTLS	Frame serving
minioconsole.mvis.online	DGX MinIO console	MinIO login (ADMIN only)	Admin only
coolify.mvis.online	DGX Coolify	Coolify login (ADMIN only)	Deployment
routes.mvis.online	VPS DomainManager	App login (ADMIN only)	Route management
deploy.mvis.online	VPS Dokploy	Dokploy login (ADMIN only)	Deployment

12. Observability Architecture

12.1 Stack Overview



12.2 Metrics: What Each Exporter Collects

GPU — dcgm-exporter

dcgm_gpu_utilization	# % GPU compute used – alert < 50% during session
dcgm_fb_used_mb	# GPU memory used – alert > 90% of total
dcgm_fb_total_mb	# GPU memory total
dcgm_gpu_temp	# GPU temperature – alert > 85°C
dcgm_pcie_tx_bytes_total	# PCIe TX throughput (frame data)
dcgm_pcie_rx_bytes_total	# PCIe RX throughput
dcgm_sm_clock_mhz	# SM clock – low value indicates throttling
dcgm_power_usage_watts	# GPU power draw – alert if throttling observed

Host — node-exporter

node_cpu_seconds_total	# CPU usage per core
node_memory_MemAvailable_bytes	# Available RAM – alert < 10%
node_filesystem_avail_bytes{mountpoint="/data"}	# Data disk space – alert < 15%
node_filesystem_avail_bytes{mountpoint="/"}	# OS disk space – alert < 20%
node_network_transmit_bytes_total{device="tailscale0"}	# Tailscale TX (sync bandwidth)
node_disk_io_time_seconds_total	# Disk I/O saturation
node_load1	# 1-minute load average

Redis — redis-exporter

```

redis_stream_length{stream="raw_frames"}      # alert > 8000 (80% of MAXLEN)
redis_stream_length{stream="detections"}      # alert > 40000
redis_stream_pending_entries_count            # PEL size - alert > 500 per stream
redis_memory_used_bytes                       # alert > 80% of maxmemory
redis_memory_max_bytes                       # configured maxmemory
redis_connected_clients                      # alert == 0
redis_rdb_last_save_time                     # alert > 600s (10 min)
redis_aof_enabled                            # alert == 0 (AOF disabled)
redis_commands_total{cmd="xadd"}              # publish rate
redis_commands_total{cmd="xreadgroup"}        # consume rate

```

PostgreSQL — postgres-exporter

```

pg_up                                         # alert == 0
pg_stat_activity_count{state="active"}      # active queries
pg_stat_bgwriter_checkpoint_req_total        # checkpoint frequency
pg_database_size_bytes                      # alert > 80% of disk
pg_stat_replication_pg_wal_lsn_diff          # replication lag (VPS master)
pg_locks_count                              # alert > 50 (lock contention)
pg_stat_user_tables_n_dead_tup{relname="frames_metadata"} # alert if VACUUM needed

```

Pipeline Services — Custom Metrics

Each pipeline service (preprocessing, yolo, ocr, correlation, result-persister) exposes:

```

from prometheus_client import Counter, Histogram, Gauge, start_http_server

# Expose on GET /metrics (port 9100+service_offset)
frames_received_total = Counter(
    "mvis_frames_received_total",
    "Frames read from input stream",
    ["service", "stream"]
)
frames_processed_total = Counter(
    "mvis_frames_processed_total",
    "Frames successfully processed",
    ["service"]
)
frames_failed_total = Counter(
    "mvis_frames_failed_total",
    "Frames that failed processing",
    ["service", "reason"]
)
frames_dlq_total = Counter(
    "mvis_frames_dlq_total",
    "Frames moved to dead letter queue",
    ["service", "stream"]
)
processing_latency_seconds = Histogram(
    "mvis_processing_latency_seconds",
    "Time to process one frame",
    ["service"],
    buckets=[0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1.0, 2.5]
)
stream_lag_frames = Gauge(
    "mvis_stream_consumer_lag_frames",
    "Estimated frames behind in stream",
    ["service", "stream"]
)
correlation_buffer_size = Gauge(
    "mvis_correlation_buffer_size",
    "Current frames in correlation join buffer",
    []
)
correlation_timeouts_total = Counter(
    "mvis_correlation_timeouts_total",
    "Frames emitted after join timeout (partial results)",
    []
)
sync_batches_sent_total = Counter(
    "mvis_sync_batches_sent_total", "Sync batches sent to VPS", ["status"]
)
sync_rows_pending = Gauge(
    "mvis_sync_rows_pending", "Rows in DGX DB not yet synced", ["table"]
)

```

12.3 Alerting Rules

```
# alertmanager/rules/mvis.yml

groups:
- name: mvis.pipeline
  rules:
    - alert: PipelineStalled
      expr: rate(mvis_frames_processed_total{service="yolo"}[2m]) == 0
      for: 30s
      labels:
        severity: critical
      annotations:
        summary: "YOLO inference pipeline has stalled"
        description: "No frames processed by YOLO for 30 seconds during active session"

    - alert: PipelineHighLatency
      expr: histogram_quantile(0.99, mvis_processing_latency_seconds{service="yolo"}) > 0.5
      for: 60s
      labels:
        severity: warning
      annotations:
        summary: "YOLO P99 latency exceeds 500ms"

    - alert: StreamApproachingMaxLen
      expr: redis_stream_length / redis_stream_max_length > 0.8
      for: 10s
      labels:
        severity: critical
      annotations:
        summary: "Redis stream {{ $labels.stream }} at {{ $value | humanizePercentage }} capacity"

    - alert: DeadLetterQueueGrowing
      expr: rate(mvis_frames_dlq_total[5m]) > 0
      for: 30s
      labels:
        severity: warning
      annotations:
        summary: "Dead letter queue receiving messages - poison frames detected"

    - alert: CorrelationTimeoutRising
      expr: rate(mvis_correlation_timeouts_total[5m]) > 1
      for: 60s
      labels:
        severity: warning
      annotations:
        summary: "Correlation timeouts: one or both inference models may be degraded"

- name: mvis.infrastructure
  rules:
    - alert: DGXDiskCritical
      expr: node_filesystem_avail_bytes{mountpoint="/data", job="dgx-node"} /
        node_filesystem_size_bytes{mountpoint="/data", job="dgx-node"} < 0.10
      for: 1m
      labels:
        severity: critical
      annotations:
        summary: "DGX /data disk below 10% - frame capture at risk"

    - alert: GPUMemoryCritical
      expr: dcmg_fb_used_mb / dcmg_fb_total_mb > 0.90
      for: 30s
      labels:
        severity: critical
      annotations:
        summary: "GPU memory above 90% - OOM risk"

    - alert: GPUTemperatureHigh
      expr: dcmg_gpu_temp > 85
      for: 60s
      labels:
        severity: warning
      annotations:
        summary: "GPU temperature {{ $value }}°C - check cooling"

    - alert: RedisAOFDisabled
      expr: redis_aof_enabled == 0
      for: 0s
      labels:
        severity: critical
      annotations:
        summary: "Redis AOF persistence is disabled - pipeline data is not durable"
```

```

- name: mvis.sync
  rules:
    - alert: SyncEngineFailing
      expr: rate(mvis_sync_batches_sent_total{status="failure"}[5m]) > 0
      for: 2m
      labels:
        severity: warning
      annotations:
        summary: "Sync engine batch failures detected"

    - alert: SyncBacklogCritical
      expr: mvis_sync_rows_pending > 100000
      for: 5m
      labels:
        severity: critical
      annotations:
        summary: "{{ $value }}" rows pending sync - DGX DB may overflow"

    - alert: SessionHeartbeatMissed
      expr: time() - mvis_session_last_heartbeat_timestamp > 120
      for: 0s
      labels:
        severity: critical
      annotations:
        summary: "Session heartbeat not received for > 2 minutes - DGX may be offline"

```

12.4 Grafana Dashboards

Dashboard	Panels
Pipeline Overview	Frame throughput, per-stage latency, stream lengths, DLQ rate
GPU Health	Utilization, memory, temperature, clock speed, PCIe throughput
Session Monitor	Active sessions, frame count, anomaly count, sync lag
Infrastructure	CPU, RAM, disk, network per machine
Sync Status	Pending rows, batch success rate, last sync timestamp
Alerts	Current firing alerts, alert history

12.5 Structured Logging

Every log line is JSON with a mandatory field set:

```

{
  "timestamp": "2026-08-02T14:23:01.234Z",
  "level": "INFO",
  "service": "vande Bharat-yolo",
  "version": "2.1.0",
  "trace_id": "8f3a2b1c-4d7e-9f2a-b3c6-1e5d8f2a4b7c",
  "session_id": "a1b2c3d4- ... ",
  "frame_id": "01JA4XYZABC ... ",
  "camera_id": "cam_03",
  "seq_num": 47832,
  "event": "frame_processed",
  "duration_ms": 18.4,
  "detection_count": 3,
  "model_version": "yolov8x-v2.1.0",
  "gpu_memory_mb": 4200
}

```

Log aggregation:

DGX services → Promtail → Loki (VPS) → Grafana
 VPS services → Promtail → Loki (VPS) → Grafana

Retention:

CRITICAL / ERROR: 1 year
 WARNING: 90 days
 INFO: 30 days
 DEBUG: 7 days (off in production, toggled on demand)

12.6 Distributed Tracing

OpenTelemetry SDK is embedded in every service. The `trace_id` assigned at camsvr becomes the OTel trace ID, enabling a single Tempo query to show the full journey of any frame from capture through report inclusion.

```
from opentelemetry import trace
from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import OTLPSpanExporter
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor

provider = TracerProvider()
provider.add_span_processor(
    BatchSpanProcessor(OTLPSpanExporter(endpoint="http://100.111.143.3:4317"))
)
trace.set_tracer_provider(provider)

tracer = trace.get_tracer("vande Bharat-yolo")

# In the frame processing loop:
with tracer.start_as_current_span("yolo.infer",
    context=propagate.extract({"traceparent": f"00-{trace_id}-..."})) as span:
    span.set_attribute("frame.id", frame_id)
    span.set_attribute("session.id", session_id)
    result = model.infer(frame)
    span.set_attribute("detection.count", len(result))
```

13. Deployment Strategy

13.1 Service Health Checks

Every service exposes standardized health endpoints:

```
@app.get("/healthz")
async def liveness():
    """Returns 200 if process is alive. Fail only on unrecoverable state."""
    return {"status": "ok", "service": SERVICE_NAME, "version": VERSION}

@app.get("/readyz")
async def readiness():
    """Returns 200 if service is ready to process traffic."""
    checks = {
        "redis": await check_redis_connection(),
        "postgres": await check_postgres_connection(),
        "gpu": check_gpu_available(),
    }
    if all(checks.values()):
        return {"status": "ready", "checks": checks}
    raise HTTPException(503, detail={"status": "not_ready", "checks": checks})

@app.get("/metrics")
async def metrics():
    return Response(generate_latest(), media_type=CONTENT_TYPE_LATEST)
```

Docker health check (applied to every service):

```
healthcheck:
  test: ["CMD", "curl", "-sf", "http://localhost:${PORT}/healthz"]
  interval: 10s
  timeout: 5s
  retries: 3
  start_period: 30s
restart: unless-stopped
```

13.2 Deployment Flow (Blue-Green)

- 1. Build new image → tag as :v{version}
- 2. Push to container registry
- 3. Deploy new container (green) alongside existing (blue)
- 4. Wait for /readyz to return 200 on green
- 5. Update Traefik routing to point to green
- 6. Monitor error rate for 5 minutes
- 7. If error rate elevated → revert Traefik to blue immediately
- 8. If stable → stop and remove blue container

13.3 Rollback Procedure

```
# Immediate rollback - switch Traefik back to blue
# In Coolify: redeploy previous version (stored in deployment history)

# For DGX services (Coolify):
coolify redeploy --service vandebharat-yolo --version v2.0.1

# For VPS services (Dokploy):
# Dokploy maintains deployment history - click "Rollback" in UI
# Or via API:
curl -X POST https://deploy.mvis.online/api/deploy/rollback \
-H "Authorization: Bearer $DOKPLOY_TOKEN" \
-d '{"appId": "vandebharat-backend", "deploymentId": "previous"}'
```

13.4 Environment Separation

Environment	DGX	VPS	Storage	Purpose
development	Local machine	Docker Compose	Local MinIO	Feature development
staging	DGX (separate Coolify project)	VPS (separate Dokploy project)	MinIO (staging bucket)	Integration testing
production	DGX (production Coolify)	VPS (production Dokploy)	MinIO → DO Spaces	Live operations

13.5 Secret Injection

```
# Coolify (DGX): set via UI or API - injected as container env at deploy time
# Dokploy (VPS): set via UI or API - injected as container env at deploy time
# Neither platform stores secrets in the image or in git

# Pre-deploy validation - check all required secrets are set:
REQUIRED_SECRETS=(
  S3_ACCESS_KEY S3_SECRET_KEY S3_ENDPOINT S3_BUCKET
  POSTGRES_URL VPS_POSTGRES_URL SYNC_SERVICE_TOKEN
  REDIS_URL JWT_PRIVATE_KEY JWT_PUBLIC_KEY
)
for secret in "${REQUIRED_SECRETS[@]}; do
  if [ -z "${!secret}" ]; then
    echo "ERROR: Required secret $secret is not set"
    exit 1
  fi
done
```

14. Implementation Phases

Phase A — Move Presentation Layer to VPS

Goal: `vandebharat-backend`, `vandebharat-frontend`, `report-generator`, `grafana`, `prometheus` running on VPS. DGX services unchanged.

1. Create VPS PostgreSQL via Dokploy
2. Run DB schema migration (`v2.0.0`) on VPS PostgreSQL — includes all indexes and partitioning
3. Deploy `vandebharat-backend` on VPS — `DATABASE_URL` → DGX PostgreSQL over Tailscale (temporary, Phase B migrates this)
4. Deploy `vandebharat-frontend` on VPS
5. Deploy `vandebharat-report-generator` on VPS
6. Deploy `prometheus` on VPS — scrape configs include DGX exporters via Tailscale
7. Deploy `grafana` on VPS — point to VPS Prometheus
8. Deploy `loki` on VPS — configure Promtail on both machines
9. Verify `api.mvis.online`, `frontend.mvis.online`, `grafana.mvis.online` all work
10. Remove `vandebharat-backend`, `vandebharat-frontend`, `report-generator`, `grafana` from DGX

Done when: Dashboard at `frontend.mvis.online` loads live data from DGX over Tailscale.

Phase B — VPS PostgreSQL as Master

Goal: VPS PostgreSQL is the system of record. sync-engine implements the production state machine.

1. Deploy production sync-engine on DGX with env:

```
VPS_POSTGRES_URL=postgres://mvis_admin:<pw>@100.111.143.3:5432/mvis
SYNC_INTERVAL_SECONDS=5
SYNC_BATCH_SIZE=1000
SYNC_DEVICE_ID=dgx-spark
SYNC_STATION_ID=station-01
```

2. Deploy session-heartbeat-monitor on VPS
3. Update VPS backend `DATABASE_URL` to point to VPS PostgreSQL
4. Test: start session on DGX, verify rows in VPS DB within 5s
5. Test: kill Tailscale on DGX for 10 minutes, verify DGX continues capturing, verify backlog syncs on reconnect

Done when: `frontend.mvis.online` reads from VPS PostgreSQL, DGX DB is hot buffer only.

Phase C — Frame Storage Restructure

Goal: All frames saved to MinIO with correct bucket structure and async upload path.

1. Create MinIO bucket `vandebharat`, set all-private bucket policy
2. Create per-service MinIO access keys (see Section 11.4)
3. Update `camsvr`: write raw frames to local NVMe `/data/capture/`, publish local path to Redis
4. Deploy `async-uploader` on DGX: reads raw_frames stream, uploads to MinIO, deletes local files
5. Update `vandebharat-preprocessing`: write preprocessed frames to MinIO `/sessions/{id}/frames/`
6. Update `vandebharat-correlation`: copy anomaly frames to `/sessions/{id}/anomalies/`
7. Update `vandebharat-backend`: return 5-minute presigned URLs (not 15-minute)
8. Update `vandebharat-frontend`: fetch frames directly from presigned URLs
9. Deploy `frame-lifecycle-manager`: monitors `/data/capture/` disk usage, alerts at 80%, pauses at 95%

Done when: Frame images in dashboard load directly from `minio.mvis.online`, DGX disk stays bounded.

Phase D — Full Session Lifecycle

Goal: Session END triggers automatic, end-to-end report generation with no manual steps.

1. sync-engine: on DONE state, POST `/api/sessions/{id}/complete` to VPS backend
2. VPS backend: enqueue report job in VPS Redis `report_jobs` stream
3. `report-generator`: picks job, queries VPS PostgreSQL, fetches anomaly frames with checksum verification, generates PDF with digital signature
4. PDF uploaded to MinIO: `/sessions/{id}/reports/{id}.pdf`
5. `reports` table updated with presigned URL (7-day expiry, refreshed on access)
6. Dashboard shows "Report Ready" with download link
7. Session marked `archived` after frame-archiver confirms DO Spaces upload

Done when: End-to-end — train clears cameras → automatic PDF report in dashboard — zero manual steps.

Phase E — Direct DGX → DO Spaces Upload

Goal: Frames upload directly from DGX to DO Spaces. VPS bandwidth is never used for frame data.

1. Acquire DO Spaces credentials (BLR1 region)
2. Deploy `frame-archiver` on DGX (replaces mc mirror):
 - Reads from MinIO per-session after sync-engine signals CONFIRMED
 - Uploads each object with checksum verification to DO Spaces
 - Tracks per-object upload status in local SQLite
 - Retries with exponential backoff; alerts after 3 consecutive failures per object
3. Update `S3_ENDPOINT` env vars across all services to DO Spaces
4. Configure DO Spaces lifecycle rules (Section 9.5)
5. Verify presigned URLs in dashboard resolve to `blr1.digitaloceanspaces.com`

Done when: Frame URLs in dashboard resolve to DO Spaces. DGX MinIO is a cache, not permanent storage.

Phase F — DGX Port Lockdown

Goal: No DGX service accessible from the public internet. All access via Tailscale.

1. Remove `0.0.0.0:5432` binding from `vandebharat-postgres-1` (critical — do this first)
2. Remove public bindings from all DGX services except MinIO (`:9010`) and mvis-dashboard (`:9003`)
3. Add UFW rules on DGX to restrict MinIO and dashboard to Tailscale CIDR (`100.64.0.0/10`) only
4. Confirm Portainer is accessible only via Tailscale IP
5. Verify: `ss -tlnp | grep '0.0.0.0'` returns only DGX Coolify management ports

Done when: `nmap 64.227.138.37 -p 5432,8001,8091,9002` returns all closed/filtered.

15. Service Level Objectives

15.1 Production SLOs

Objective	SLI	Target	Alert Threshold
Frame capture rate	Frames captured / frames expected per second	≥ 99% (594/600 fps)	< 95% for 30s
YOLO inference latency (P99)	Time from preprocessed_frames XADD to detections XADD	≤ 100ms	> 200ms for 60s
OCR inference latency (P99)	Time from preprocessed_frames to ocr_results	≤ 150ms	> 300ms for 60s
End-to-end pipeline latency (P99)	Frame capture → inspection_results committed	≤ 500ms	> 1s for 60s
Sync latency	Time for DGX result → visible in VPS dashboard	≤ 10s	> 30s for 60s
Report generation time	Session complete → PDF available	≤ 5 minutes	> 15 minutes
API response time (P99)	GET /api/sessions response time	≤ 500ms	> 1s for 60s
Dashboard load time	Time to first contentful paint	≤ 3s	> 5s
Frame fetch latency (P99)	Presigned URL → first byte from MinIO	≤ 200ms	> 500ms
System uptime (operational hours)	Available minutes / total operational minutes	≥ 99.5%	< 99% rolling 7d
Data durability	Inspection results lost / total results	0%	Any loss
Sync engine success rate	Successful batches / total batches	≥ 99.9%	< 99% rolling 1h
Pipeline error rate	Frames to DLQ / total frames	≤ 0.01%	> 0.1%
GPU utilization during session	dcpm_gpu_utilization during active session	≥ 70%	< 40% for 5min

15.2 SLO Measurement

```
# SLO compliance is computed nightly and stored in VPS PostgreSQL
# Feed to Grafana for rolling 7-day, 30-day SLO dashboards

INSERT INTO slo_measurements (
  date, metric_name, target, actual, compliant
) VALUES
  (NOW()::date, 'pipeline_latency_p99_ms', 500, measured_p99, measured_p99 ≤ 500),
  (NOW()::date, 'sync_latency_seconds', 10, avg_sync_delay, avg_sync_delay ≤ 10),
  ...
```

16. Capacity Planning

16.1 Assumptions

Parameter	Value	Basis
Cameras per session	12	Current hardware
Frame rate	50 FPS per camera	Current config
Total frame rate	600 FPS	12 × 50
Average session duration	10 minutes	Estimated train transit
Average JPEG size (raw)	200 KB	1080p, 80% quality
Average JPEG size (preprocessed)	80 KB	Resized, denoised
Detections per frame (average)	5	Model estimate
OCR regions per frame (average)	2	Model estimate
Sessions per day (Tier 1)	5	Current deployment
Sessions per day (Tier 2)	50	6-month target
Sessions per day (Tier 3)	500	12-month target

16.2 Storage Growth

Tier	Sessions/day	Raw frames	Preprocessed	Anomalies (~5%)	Reports	Total/day
1	5	36 GB	14.4 GB	1.8 GB	50 MB	~52 GB/day
2	50	360 GB	144 GB	18 GB	500 MB	~522 GB/day
3	500	3.6 TB	1.44 TB	180 GB	5 GB	~5.2 TB/day

Calculation: 600 frames/s × 10 min × 200 KB = 72 GB raw per session.

DGX MinIO local storage requirement: DGX MinIO is a cache, not permanent storage. Local storage must hold peak concurrent sessions × session size + upload lag buffer.

```
Minimum local storage: (max_concurrent_sessions × 72 GB raw) + 50% buffer
Tier 1:                3 concurrent × 72 GB × 1.5 = 324 GB
Recommended:          1 TB NVMe (separate from OS drive)
```

DO Spaces permanent storage:

```
Tier 1: 52 GB/day × 365 = 18.9 TB/year → ~$380 USD/year at DO pricing
Tier 2: 522 GB/day × 365 = 190 TB/year → ~$3,800 USD/year
Tier 3: Consider dedicated CDN + tiered storage
```

16.3 Redis Memory

```
Stream message overhead: ~500 bytes per message (metadata + headers)

raw_frames (MAXLEN 10,000):    10,000 × 500 B = 5 MB
preprocessed_frames (MAXLEN 5,000):  5,000 × 500 B = 2.5 MB
detections (MAXLEN 50,000):      50,000 × 500 B = 25 MB
ocr_results (MAXLEN 20,000):     20,000 × 500 B = 10 MB
inspection_results (MAXLEN 5,000):  5,000 × 500 B = 2.5 MB
DLQ streams (~10,000 total):      5 MB

Total stream memory: ~50 MB
Redis overhead + VPS job queues: ~100 MB

Recommended maxmemory: 512 MB (DGX), 256 MB (VPS)
```

16.4 PostgreSQL Row Growth

Per session:

- frames_metadata: $600 \text{ fps} \times 600\text{s} = 360,000 \text{ rows} \times \sim 200 \text{ bytes} = 72 \text{ MB}$
- detections: $360,000 \times 5 \text{ avg} = 1,800,000 \text{ rows} \times \sim 150 \text{ bytes} = 270 \text{ MB}$
- ocr_results: $360,000 \times 2 \text{ avg} = 720,000 \text{ rows} \times \sim 200 \text{ bytes} = 144 \text{ MB}$
- Total per session: $\sim 486 \text{ MB}$

DGX PostgreSQL: Holds 1-2 sessions max before sync clears – need $\sim 2 \text{ GB}$ safety buffer

VPS PostgreSQL (Tier 1, 1 year):

- $5 \text{ sessions/day} \times 365 \text{ days} \times 486 \text{ MB} = \sim 888 \text{ GB}$
- Partitioning is mandatory
- Recommend DigitalOcean Managed PostgreSQL starting at 20 GB , scaling up

16.5 Network Bandwidth

Tailscale (DGX → VPS):

- Result batches: $360,000 \text{ frames} \times 620 \text{ bytes per row-set} = \sim 223 \text{ MB/session}$
- At 5 sessions/day Tier 1: $\sim 1.1 \text{ GB/day}$ (tiny)
- At 50 sessions/day Tier 2: $\sim 11 \text{ GB/day}$ (still manageable on 1 Gbps VPS)

DGX → DO Spaces (direct, not via VPS):

- $72 \text{ GB raw} + 29 \text{ GB preprocessed} = 101 \text{ GB/session}$
- At Tier 1 (5/day): 505 GB/day → requires sustained 50+ Mbps upload from DGX
- At Tier 2 (50/day): 5 TB/day → requires 500 Mbps upload → evaluate compression

16.6 GPU Utilization

YOLO (YOLOv8x): $\sim 15\text{ms}$ per frame on DGX Spark RTX

OCR (PaddleOCR): $\sim 10\text{ms}$ per frame

Preprocessing: $\sim 3\text{ms}$ per frame

Pipeline saturation point (single GPU):

- Bottleneck: YOLO at 15ms/frame → max 66 FPS single-threaded
- Solution: Batch inference (batch_size=8) → $\sim 120 \text{ FPS}$ effective throughput
- For 600 FPS: 5 parallel GPU streams required
- DGX Spark has 4x RTX GPUs → can handle current load with 80% headroom

Alert thresholds:

- GPU utilization < 70% during session = model not batching correctly
- GPU utilization > 95% for > 2 minutes = approaching saturation, alert

16.7 Scaling Thresholds

Metric	Scale-up Trigger	Action
DGX GPU utilization	> 85% sustained 30min	Add second DGX or increase batch size
DGX disk utilization	> 80%	Alert; > 90% pause new sessions
VPS PostgreSQL size	> 80% of allocated	Scale up DigitalOcean Managed DB
Sync backlog	> 100,000 unsynced rows	Alert; > 500,000 escalate
Redis memory	> 75% of maxmemory	Increase maxmemory or investigate consumer lag
Report generation queue	> 20 pending jobs	Scale report-generator horizontally

17. Session State Machine

17.1 State Diagram

```

stateDiagram-v2
    [*] --> IDLE : System ready

    IDLE --> ACTIVE : Operator starts session\n(camsvr creates session_id)
    ACTIVE --> ACTIVE : Frames captured, pipeline running\nHeartbeats sent every 30s
    ACTIVE --> DRAINING : Operator stops session\n(train clears sensors)
    DRAINING --> PROCESSING : All pipeline streams flushed\nAll frames XACK'd
    PROCESSING --> SYNCING : DGX PostgreSQL fully written\nsync-engine begins final flush
    SYNCING --> CONFIRMING : All rows sent to VPS\nVPS returns confirmed_ids
    CONFIRMING --> ARCHIVING : VPS commits confirmed\nframe-archiver triggered
    ARCHIVING --> COMPLETE : DO Spaces upload verified\nDGX local frames purged
    COMPLETE --> IDLE : Cleanup done\nSession available in dashboard

    ACTIVE --> INTERRUPTED : DGX heartbeat missed > 2min\n(VPS marks interrupted)
    DRAINING --> INTERRUPTED : Drain timeout exceeded (5min)
    SYNCING --> DEGRADED : VPS unreachable > 15min
    DEGRADED --> SYNCING : Network recovered
    DEGRADED --> ERROR : Disk full OR data corruption

    INTERRUPTED --> ACTIVE : Operator resumes session (if train still present)
    INTERRUPTED --> PROCESSING : Operator forces session end
    ERROR --> [*] : Manual operator intervention + data audit

```

17.2 State Definitions

State	Entry Condition	Exit Condition	Possible Failures	Recovery
IDLE	System start, or previous session COMPLETE	Operator clicks Start Session	—	—
ACTIVE	Session START signal from camsvr	Operator clicks Stop OR sensors detect train clear	DGX crash, GPU OOM, disk full	Restart services; session may be INTERRUPTED
DRAINING	Stop signal received	All in-flight Redis stream messages XACK'd	Stream drain timeout (5 min)	Force-close: mark remaining pending as failed, proceed to PROCESSING
PROCESSING	Streams drained	DGX PostgreSQL has all results written	DB write failure	Retry writes; if PostgreSQL unavailable, wait for restart
SYNCING	sync-engine begins final flush	All rows confirmed by VPS	Network outage	Exponential backoff retry; session enters DEGRADED after 15min
CONFIRMING	VPS batch endpoint returns confirmed_ids	VPS confirms is_final=true batch	VPS returns partial confirmed list	Re-send unconfirmed rows; VPS idempotently ignores duplicates
ARCHIVING	frame-archiver triggered	All objects checksum-verified in DO Spaces	DO Spaces unavailable	Retry per-object; alert after 3 consecutive failures per object
COMPLETE	Archive confirmed, DGX local frames purged	Report generation triggered	Report generator failure	Enqueue new report job; report failure does not block session completion
INTERRUPTED	DGX heartbeat missed > 2 min	Operator decision (resume or force-end)	Ongoing network partition	DGX may have continued operating; sync-engine will flush on reconnect
DEGRADED	sync-engine retries exhausted after 15 min	Network recovered	Disk full on DGX	Alert; pause new sessions; manual operator action
ERROR	Disk full OR detected data corruption	Manual operator reset	—	Requires data audit before resuming

18. Failure Matrix

Component	Failure Scenario	Impact	Detection	Auto Recovery	Manual Recovery	Expected Downtime	Data Loss Risk
Redis	OOM kill during session	All in-flight pipeline frames lost (without AOF)	redis_connected_clients == 0	Docker restart (restart:always); AOF replay	Verify AOF enabled; check XPENDING after restart	< 30s with AOF	At-most-1s of frames (with AOF); entire session (without AOF)
Redis	AOF disk full	Redis refuses writes; pipeline stalls	redis_aof_enabled alert fires	None	Free disk space; restart Redis	Variable	None (AOF writes blocked, no data written)
DGX PostgreSQL	Container crash	result-persister cannot write; inspection_results stream backs up	pg_up == 0 alert	Docker restart	pg_restore from WAL backup if corruption	< 60s	In-Redis-PEL frames reprocessed after restart
DGX PostgreSQL	Data corruption	Sync-engine sending corrupt rows to VPS	Checksum errors in sync log	None	pg_restore from last session-end backup; re-sync from Redis if AOF intact	15-60 min	Potentially one session's results
MinIO (DGX)	Container crash	async-uploader cannot write; local disk fills	Uploader error rate spike	Docker restart	Restart; uploader resumes from local disk (SQLite tracking)	< 60s	None — local files intact
MinIO (DGX)	Local disk full	async-uploader writes fail; local NVMe fills; frame capture may stall	DGXDiskCritical alert	frame-lifecycle-manager purges oldest completed session	Manual cleanup; pause new sessions	Variable	None if purge logic runs correctly
YOLO service	GPU OOM crash	No new detections; correlation emits partial results after 60s timeout	PipelineStalled alert; YOLO container exit	Docker restart; XAUTOCLAIM reclaims pending frames	Verify GPU memory; reduce batch size	15-30s restart + reprocessing	Partial detections for ~60s window (flagged as partial_result=1)
OCR service	CUDA driver error	No new OCR results; correlation emits partial results	PipelineStalled alert for OCR service	Docker restart	Check CUDA driver; restart Docker daemon if needed	30-60s	Partial OCR for ~60s window (flagged)
correlation	Crash with frames in join buffer	Buffered join state lost; partial results for in-buffer frames	StreamApproachingMaxLen on inspection_results going silent	Docker restart; XAUTOCLAIM handles unacked msgs	Manual audit of partial_result frames in that time window	< 30s	~60s of frames may have partial results
sync-engine	Network to VPS down	DGX PostgreSQL accumulates unsynced rows; VPS falls behind	SyncEngineFailing alert; SessionHeartbeatMissed on VPS	Exponential backoff retry; bulk-sync on recovery	If > 1 hour: check Tailscale, check VPS status	Variable (VPS catches up on recovery)	None — rows preserved in DGX PostgreSQL
sync-engine	Crash mid-sync	Some rows may be in-flight (sent but not confirmed)	sync_rows_pending gauge stops decreasing	Docker restart; resumes from last confirmed watermark	Verify no duplicate rows in VPS (ON CONFLICT DO	< 60s	None

NOTHING is idempotent)							
VPS	Droplet unavailable	Dashboard inaccessible; sync-engine buffers on DGX; report generation halted	All VPS health checks fail	DigitalOcean automatic droplet restart (if hardware issue)	Restore from DO snapshot; restore from managed PostgreSQL (if separate)	15-60 min	None for inspection data; may lose recent sync_log entries
VPS PostgreSQL	Unavailable	Sync-engine cannot deliver; backend API returns 500	pg_up == 0; API error rate spike	Docker restart; pg_restore from WAL if corruption	pg_restore from DO managed backup	5-30 min	Sync-engine buffers on DGX; no inspection data lost
Tailscale	VPN down	DGX-VPS link severed; sync halts; proxied DGX services unavailable	SessionHeartbeatMissed; SyncEngineFailing	Tailscale auto-reconnect	Restart tailscaled on affected machine; check Headscale status	1-5 min typically	None — DGX continues independently
DGX machine	Power loss / hardware failure	Active session aborted; in-flight Redis data lost (last 1s with AOF)	All DGX metrics disappear; SessionHeartbeatMissed	None (hardware)	UPS prevents this; restore from backup; session marked INTERRUPTED	Hours (hardware)	Active session data (last 1s of Redis PEL)
frame-archiver	DO Spaces network outage	Frames accumulate on DGX MinIO; DGX disk fills if prolonged	frame_archiver_failures_total alert	Retry per-object with backoff	If disk critical: compress raw frames; alert operator	Variable	None — frames on DGX MinIO until DO Spaces recovers
GPU hardware	GPU failure (hardware)	YOLO/OCR/preprocessing stop; pipeline halts	GPU metrics disappear; CUDA error in service logs	None (hardware)	Replace GPU; restart inference services	Hours (hardware)	Active session data in Redis streams
camsvr	Crash during session	Frame capture stops; session left in ACTIVE state	Frame capture rate drops to 0	Docker restart; session-manager detects gap	Restart; determine if session can be resumed or must be ended	< 60s	Frames during crash window (not captured)
Traefik (VPS)	Misconfiguration after deploy	All *.mvis.online subdomains return 502/503	Synthetic monitoring hits all subdomains	Dokploy restarts Traefik automatically	Rollback to previous Traefik config	< 5 min	None

19. Disaster Recovery

19.1 Recovery Objectives

Scenario	RPO	RTO
Redis crash (AOF enabled)	≤ 1 second	< 30 seconds
DGX container failure	≤ 5 minutes (last RDB snapshot)	< 5 minutes
DGX OS drive failure	≤ 24 hours (last pg_dump)	2-4 hours
DGX complete hardware loss	≤ 24 hours	4-8 hours
VPS unavailable (droplet restart)	≤ 1 hour (DO managed PostgreSQL)	15-30 minutes
VPS + database destroyed	≤ 24 hours (last DO snapshot)	30-60 minutes
Network isolated (DGX↔VPS)	0 (DGX continues independently)	N/A (no interruption)
DO Spaces unavailable	0 (frames on DGX MinIO)	N/A (archiver retries)

19.2 Backup Schedule

Data	Location	Schedule	Retention	Tool
DGX Redis RDB	<code>/data/redis-backups/</code> + DO Spaces	Every 5 minutes	7 days	cron + rclone
DGX PostgreSQL	DO Spaces <code>/backups/dgx-postgres/</code>	After every session completion + daily full	30 days	pg_dump + rclone
DGX WAL archive	<code>/data/wal-archive/</code> (separate drive)	Continuous (WAL streaming)	7 days	PostgreSQL <code>archive_command</code>
VPS PostgreSQL	DO Managed Postgres (managed)	Daily automated	7 days	DO managed
VPS PostgreSQL	DO Spaces <code>/backups/vps-postgres/</code>	Weekly full	90 days	pg_dump + rclone
VPS droplet state	DigitalOcean Snapshots	Weekly	4 snapshots	DO Snapshots API
Frame objects	DO Spaces (primary), DGX MinIO (cache)	Continuous (frame-archiver)	Per lifecycle rules	frame-archiver

19.3 Recovery Procedures

Scenario A: DGX Redis Crash

```
# 1. Docker auto-restarts Redis (restart:always)
# 2. Verify AOF replay completed:
docker logs vande Bharat-redis-1 | grep -E "AOF|ready to accept"
# Expected: "DB loaded from append only file" + "Ready to accept connections"

# 3. Check consumer pending entries (should be reclaimed by XAUTOCLAIM):
redis-cli XPENDING raw_frames preprocessing-group - + 100

# 4. Verify pipeline resumes:
watch -n2 "redis-cli XLEN inspection_results"
# Should be increasing if pipeline is healthy

# 5. If recovery stalls - manually trigger XAUTOCLAIM in each service:
# Restart service containers to trigger startup XAUTOCLAIM
docker restart vande Bharat-preprocessing-1 vande Bharat-yolo-1 vande Bharat-ocr-1
```

Scenario B: DGX PostgreSQL Corruption


```
# 1. Stop sync-engine and result-persister to prevent further writes
docker stop vande Bharat-sync-engine-1 vande Bharat-worker-1

# 2. Restore from most recent pg_dump backup
LATEST_BACKUP=$(aws s3 ls s3://vande Bharat/backups/dgx-postgres/ | sort | tail -1 | awk '{print $4}')
aws s3 cp s3://vande Bharat/backups/dgx-postgres/$LATEST_BACKUP /tmp/dgx-restore.sql.gz

# 3. Drop and recreate database
docker exec vande Bharat-postgres-1 psql -U postgres -c "DROP DATABASE vande Bharat;"
docker exec vande Bharat-postgres-1 psql -U postgres -c "CREATE DATABASE vande Bharat;"
gunzip -c /tmp/dgx-restore.sql.gz | docker exec -i vande Bharat-postgres-1 psql -U postgres vande Bharat

# 4. Resume services
docker start vande Bharat-sync-engine-1 vande Bharat-worker-1

# 5. Verify sync resumes from last confirmed watermark
docker logs vande Bharat-sync-engine-1 | grep "sync_batch" | tail -10
```

Scenario C: VPS Complete Failure

```
# 1. Create new VPS from latest DigitalOcean snapshot
doctl compute droplet create mvis-vps-2 \
  --region blr1 --size s-2vcpu-4gb \
  --image $(doctl compute snapshot list --format ID,Name | grep mvis-vps | tail -1 | awk '{print $1}')

# 2. Update DNS to new VPS IP
doctl compute domain records update mvis.online \
  --record-id <id> --record-data <new-ip>

# 3. If using DigitalOcean Managed PostgreSQL - no action needed
# VPS connects to same managed DB with existing credentials

# 4. Verify services come up (Dokploy auto-restarts)
curl -sf https://api.mvis.online/healthz

# 5. Update sync-engine VPS_POSTGRES_URL if VPS IP changed
# (In Coolify on DGX - update env var and redeploy sync-engine)
```

Scenario D: Network Isolated (DGX Cannot Reach VPS for Hours)

```
# No action required on DGX - it continues operating independently
# sync-engine is in DEGRADED state, retrying every 5 minutes

# Monitor DGX disk usage:
ssh dgx "df -h /data"

# If DGX disk at > 80% during prolonged isolation:
ssh dgx "docker exec vande Bharat-minio-1 mc rm --recursive --force \
  dgx/vande Bharat/sessions/<oldest-archived-session>/"

# When network recovers, verify sync-engine catches up:
watch -n5 "curl -sf http://100.76.12.44:5004/metrics | grep mvis_sync_rows_pending"
# Gauge should decrease to 0 within ~10 minutes
```

19.4 Disaster Recovery Testing Schedule

Test	Frequency	Method	Pass Criteria
Redis crash recovery	Monthly	Kill Redis container during session	Pipeline resumes < 30s; no frames lost (confirmed by sequence audit)
YOLO crash recovery	Monthly	Kill YOLO container during session	YOLO restarts < 30s; XAUTOCLAIM reclaims pending; partial_result flag in DB
Network isolation	Quarterly	Disconnect Tailscale for 30 minutes	DGX continues capturing; full backlog synced within 10 minutes of reconnect
VPS restore	Quarterly	Restore VPS from snapshot to test environment	All services up; data accessible; API responds correctly
Full DR test	Annually	Simulate DGX hardware failure; restore from backups	RTO verified; RPO verified; audit trail intact

20. Runbooks

20.1 Runbook: Redis Failure

Symptoms:

- PipelineStalled alert fires
- Dashboard shows frames not advancing
- redis_connected_clients metric drops to 0
- Service logs show ConnectionRefusedError or LOADING Redis is loading the dataset in memory

Diagnostics:

```
# Check Redis container status
docker ps | grep redis
docker logs vande Bharat-redis-1 --tail 50

# Check Redis memory
docker exec vande Bharat-redis-1 redis-cli INFO memory | grep used_memory_human

# Check AOF status
docker exec vande Bharat-redis-1 redis-cli CONFIG GET appendonly
docker exec vande Bharat-redis-1 redis-cli INFO persistence | grep aof
```

Recovery Steps:

```
# Step 1: If container is stopped - restart it
docker start vande Bharat-redis-1
# Wait 30-60 seconds for AOF replay

# Step 2: Verify AOF loaded successfully
docker logs vande Bharat-redis-1 | grep -E "DB loaded|ready to accept|Aborting"

# Step 3: Verify streams are intact
docker exec vande Bharat-redis-1 redis-cli XLEN raw_frames
docker exec vande Bharat-redis-1 redis-cli XLEN detections

# Step 4: Trigger XAUTOCLAIM in all consumers (restart services)
docker restart vande Bharat-preprocessing-1 vande Bharat-yolo-1 vande Bharat-ocr-1 \
    vande Bharat-correlation-1 vande Bharat-worker-1

# Step 5: Verify pipeline resumes
watch -n2 "docker exec vande Bharat-redis-1 redis-cli XLEN inspection_results"
```

Validation:

- redis_connected_clients > 4 (all pipeline services reconnected)
- mvis_frames_processed_total{service="yolo"} counter increasing

- No `PipelineStalled` alert firing after 90 seconds

Escalation: If AOF replay fails or data corruption detected → restore from RDB backup → escalate to on-call engineer.

20.2 Runbook: PostgreSQL Failure (DGX)

Symptoms:

- `pg_up{job="dgx-postgres"} == 0` alert fires
- `result-persister` logs: `could not connect to server`
- `inspection_results` stream length growing (persister not consuming)

Diagnostics:

```
docker ps | grep postgres
docker logs vande Bharat-postgres-1 --tail 100
df -h /var/lib/docker/volumes/ # Check disk space
```

Recovery Steps:

```
# Step 1: Attempt restart
docker start vande Bharat-postgres-1
sleep 10
docker exec vande Bharat-postgres-1 pg_isready -U postgres

# Step 2: If startup fails - check logs for corruption
docker logs vande Bharat-postgres-1 | grep -iE "PANIC|FATAL|corruption"

# Step 3: If corruption - restore from last backup (see Section 19.3 Scenario B)

# Step 4: After PostgreSQL up - result-persister reconnects automatically
# Verify inspection_results stream is draining:
watch -n2 "docker exec vande Bharat-redis-1 redis-cli XLEN inspection_results"
```

Validation:

- `pg_up{job="dgx-postgres"} == 1`
- `mvis_sync_rows_pending` decreasing

20.3 Runbook: MinIO Failure (DGX)

Symptoms:

- `async-uploader` logs: `connection refused` or `S3Error`
- Local `/data/capture/` disk usage growing (uploader not clearing files)
- `DGXDiskCritical` alert approaching

Diagnostics:

```
docker ps | grep minio
curl -sf http://localhost:9010/minio/health/live
docker logs vande Bharat-minio-1 --tail 50
df -h /data # Check if data disk is full
```

Recovery Steps:

```
# Step 1: Restart MinIO
docker restart vande Bharat-minio-1
sleep 5
curl -sf http://localhost:9010/minio/health/live

# Step 2: Verify async-uploader resumes (restart it to trigger retry)
docker restart vande Bharat-async-uploader-1

# Step 3: Monitor local disk drain
watch -n10 "df -h /data && ls -la /data/capture/ | wc -l"
```

Validation:

- `curl -sf http://localhost:9010/minio/health/live` returns 200
- `/data/capture/` file count decreasing (async-uploader clearing files)

20.4 Runbook: GPU / YOLO Failure

Symptoms:

- `PipelineStalled` alert for YOLO
- `detections` stream length not increasing
- YOLO container logs: `CUDA out of memory` or `Segmentation fault`
- `dsgm_fb_used_mb` was near `dsgm_fb_total_mb` before crash

Diagnostics:

```
docker ps | grep yolo
docker logs vande Bharat-yolo-1 --tail 50 | grep -iE "cuda|error|killed|oom"
nvidia-smi # Check GPU memory and process status
docker exec vande Bharat-redis-1 redis-cli XPENDING detections correlation-group - + 5
```

Recovery Steps:

```
# Step 1: Restart YOLO (Docker restart policy handles this automatically)
docker restart vande Bharat-yolo-1

# Step 2: If OOM - reduce batch size before restart
docker exec vande Bharat-yolo-1 env | grep BATCH_SIZE
# Update BATCH_SIZE in Coolify env vars, redeploy

# Step 3: XAUTOCLAIM runs on startup - pending frames reclaimed automatically
# Verify after 30 seconds:
docker exec vande Bharat-redis-1 redis-cli XPENDING preprocessed_frames yolo-group - + 5
# Pending count should decrease

# Step 4: Check for partial results in DB (frames with partial_result=1)
docker exec vande Bharat-postgres-1 psql -U postgres vande Bharat -c \
  "SELECT COUNT(*) FROM frames_metadata WHERE partial_result=TRUE AND captured_at > NOW() - INTERVAL '1 hour';"
```

Validation:

- `mvis_frames_processed_total{service="yolo"}` counter increasing
- `PipelineStalled` alert resolved after 90 seconds

20.5 Runbook: Synchronization Engine Failure

Symptoms:

- `SyncEngineFailing` alert

- `mvis_sync_rows_pending` gauge increasing
- VPS dashboard showing stale data
- sync-engine logs: repeated `NetworkError` or `Timeout`

Diagnostics:

```
docker logs vande Bharat-sync-engine-1 --tail 100 | grep -E "ERROR|WARN|retry"
# Check Tailscale connectivity:
ping 100.111.143.3 # VPS Tailscale IP
curl -sf https://api.mvis.online/healthz
tailscale status
```

Recovery Steps:

```
# Step 1: If Tailscale down - restart Tailscale
sudo systemctl restart tailscaled
sleep 5
tailscale status | grep 100.111.143.3

# Step 2: If VPS API down - check VPS health, sync-engine will auto-retry

# Step 3: If sync-engine itself crashed - restart
docker restart vande Bharat-sync-engine-1
docker logs vande Bharat-sync-engine-1 --tail 20

# Step 4: Monitor backlog draining
watch -n5 "curl -sf http://localhost:5004/metrics | grep mvis_sync_rows_pending"
```

Validation:

- `mvis_sync_rows_pending` decreasing to 0
- `mvis_sync_batches_sent_total{status="success"}` increasing
- VPS dashboard showing updated data

20.6 Runbook: Camera Failure

Symptoms:

- Operator reports black feed for one or more cameras in dashboard
- `mvis_frames_received_total{cam_id="cam_XX"}` counter stalls
- camsvr logs: `Camera cam_XX disconnected` or `Timeout reading from cam_XX`

Diagnostics:

```
docker logs mvis-camsvr-1 --tail 50 | grep -iE "camera|disconnect|timeout|cam_0"
# Check if camera is physically connected and powered
ping <camera_ip> # If IP cameras
```

Recovery Steps:

```
# Step 1: camsvr auto-reconnects (built-in reconnect logic)
# Wait 30 seconds and check if reconnected
docker logs mvis-camsvr-1 --tail 20 | grep "Reconnected"

# Step 2: If not reconnected - restart camsvr
# WARNING: restarting camsvr during an active session will cause frame capture to pause
# Confirm with operator before proceeding
docker restart mvis-camsvr-1

# Step 3: If hardware issue - check physical cable, camera power, and network switch
# Replace camera if hardware confirmed defective
```

Validation:

- `mvis_frames_received_total{cam_id="cam_XX"}` counter increasing
- Operator confirms live feed restored in dashboard

21. Architecture Decision Records

ADR-001: Redis Streams for Pipeline Backbone

Status: Accepted

Context: The pipeline requires fan-out (same frames processed by YOLO, OCR, and storage-writer independently), backpressure, at-least-once delivery, and consumer group semantics across 5+ processing stages.

Decision: Use Redis Streams with consumer groups.

Alternatives Considered:

Alternative	Reason Rejected
Apache Kafka	5× operational overhead (JVM, ZooKeeper/KRaft, broker management) with no benefit at single-machine scale. Network latency between Kafka broker and consumers would add ms-level overhead to every frame.
NATS JetStream	Good alternative, but adds a new dependency with no advantage over Redis (already in stack). Re-evaluate at Tier 3 when cross-machine streaming is needed.
RabbitMQ	No native stream semantics; fan-out requires exchange topology; higher operational complexity than Redis.
Direct service calls (HTTP/gRPC)	Tight coupling; no backpressure; cascade failures when a consumer is slow; no at-least-once delivery without custom retry logic.
ZeroMQ	No persistence; consumer crash loses in-flight messages. Not suitable for a pipeline where we need at-least-once delivery.

Advantages:

- Already in stack — zero new infrastructure
- Microsecond latency — critical for 600 FPS throughput
- Consumer groups provide independent fan-out cursors
- XAUTOCLAIM enables at-least-once delivery on consumer crash
- AOF persistence makes streams durable across Redis restarts
- Near-zero operational overhead

Disadvantages:

- Single Redis instance is a SPOF (mitigated by AOF + Docker restart)
- No built-in multi-machine stream replication (relevant at Tier 3)
- DLQ implementation is manual (no built-in poison message handling)

Future Migration Path: At Tier 3 (10+ DGX nodes), evaluate NATS JetStream for cross-machine stream distribution. Migration is straightforward: replace XADD/XREADGROUP with NATS JetStream publish/subscribe.

ADR-002: Two-Database Pattern (DGX Hot + VPS Master)

Status: Accepted

Context: The pipeline produces ~1,800,000 detection rows per 10-minute session at 600 FPS. Writing these over the network to VPS PostgreSQL in real-time is not feasible (latency × row count = seconds of blocking writes). But all historical data must be available on VPS for reporting and dashboard.

Decision: DGX PostgreSQL is a high-throughput write buffer. VPS PostgreSQL is the system of record. sync-engine batches writes every 5 seconds.

Alternatives Considered:

Alternative	Reason Rejected
Write directly to VPS PostgreSQL over Tailscale	3-8ms round-trip × 1,800,000 rows = 1.5-4 hours of blocking time per session. Infeasible.
Write to VPS via async queue	Adds a third queue system; same semantic as sync-engine but more complex.
Columnar store on DGX (e.g., ClickHouse)	Good for analytics but adds a new dependency and doesn't solve the sync problem.
Time-series DB (TimescaleDB)	Valid choice for frames_metadata. Evaluate as VPS PostgreSQL extension at Tier 2.

Advantages:

- DGX writes at local speed (microseconds, no network)
- VPS has complete history for reporting
- Network outage does not interrupt capture or inference
- Sync engine is the only service that crosses the network — easy to monitor

Disadvantages:

- Eventual consistency (5s lag between DGX and VPS)
- Sync engine is a critical single service — must be robust
- Schema must be kept identical between DGX and VPS

Future Migration Path: At Tier 2 (multiple DGX nodes), each DGX runs its own sync-engine writing to the same VPS PostgreSQL. Partition VPS tables by `device_id` to avoid write contention.

ADR-003: MinIO → DO Spaces Migration Path

Status: Accepted

Context: During development and early production, DO Spaces is not yet provisioned. A permanent store is needed for frames from day one. The storage abstraction must be zero-cost to migrate.

Decision: Run MinIO on DGX as a development stand-in. Use the S3-compatible API exclusively. Migrate to DO Spaces by changing 3 environment variables.

Alternatives Considered:

Alternative	Reason Rejected
Use VPS local storage	VPS has limited bandwidth and disk; would require frame data to cross Tailscale
Use NFS on DGX	No S3 API; every service needs custom NFS mount logic
Use Google Cloud Storage	More expensive, higher latency from BLR1, different SDK
Start with DO Spaces immediately	Not available; need to prototype without it

Advantages: Zero migration cost; SDK calls are identical; presigned URLs work the same way.

Disadvantages: MinIO on DGX is a SPOF — if the container crashes, uploads pause until restart.

ADR-004: Tailscale for DGX-VPS Private Network

Status: Accepted, with future migration to Headscale

Context: DGX has no public IP. VPS needs to reach DGX services (MinIO, dashboard proxy) and DGX needs to reach VPS (sync-engine). A private, encrypted network is required.

Decision: Use Tailscale (WireGuard-based mesh VPN). Migrate control plane to self-hosted Headscale on VPS.

Alternatives Considered:

Alternative	Reason Rejected
Manual WireGuard	Requires key distribution, routing config, maintenance; Tailscale handles all of this automatically
OpenVPN	Higher latency, more complex configuration, no mesh routing
SSH tunnels	Fragile, not suitable for persistent service-to-service traffic
VPN via cloud provider	Adds cost and complexity; not available on DGX

Advantages: Zero-configuration mesh VPN; handles NAT traversal; WireGuard kernel module for maximum performance; 3-8ms between BLR1 VPS and DGX (measured).

Disadvantages: Dependency on Tailscale Inc. control plane (mitigated by Headscale migration); Tailscale daemon failure isolates DGX from VPS.

Migration Path: Deploy Headscale on VPS. Register all devices against self-hosted control plane. Eliminates external dependency entirely.

ADR-005: Event-Driven Architecture for Pipeline

Status: Accepted

Context: The inference pipeline has 6+ services that must process each frame independently and in parallel (YOLO and OCR process the same preprocessed frame simultaneously). Services must be independently restartable.

Decision: Pure event-driven architecture using Redis Streams. No direct service-to-service HTTP/gRPC calls within the pipeline.

Advantages: Independent restartability; natural backpressure; fan-out at no cost; observable stream lengths; clean separation of concerns.

Disadvantages: Harder to trace individual frame journeys (mitigated by trace_id throughout); slightly more complex than direct calls for simple cases.

ADR-006: Local Disk First (Not Direct MinIO Write)

Status: Accepted

Context: At 600 FPS, writing frames directly to MinIO from camsvr blocks the capture thread on storage I/O. MinIO write latency (even on local machine) varies from 1-20ms depending on disk load.

Decision: camsvr writes to local NVMe (/data/capture/), publishes the local path to Redis, and returns immediately. A separate async-uploader service reads the stream and uploads to MinIO without blocking capture.

Advantages: Capture rate is bounded only by NVMe write speed (< 1ms); MinIO availability does not affect capture; local disk survives Docker restarts.

Disadvantages: Requires disk space management (frame-lifecycle-manager); two-hop write (local → MinIO).

22. Future Scaling Roadmap

22.1 Tier 1 → Tier 2: Multiple DGX Nodes

Trigger: GPU utilization > 85% sustained, or > 10 concurrent train sessions required.

Required changes:

1. **Database schema** — `station_id` and `device_id` columns added to sessions, frames_metadata, detections, ocr_results (already included in v2.0 schema above).
2. **VPS PostgreSQL** — partition by `device_id` or `station_id` to prevent write contention from multiple sync-engines.
3. **sync-engine parameterization** — each DGX instance has its own `SYNC_DEVICE_ID` and `SYNC_STATION_ID` env vars. No code changes.
4. **Headscale** — register all DGX nodes. VPS Traefik routes per device subdomain.
5. **Grafana** — add device dimension to all dashboards.

Zero code changes: schema is already multi-device ready in v2.0.

22.2 Tier 2 → Tier 3: AGX Orin Edge Nodes

Trigger: Require lightweight capture nodes at remote stations without DGX infrastructure.

Required changes:

1. **Model tiering** — AGX Orin runs YOLOv8n (lightweight); DGX runs YOLOv8x. `model_version` field in detections distinguishes results. Report generation must handle heterogeneous model outputs.
2. **Pipeline on AGX** — same Redis Streams pipeline, lighter models, smaller MAXLEN values (AGX has less RAM).
3. **Inter-device coordination** — session assignment service on VPS routes camera sessions to the least-loaded available device.
4. **sync-engine** — same binary, different env config. No code changes.
5. **Storage** — AGX Orin may not have local MinIO capacity for full raw frames. Options: (a) upload raw frames directly to DO Spaces (bypassing local MinIO), (b) only save processed frames locally.

22.3 Tier 3 → Regional Deployment

Trigger: Multiple railway stations, hundreds of trains per day per station.

Required changes:

1. **Event streaming** — migrate from Redis Streams to NATS JetStream for cross-machine stream distribution. DGX nodes at different stations can share preprocessing load.
2. **Database** — VPS PostgreSQL partitioned by `station_id`; evaluate TimescaleDB extension for frames_metadata. Consider ClickHouse for analytics queries across sessions.
3. **Storage** — DO Spaces per-region buckets. Frame-archiver uploads to station's nearest region.
4. **Orchestration** — migrate from Coolify/Dokploy to K3s (lightweight Kubernetes) with NVIDIA GPU Operator. Enables unified deployment across all nodes.
5. **Service mesh** — introduce Istio or Linkerd for mTLS, observability, and traffic management across 50+ services.
6. **Headscale** — clustered Headscale with HA (multiple control plane nodes).

22.4 National Deployment Architecture

```

graph TB
    subgraph NORTH["Northern Zone"]
        DGX_N1[DGX Node 1] & DGX_N2[DGX Node 2]
        AGX_N[AGX Onin nodes x5]
        SPACES_N[D0 Spaces DEL1]
    end
    subgraph SOUTH["Southern Zone"]
        DGX_S1[DGX Node 1]
        AGX_S[AGX Onin nodes x3]
        SPACES_S[D0 Spaces BLR1]
    end
    subgraph CENTRAL["Central Control - VPS Cluster"]
        API_GW[API Gateway]
        VPS_DB[(Managed PostgreSQL\nRead replica per zone)]
        NATS_CLUSTER[NATS JetStream Cluster]
        GRAFANA_CENTRAL[Grafana - Central Dashboard]
    end

    DGX_N1 & DGX_N2 --> NATS_CLUSTER
    DGX_S1 --> NATS_CLUSTER
    AGX_N & AGX_S --> NATS_CLUSTER
    DGX_N1 & DGX_N2 --> SPACES_N
    DGX_S1 --> SPACES_S
    NATS_CLUSTER --> API_GW
    API_GW --> VPS_DB
    GRAFANA_CENTRAL --> VPS_DB & NATS_CLUSTER

```

23. Risks and Mitigations

23.1 Technical Risks

Risk	Probability	Impact	Mitigation
Redis crash without AOF enabled	Medium	Critical — session data loss	Immediate: Enable AOF (<code>appendonly yes</code>). Verify with <code>redis-cli CONFIG GET appendonly</code> .
DGX disk full during session	Medium	High — frame capture stalls	<code>frame-lifecycle-manager</code> monitors disk; alerts at 80%; pauses sessions at 95%. Separate NVMe for data.
GPU OOM during YOLO inference	Medium	Medium — partial session results	Docker <code>restart:always</code> ; XAUTOCLAIM reclaims pending; <code>partial_result</code> flag preserves integrity. Reduce batch size if recurring.
Correlation service produces wrong results	Low	Critical — false inspection reports	Join buffer with 60s timeout; <code>partial_result=1</code> flag on timeouts; two-pass verification (Tier 2).
sync-engine data loss (optimistic delete)	Low	Critical — permanent data loss	Immediate: Implement confirmed-delete pattern (Section 10.3). Never delete from DGX until VPS returns <code>confirmed_ids</code> .
YOLO model producing overconfident detections	Low	High — false positives in reports	Confidence threshold tuning; two-pass verification; human review for <code>confidence < 0.7</code> .

23.2 Operational Risks

Risk	Probability	Impact	Mitigation
Operator forgets to stop session (train leaves without Stop click)	Medium	Medium — session never completes	Auto-detect session end via sensor trigger or inactivity timeout (no new frames for 60s → auto-stop).
DGX time drift (NTP not synced)	Medium	Medium — timestamps inaccurate	Configure <code>chrony</code> on DGX with GPS-synced NTP server. For legal compliance, investigate PTP hardware timestamping (Tier 2).
Report generated with stale data (sync not complete)	Low	High — incomplete report	Report generation is only triggered after sync-engine confirms <code>is_final=true</code> batch accepted by VPS.
Operator accidentally deletes session from dashboard	Low	High — data loss	Soft delete only — set <code>status='deleted'</code> in sessions table. Actual data retained for 30 days.

23.3 Infrastructure Risks

Risk	Probability	Impact	Mitigation
VPS provider outage (DigitalOcean BLR1)	Low	Medium — management plane unavailable	DGX continues independently; migrate VPS to managed database (separate SPOF); maintain DO Snapshots for fast restore.
Tailscale Inc. control plane outage	Low	Medium — DGX-VPS link degraded	Deploy Headscale (self-hosted control plane on VPS). Direct WireGuard fallback path.
DO Spaces unavailable	Low	Low — frame archiving paused	frame-archiver retries with backoff; DGX MinIO is the primary store until DO Spaces recovers.
DGX hardware failure (GPU, disk, power)	Very Low	Critical — inspection capability lost	UPS mandatory. Separate data drives from OS drive. Maintain spare DGX Spark or AGX Orin for emergency substitution.
Coolify/Dokploy upgrade breaks services	Low	Medium — deployment platform unavailable	Pin platform versions. Test upgrades on staging environment. Maintain rollback procedure.

23.4 Security Risks

Risk	Probability	Impact	Mitigation
PostgreSQL on DGX publicly exposed	Current	Critical — direct database access from internet	Immediate: Remove <code>0.0.0.0:5432</code> binding. Restrict to Tailscale CIDR only.
MinIO root credentials used by services	Current	High — full object store access on compromise	Immediate: Create per-service MinIO credentials with least-privilege policies (Section 11.4).
JWT secret rotation not implemented	Medium	High — stolen token valid indefinitely	Implement RS256 JWT with 15-minute access tokens. 90-day key rotation.
Rogue device injecting false inspection data	Low	Critical — fraudulent inspection reports	mTLS on sync-engine → VPS backend: only registered devices with valid certificates can submit sync batches.

23.5 Scalability Risks

Risk	Probability	Impact	Mitigation
VPS PostgreSQL hits row count limits (no partitioning)	High (at Tier 2)	High — queries time out	Implement partitioning before Tier 2. pg_partman manages monthly partitions automatically.
DGX disk fills faster than frame-archiver uploads	Medium	High — new sessions cannot start	Monitor disk per Section 16.7. Implement lifecycle-manager hard pause at 95%. Compress raw frames if network is the bottleneck.
detections stream trimming data before correlation reads it	Current risk	Critical — silent detection loss	Immediate: Increase detections MAXLEN from 2,000 to 50,000.

24. Performance Benchmarks

24.1 Baseline Benchmarks (Collect Before Production Launch)

These benchmarks must be run and recorded before the first live train inspection. They establish the baseline for regression detection.

Benchmark	Tool	Metric to Record	Pass Threshold
Frame capture rate	camsvr internal counter	Frames captured/sec sustained over 5 minutes	≥ 594 fps (99% of 600)
YOLO inference throughput	mvis_processing_latency_seconds histogram	P50, P95, P99 latency per frame	P99 ≤ 100ms
OCR inference throughput	mvis_processing_latency_seconds histogram	P50, P95, P99 latency per frame	P99 ≤ 150ms
End-to-end pipeline latency	trace timestamps: camsvr → inspection_results	P99 capture-to-commit latency	P99 ≤ 500ms
DGX → VPS sync throughput	mvis_sync_rows_pending drain rate	Rows synced per second	≥ 10,000 rows/s
MinIO write throughput	minio server metrics	PUT objects/sec, MB/s	≥ 500 objects/s
Report generation time	reports.generated_at - sessions.ended_at	P99 report generation	≤ 5 minutes
Redis stream throughput	XADD/XREADGROUP rates	Messages/sec per stream	raw_frames ≥ 600 msgs/s
Dashboard API response	Prometheus API latency histogram	P99 GET /api/sessions/{id}/frames	≤ 500ms
Frame presigned URL generation	API latency for 100-frame page	Time to generate 100 presigned URLs	≤ 200ms

24.2 Load Test Scenarios

```
# Scenario 1: Sustained capture rate
# Run for 30 minutes; measure frame drop rate
python scripts/load_test_capture.py --cameras 12 --fps 50 --duration 1800

# Scenario 2: Pipeline saturation
# Inject synthetic frames at 120% of capacity for 5 minutes
python scripts/load_test_pipeline.py --fps 720 --duration 300

# Scenario 3: Sync engine stress
# 10 concurrent sessions, all syncing simultaneously
python scripts/load_test_sync.py --concurrent-sessions 10

# Scenario 4: Dashboard concurrent users
# 50 concurrent users fetching frame grids
locust -f scripts/locust_dashboard.py --users 50 --spawn-rate 5
```

24.3 Performance Regression Gates

After every major release (new model version, schema migration, infra change), the following gates must pass before production deployment:

1. All P99 latencies within 120% of baseline (tolerate 20% regression before rollback)
2. Zero messages in DLQ after 30-minute load test
3. Zero `partial_result=1` rows after 30-minute load test with both YOLO and OCR healthy
4. sync-engine batch failure rate < 0.1% during 30-minute network stability test
5. DGX disk usage growth rate < 2 GB/minute (lifecycle manager is keeping up)

25. Production Readiness Checklist

25.1 Security

- ☐ **P0.1** `vandebharat-postgres-1` port binding changed from `0.0.0.0:5432` to internal-only
- ☐ **P0.2** Redis AOF persistence enabled (`appendonly yes` , `appendfsync everysec`)
- ☐ **P0.3** Per-service MinIO credentials created with least-privilege policies
- ☐ **P0.4** JWT RS256 signing implemented (not HS256)
- ☐ **P0.5** mTLS on sync-engine → VPS backend endpoint
- ☐ **P0.6** RBAC roles defined and enforced (OPERATOR, INSPECTOR, ADMIN, AUDITOR, SERVICE)
- ☐ **P0.7** Audit log table deployed and populated on all state-modifying API calls

- ☐ **P0.8** All secrets managed via Doppler or equivalent (never in git, never in image)
- ☐ **P0.9** DGX UFW firewall restricts MinIO and dashboard to Tailscale CIDR only
- ☐ **P0.10** Report presigned URLs use 5-minute expiry (not 15-minute)
- ☐ **P0.11** All MinIO objects are private (no public bucket policy)

25.2 Data Integrity

- ☐ **D0.1** detections stream MAXLEN increased to 50,000
- ☐ **D0.2** ocr_results stream MAXLEN increased to 20,000
- ☐ **D0.3** raw_frames stream MAXLEN increased to 10,000
- ☐ **D0.4** XAUTOCLAIM implemented in all pipeline consumers (reclaim idle > 30s)
- ☐ **D0.5** Dead letter queue implemented (after 3 delivery attempts)
- ☐ **D0.6** DLQ handler deployed and alerting on DLQ messages
- ☐ **D0.7** sync-engine uses confirmed-delete pattern (only deletes DGX rows confirmed by VPS)
- ☐ **D0.8** VPS sync endpoint uses `ON CONFLICT DO NOTHING` on all INSERTs
- ☐ **D0.9** sync-engine batch_id idempotency check implemented
- ☐ **D0.10** Checksum verification on all MinIO object writes and reads
- ☐ **D0.11** Object naming uses sequence numbers (not timestamps)
- ☐ **D0.12** `model_version` field populated in all detections and ocr_results rows
- ☐ **D0.13** `trace_id` propagated through all pipeline stages and all DB rows

25.3 High Availability

- ☐ **H0.1** All containers have `restart: unless-stopped` policy
- ☐ **H0.2** All services expose `/healthz`, `/readyz`, `/metrics` endpoints
- ☐ **H0.3** Docker health checks configured on all services
- ☐ **H0.4** Session heartbeat monitor deployed on VPS
- ☐ **H0.5** sync-engine state machine implements DEGRADED state and alerting
- ☐ **H0.6** frame-lifecycle-manager deployed and pausing sessions at 95% disk
- ☐ **H0.7** Headscale deployed on VPS (Tailscale control plane self-hosted)
- ☐ **H0.8** DGX data drives separate from OS drive (physical separation)
- ☐ **H0.9** UPS installed for DGX Spark (mandatory for production facility)

25.4 Observability

- ☐ **O0.1** Prometheus deployed on VPS, scraping DGX exporters via Tailscale
- ☐ **O0.2** dcgm-exporter deployed on DGX (GPU metrics)
- ☐ **O0.3** redis-exporter deployed on DGX
- ☐ **O0.4** postgres-exporter deployed on DGX and VPS
- ☐ **O0.5** node-exporter deployed on DGX and VPS
- ☐ **O0.6** All pipeline services expose custom Prometheus metrics
- ☐ **O0.7** Alertmanager deployed with all rules from Section 12.3
- ☐ **O0.8** Grafana dashboards deployed for all 6 panels in Section 12.4
- ☐ **O0.9** Loki deployed on VPS; Promtail deployed on DGX and VPS
- ☐ **O0.10** All services emit structured JSON logs with trace_id
- ☐ **O0.11** OpenTelemetry SDK embedded in all services; Tempo deployed on VPS
- ☐ **O0.12** Alert routing configured (Telegram + Email for critical; Email for warning)

25.5 Database

- ☐ **DB0.1** All DGX table indexes created (Section 8.2)
- ☐ **DB0.2** All VPS table indexes created (Section 8.3)

- ☐ **DB0.3** VPS tables partitioned by date with `pg_partman`
- ☐ **DB0.4** DGX PostgreSQL: `synchronous_commit = off` (hot buffer, not source of truth)
- ☐ **DB0.5** DGX PostgreSQL: WAL archiving configured to separate directory
- ☐ **DB0.6** VPS PostgreSQL: DigitalOcean Managed Database (separate from VPS droplet)
- ☐ **DB0.7** Weekly `pg_dump` backup to DO Spaces for both databases

25.6 Disaster Recovery

- ☐ **DR0.1** Redis RDB backup cron running (every 5 minutes → DO Spaces)
- ☐ **DR0.2** DGX `pg_dump` cron after every session completion
- ☐ **DR0.3** VPS managed PostgreSQL automated backups enabled (DO default: 7 days)
- ☐ **DR0.4** VPS weekly DO Snapshot configured
- ☐ **DR0.5** All runbooks (Section 20) reviewed and validated by engineering team
- ☐ **DR0.6** DR test schedule documented and first test completed
- ☐ **DR0.7** Recovery time targets (RTO/RPO from Section 19.1) validated by actual DR test

25.7 Deployment

- ☐ **DEP0.1** All services pinned to specific image tags (not `latest`)
- ☐ **DEP0.2** Blue-green deployment procedure documented and tested
- ☐ **DEP0.3** Rollback procedure tested for both DGX (Coolify) and VPS (Dokploy)
- ☐ **DEP0.4** Staging environment mirrors production (separate Coolify/Dokploy projects)
- ☐ **DEP0.5** Secret validation script runs before every deployment
- ☐ **DEP0.6** Performance benchmarks collected on staging before every major release
- ☐ **DEP0.7** Schema migrations are backward-compatible and tested on staging

25.8 Performance

- ☐ **P0.1** Baseline benchmarks collected and recorded (Section 24.1)
- ☐ **P0.2** Correlation join buffer implemented (60s timeout)
- ☐ **P0.3** `camsvr` writes to local NVMe first (not directly to MinIO)
- ☐ **P0.4** `async-uploader` deployed and clearing local disk during session
- ☐ **P0.5** `frame-archiver` replaces `mc mirror` (per-object tracking in SQLite)
- ☐ **P0.6** DGX PostgreSQL insert batch size tuned (100ms window, 1000 rows max)

26. Appendices

26.1 Glossary

Term	Definition
AGX Orin	NVIDIA Jetson AGX Orin — edge AI compute module used as future remote inspection nodes
AOF	Append-Only File — Redis persistence mechanism that logs every write operation
camsvr	Camera Server — the MVIS service responsible for managing camera connections and triggering the pipeline
CUDA	Compute Unified Device Architecture — NVIDIA's GPU computing platform
DGX Spark	NVIDIA DGX Spark — desktop AI supercomputer with 4× RTX GPUs; primary inference node
DLQ	Dead Letter Queue — a stream where messages that fail processing repeatedly are sent
DO Spaces	DigitalOcean Spaces — S3-compatible object storage used for permanent frame archival
Dokploy	Self-hosted deployment platform on VPS (similar to Vercel/Railway)
frame_id	Unique identifier for a single captured frame; uses ULID format (time-sortable)
GPU OOM	GPU Out of Memory — error when GPU VRAM is exhausted, causing service crash
Headscale	Open-source, self-hosted implementation of the Tailscale control plane
MinIO	S3-compatible object storage server; used on DGX as a local frame cache
mTLS	Mutual TLS — both client and server present certificates; used for sync-engine ↔ VPS
MAXLEN	Redis Stream maximum length — older messages are trimmed when this limit is reached
OCR	Optical Character Recognition — reads text from frames (train numbers, bogie IDs, etc.)
PEL	Pending Entries List — Redis tracking of messages delivered but not yet ACKed
presigned URL	Temporary URL granting time-limited access to a private S3 object
RPO	Recovery Point Objective — maximum acceptable data loss measured in time
RTO	Recovery Time Objective — maximum acceptable downtime before service restoration
session_id	Unique identifier for one complete train inspection (start → stop → report)
SLI	Service Level Indicator — a measurable metric used to evaluate an SLO
SLO	Service Level Objective — a target value for an SLI
sync-engine	Service that batches DGX PostgreSQL rows and delivers them to VPS PostgreSQL
Tailscale	WireGuard-based mesh VPN connecting DGX and VPS on a private network
trace_id	Correlation ID assigned at frame capture; follows the frame through every pipeline stage
ULID	Universally Unique Lexicographically Sortable Identifier — time-ordered UUID alternative
VPS	Virtual Private Server — DigitalOcean droplet in BLR1 (Mumbai); management plane
WAL	Write-Ahead Log — PostgreSQL mechanism for crash recovery and replication
XACK	Redis command to acknowledge a message has been processed
XADD	Redis command to publish a new message to a stream
XAUTOCLAIM	Redis command to reclaim messages idle > N milliseconds from crashed consumers
XREADGROUP	Redis command to read messages from a stream as part of a consumer group
YOLO	You Only Look Once — real-time object detection model (YOLOv8x in production)

26.2 Acronyms

Acronym	Expansion
API	Application Programming Interface
AOF	Append-Only File
CUDA	Compute Unified Device Architecture
DGX	Data Center GPU (NVIDIA product line)
DLQ	Dead Letter Queue
DO	DigitalOcean
DR	Disaster Recovery
FPS	Frames Per Second
GPU	Graphics Processing Unit
HA	High Availability
IAM	Identity and Access Management
IST	Indian Standard Time
JWT	JSON Web Token
MAXLEN	Maximum Length
MinIO	Minimal Object Storage (open-source S3-compatible server)
mTLS	Mutual Transport Layer Security
MVIS	Machine Vision Inspection System
NVMe	Non-Volatile Memory Express (fast SSD interface)
OCR	Optical Character Recognition
OOM	Out of Memory
OTel	OpenTelemetry
PDF	Portable Document Format
PEL	Pending Entries List
PITR	Point-in-Time Recovery
RBAC	Role-Based Access Control
RDB	Redis Database (snapshot persistence format)
RPO	Recovery Point Objective
RTO	Recovery Time Objective
SLI	Service Level Indicator
SLO	Service Level Objective
SRE	Site Reliability Engineering
SSH	Secure Shell
TLS	Transport Layer Security
ULID	Universally Unique Lexicographically Sortable Identifier
UPS	Uninterruptible Power Supply
UUID	Universally Unique Identifier
VPS	Virtual Private Server

WAL	Write-Ahead Log
VPN	Virtual Private Network

26.3 Architecture Assumptions

- 1. DGX Spark has at minimum 1 TB NVMe storage dedicated to `/data/` (separate from OS drive)
- 2. DGX Spark has at minimum 4× RTX GPUs with CUDA 12.x drivers
- 3. DGX Spark has a stable internet connection for DO Spaces upload (minimum 100 Mbps sustained)
- 4. VPS is a DigitalOcean BLR1 droplet with minimum 4 vCPU, 8 GB RAM, 100 GB SSD
- 5. Tailscale latency between DGX and VPS is 3-8ms (measured in BLR1 environment)
- 6. Each camera provides JPEG frames at 50 FPS; frame resolution is ≤ 1080p
- 7. Average YOLO model inference time is ≤ 15ms per frame on a single RTX GPU with batch_size=8
- 8. Sessions are operator-initiated and operator-terminated (no automatic session management initially)
- 9. The `mvis.online` domain is managed in Cloudflare with DNS-only mode (no Cloudflare proxy)
- 10. DO Spaces BLR1 is the target object storage region; no cross-region replication required at Tier 1

26.4 Naming Conventions

Entity	Convention	Example
Container names	<code>{project}-{service}-{instance}</code>	<code>vandebharat-yolo-1</code>
Stream names	<code>snake_case</code>	<code>raw_frames</code> , <code>inspection_results</code>
Consumer group names	<code>{service}-group</code>	<code>yolo-group</code> , <code>correlation-group</code>
DLQ stream names	<code>{parent_stream}:dlq</code>	<code>detections:dlq</code>
MinIO object paths	<code>/sessions/{session_id}/{type}/{cam_id}/{seq:08d}.jpg</code>	<code>/sessions/abc/raw/cam_03/00047832.jpg</code>
Database tables	<code>snake_case</code>	<code>frames_metadata</code> , <code>ocr_results</code>
Database indexes	<code>idx_{table_short}_{column(s)}</code>	<code>idx_frames_session</code> , <code>idx_det_trace</code>
Subdomains	<code>{service}.mvis.online</code>	<code>api.mvis.online</code> , <code>cam.mvis.online</code>
Tailscale hostnames	<code>{device_type}-{id}</code>	<code>dgx-spark</code> , <code>agx-orin-1</code>
Environment variables	<code>SCREAMING_SNAKE_CASE</code>	<code>S3_ACCESS_KEY</code> , <code>VPS_POSTGRES_URL</code>
Docker volumes	<code>{project}_{purpose}</code>	<code>vandebharat_postgres_data</code>
Prometheus metric names	<code>mvis_{service}_{metric}_{unit}</code>	<code>mvis_yolo_frames_processed_total</code>
Log event names	<code>{service}.{action}.{result}</code>	<code>yolo.frame.processed</code> , <code>sync.batch.failed</code>

26.5 Version History

Version	Date	Author	Changes
1.0	2026-07-29	MVIS Engineering	Initial architecture document
2.0	2026-08-02	MVIS Engineering	Complete rewrite: added 17 new sections, production hardening, correlation join buffer, sync-engine state machine, local disk write path, DLQ, mTLS, RBAC, MAXLEN corrections, table partitioning, trace_id propagation, model_version tracking, Mermaid diagrams, all runbooks, ADRs, capacity planning, SLOs

27. AI Inference Layer — NVIDIA Triton Architecture

Section scope: This section supersedes the inference-related parts of Sections 5, 6, 7, 12, 16, 18, 20, 21, 24, and 25. All other sections remain unchanged. Read this section in full before implementing any GPU inference service.

27.1 DGX Spark Hardware Reality

Before designing the inference layer, the hardware must be understood precisely. The DGX Spark is **not** a multi-GPU system with discrete VRAM pools. It is a **Grace Blackwell Superchip** — a single integrated package.

Property	Value	Implication for Inference Design
GPU	1× NVIDIA GB10 Blackwell GPU	Single GPU; all model instances share one set of Tensor Cores
CPU	72-core ARM Neoverse V2 (Grace)	High-core-count CPU; preprocessing and Redis consumers scale well
Memory	128 GB LPDDR5X unified (not discrete VRAM)	CPU and GPU share the same physical memory pool via NVLink-C2C
CPU→GPU bandwidth	900 GB/s NVLink-C2C	Zero-copy between CPU and GPU — no H2D/D2H transfer overhead
FP8 Tensor Cores	~1 PFLOPS FP8	Native FP8 inference; ~7× faster than PyTorch FP32 for YOLO
FP16 Tensor Cores	~500 TFLOPS FP16	Recommended minimum precision for detection accuracy
TensorRT	10.x (Blackwell-optimized)	TRT FP8 with INT8 calibration is the target optimization path
CUDA	12.6+	Full support for CUDA graphs, streams, and MPS

Critical correction to earlier document sections: The DGX Spark does **not** have "4× RTX GPUs." It has one GB10 Blackwell GPU with unified memory. This changes the instance group strategy (see Section 27.6) and removes the possibility of naive multi-GPU data parallelism. The correct scaling primitive is **model instance concurrency on CUDA streams**, not multi-GPU replication.

Why unified memory changes everything:

In a traditional GPU system (e.g., DGX A100 with discrete 80 GB HBM), a frame read from disk must be copied Host → GPU memory before inference. On the GB10:

Traditional (A100):	DGX Spark (GB10):
CPU reads JPEG from disk	CPU reads JPEG from disk
↓ PCIe H2D (64 GB/s max)	↓ NVLink-C2C (900 GB/s, zero-copy)
GPU VRAM receives tensor	GPU reads from SAME memory
↓ Tensor Core inference	↓ Tensor Core inference
GPU VRAM stores result	CPU reads result from SAME memory
↓ PCIe D2H copy	
CPU receives result	
Effective H2D+D2H: ~50 GB/s	Effective: 900 GB/s, latency near zero

For MVIS at 600 FPS × ~200 KB/frame = 120 MB/s input rate — even traditional PCIe handles this. But for batch-8 (1.6 MB batch at 1080p): the H2D copy takes ~25 µs on PCIe vs <3 µs on NVLink-C2C. This compounds to ~15 ms saved per second — meaningful at scale.

27.2 Current Inference Bottleneck Analysis

The current architecture runs inference as standalone Docker containers — `vandebharat-yolo-1` and `vandebharat-ocr-1` — each containing a PyTorch/Ultralytics model loaded at startup and a Redis consumer loop.

Bottleneck 1: Sequential Frame Processing

```
Consumer reads 1 message from Redis (XREADGROUP)
→ decode JPEG (CPU)
→ model.predict(frame)      ← GPU busy for ~15-60ms
→ publish results to stream ← GPU idle during this
→ XACK
→ reads next message
```

During the `publish → XACK → read next` segment (~2-5ms on Redis), the GPU is completely idle. At 15ms inference + 5ms Redis overhead, GPU utilization is capped at $15/(15+5) = 75\%$ even in a perfect world. In practice it is 40-60% because of Python GIL contention and decode overhead.

Bottleneck 2: No Batching

PyTorch with Ultralytics' default API processes one frame per `model.predict()` call unless explicitly batched. A single-frame forward pass uses a fraction of the available Tensor Cores — the remainder sit idle. On a Blackwell GPU with 1 PFLOPS of FP8 compute, running batch=1 YOLO uses perhaps 5% of peak throughput.

Batch size impact on GPU utilization (YOLOv8x, Blackwell):

Batch Size	Inference Time	GPU Tensor Core Util	Throughput
1	~12ms	~8%	83 FPS
4	~14ms	~30%	286 FPS
8	~18ms	~55%	444 FPS
16	~22ms	~80%	727 FPS
32	~35ms	~90%	914 FPS

The target (600 FPS) requires at least batch=8 to remain within GPU capacity, and batch=16 gives comfortable headroom.

Bottleneck 3: Separate CUDA Contexts Per Container

Each Docker container that uses CUDA initializes its own CUDA context. With `vandebharat-yolo-1` and `vandebharat-ocr-1` both running on the same GPU:

- 2 independent CUDA contexts compete for GPU compute scheduling
- Context switching adds latency (NVIDIA Multi-Process Service can help but is not configured)
- Each context has its own memory allocator — no sharing, potential fragmentation
- GPU memory manager cannot optimize allocations across both models

Bottleneck 4: PyTorch Runtime Overhead

PyTorch at inference time carries:

- Python interpreter overhead per call
- Autograd graph construction (unless `torch.no_grad()` is explicitly used — often forgotten)
- JIT tracing overhead (unless `torch.jit.script` is used)
- Dynamic shape handling even when all frames are the same resolution

Compared to a TensorRT engine compiled for a fixed input shape, PyTorch FP32 is typically 3-7× slower for YOLO-class models on NVIDIA hardware.

Bottleneck 5: Model Loading Overhead

On container restart, each service reloads the full model from disk:

- YOLOv8x: ~150MB weights → load + CUDA init: 10-30 seconds
- During this window: pipeline stalls, Redis backlog grows
- No model warmup — first inference after restart is always slow (JIT compilation)

Bottleneck 6: CPU Decode on Single Thread

JPEG decoding is CPU-bound. At 600 FPS × 200KB/frame = 120 MB/s of JPEG data requiring decode. The current service decodes frames one at a time in a single Python thread. On a 72-core Grace CPU, this wastes the available parallelism.

Summary — Current Effective Throughput Ceiling

```
Measured bottleneck chain (worst-case on current architecture):
Redis XREADGROUP:      2ms
JPEG decode (1 thread): 3ms
H2D transfer:          <0.5ms (NVLink-C2C benefit)
PyTorch inference (bs=1): 15-20ms
Result publish + XACK:  2ms

Total per-frame: ~22-27ms → max throughput: ~37-45 FPS (single container)

Current 12-camera × 50 FPS = 600 FPS demand vs 37-45 FPS capacity
→ The system is running at 6-16× over the sustainable limit of a single container.
```

This explains why multiple consumer instances would be needed currently, and why the architecture needs a fundamentally different approach.

27.3 Why Triton — First-Principles Evaluation

Before accepting Triton, every alternative was evaluated against the MVIS requirements: 600 FPS sustained, single DGX Spark (GB10), production reliability, operational simplicity.

Alternative A: TensorRT Direct (no inference server)

Convert YOLO and OCR to TRT engines. Call `trt_runtime.execute_async_v3()` directly from a Python service using CUDA streams.

Performance: Maximum possible — no server overhead, direct kernel dispatch. **Problem:** You must implement everything Triton gives you for free: batching logic, concurrent request queuing, model versioning, health endpoints, metrics, graceful reload, memory management. This is 3-6 months of infrastructure work to maintain forever. Any NVIDIA fix to TRT batching requires reimplementing on your side. Rejected — operational cost too high.

Alternative B: TorchServe

PyTorch's native model server.

Positives: Simple integration with PyTorch models. **Problems:** No TRT backend. Dynamic batching is basic compared to Triton. No ensemble pipelines. No DCGM integration. Primarily CPU-oriented in design. Performance on GPU is 40-60% of equivalent Triton TRT config. Rejected — performance ceiling too low.

Alternative C: Ray Serve

Distributed model serving, excellent for multi-node.

Positives: Horizontal scaling across machines without Kubernetes. Python-native. **Problems:** Not designed for single-GPU, single-node high-FPS scenarios. Ray's actor model adds ~1-3ms overhead per inference call vs Triton's sub-millisecond gRPC dispatch. Designed for multi-model routing across many machines — overkill for Tier 1, correct for Tier 3. **Add to roadmap for Tier 3 (national deployment) as the inference routing layer above Triton.** Rejected for current phase.

Alternative D: BentoML

Modern ML serving platform.

Positives: Good developer experience, cloud-native. **Problems:** No native TRT backend. No ensemble pipelines. Lacks Triton's dynamic batching sophistication. Rejected.

Alternative E: ONNX Runtime (ORT)

Positives: Cross-platform, supports TRT execution provider. **Problems:** When using TRT execution provider, ORT wraps TRT — you get TRT performance with ORT overhead on top. Triton's TRT backend calls TRT directly. ORT adds ~0.5-2ms overhead per call. Not significant at low FPS but measurable at 600 FPS. Rejected in favor of native TRT via Triton.

Decision: NVIDIA Triton Inference Server

Why Triton wins on every axis that matters for MVIS:

- 1. **Native TRT backend** — calls TensorRT engines directly. No wrapper overhead. Full Blackwell FP8 support through TRT 10.x.
- 2. **Dynamic batching built-in** — configurable per model. MVIS gets batch=16 for YOLO without a single line of batching code.
- 3. **Model instances on CUDA streams** — Triton manages multiple instances of the same model on separate CUDA streams, enabling true concurrent execution on the GB10 GPU.
- 4. **Ensemble/BLS pipelines** — preprocessing → inference → postprocessing can run server-side, eliminating 2-4 Redis round-trips per frame.
- 5. **Shared CUDA context** — all models loaded in Triton share one CUDA context. No multi-context competition. GPU scheduler has full visibility over all in-flight work.
- 6. **DCGM integration** — native. Triton exports per-model inference latency, queue depth, batch size, and GPU utilization via Prometheus. No custom metrics code.
- 7. **Hot model reload** — `POST /v2/repository/models/{name}/load` reloads a model without restarting the server. Model updates without pipeline downtime.
- 8. **Built-in model versioning** — model repository holds multiple versions. Traffic routing between versions without configuration changes.
- 9. **Rate limiting and request scheduling** — Triton's `--rate-limit` flag prevents GPU OOM from runaway batch queues.
- 10. **Active industry use** — NVIDIA's reference design for DGX inference workloads. Bugs fixed by NVIDIA, not by the MVIS team.

27.4 Kubernetes vs Docker — Definitive Evaluation

This decision affects operational cost for the lifetime of the project. It must be made honestly.

Evaluation Matrix

Criterion	Docker Compose (Coolify)	Docker Swarm	K3s (Lightweight K8s)	Full Kubernetes	Triton-only
Operational complexity	Very Low	Low	Medium	High	Low
Single-DGX suitability	Excellent	Good	Overkill	Overkill	N/A
Multi-DGX (3-5 nodes)	Requires manual coordination	Adequate	Good	Good	N/A
Multi-station (10+ nodes)	Inadequate	Inadequate	Excellent	Excellent	N/A
GPU scheduling	Via Coolify labels	Basic	NVIDIA GPU Operator	NVIDIA GPU Operator	Manual
Time to first deployment	Hours	Days	Days-weeks	Weeks	Hours
Maintenance overhead	Minimal	Low	Medium	High	Low
Rolling deployments	Via Coolify	Manual	Built-in	Built-in	Manual
HA for services	None (single node)	Partial	Full	Full	None
Monitoring integration	Prometheus (manual)	Prometheus (manual)	Prometheus Operator	Prometheus Operator	Triton metrics
Recovery from node failure	Manual	Semi-auto	Auto	Auto	Manual
Team expertise required	Minimal	Low	Docker + K8s basics	K8s expertise	Minimal
Estimated ops cost/month	~0 hours	~2 hours	~5 hours	~15 hours	~0 hours

Recommendation by Tier

Tier 1 — Single DGX (current): Docker Compose managed by Coolify

Kubernetes on a single node is theater. It adds 500+ MB RAM overhead, requires mastery of CRDs, RBAC, and NVIDIA GPU Operator to do what Coolify does in a web UI. The only material benefit — self-healing pod restarts — is replicated by Docker `restart: unless-stopped`. **Do not use Kubernetes for a single DGX.**

Tier 2 — 2-4 DGX nodes: K3s with NVIDIA GPU Operator

At two DGX nodes, Docker Compose with Coolify becomes coordination overhead without automation. K3s (< 100 MB RAM overhead per node) with NVIDIA GPU Operator gives:

- GPU-aware pod scheduling (`resource: nvidia.com/gpu: 1`)
- Triton deployed as a Deployment with node affinity per DGX
- Automatic pod restart and health checking
- Single `kubectl` control plane across all nodes
- Metrics Server for autoscaling experiments

Tier 3 — 10+ DGX, multiple stations: K3s clusters, one per station + central management

One K3s cluster per inspection station (3+ nodes each for HA control plane). Central VPS hosts Rancher or Flux for multi-cluster GitOps. Triton is a Deployment per cluster, exposed via internal LoadBalancer. Redis consumers as Deployments. Model repository on shared NFS or per-cluster MinIO.

Summary

```
Tier 1 (now):          Docker Compose + Coolify + Triton container
Tier 2 (6-12 months): K3s + NVIDIA GPU Operator + Triton Deployment
Tier 3 (12-24 months): K3s clusters per station + Rancher + Triton + NATS JetStream
```

27.5 Redesigned Inference Architecture

```

graph TB
    subgraph CAMERAS["Camera Array - 12x @ 50 FPS"]
        C[Cameras]
    end

    subgraph DGX["DGX Spark - GB10 Grace Blackwell (128 GB Unified Memory)"]
        direction TB

        subgraph CAPTURE["Capture Layer"]
            CAMSVR["camsvr\n(session mgmt)"]
            NVME["/data/capture/\n(local NVMe write buffer)"]
        end

        subgraph TRITON_BLOCK["Triton Inference Server\n(localhost:8000 HTTP | :8001 gRPC | :8002 metrics)"]
            direction TB
            subgraph MODEL_REPO["Model Repository"]
                PREPROC_M["preprocessing\n(Python backend)\n2 instances"]
                YOLO_M["yolov8x_trt\n(TensorRT FP8)\n4 instances\ndynamic batching: [8,16,32]"]
                OCR_M["paddleocr_trt\n(TensorRT FP16)\n2 instances\ndynamic batching: [4,8]"]
                ENSEMBLE_M["mvis_detect_pipeline\n(BLS ensemble)\npreproc→yolo→nms"]
            end
            GPU["GB10 Blackwell GPU\n(CUDA streams managed by Triton)"]
            PREPROC_M & YOLO_M & OCR_M & ENSEMBLE_M → GPU
        end

        subgraph CONSUMERS["Triton Consumer Services\n(thin Redis→Triton adapters)"]
            DETECT_CONS["detect-consumer\n(async gRPC, 64 in-flight)\nreads preprocessed_frames\n→ Triton mvis_detect_pipeline\n→ publishes detections"]
            OCR_CONS["ocr-consumer\n(async gRPC, 32 in-flight)\nreads preprocessed_frames\n→ Triton paddleocr_trt\n→ publishes ocr_results"]
            ASYNC_UP["async-uploader\n(reads raw_frames\n→ MinIO)"]
        end

        subgraph REDIS_DGX["Redis Streams (AOF enabled)"]
            S_RAW["raw_frames\nMAXLEN 10,000"]
            S_PRE["preprocessed_frames\nMAXLEN 5,000"]
            S_DET["detections\nMAXLEN 50,000"]
            S_OCR["ocr_results\nMAXLEN 20,000"]
            S_INSP["inspection_results\nMAXLEN 5,000"]
        end

        subgraph DOWNSTREAM["Downstream Services"]
            CORR["correlation\n(join buffer)"]
            PERSIST["result-persister"]
            PGDGX["PostgreSQL\n(hot buffer)"]
        end

        ASYNC_PREP["async-preprocessor\n(72-core Grace CPU)\n→ 8 parallel decode workers\n→ XADD preprocessed_frames\n(frame as local path,\nnot JPEG bytes)"]

        CAMERAS → CAMSVR
        CAMSVR → NVME
        CAMSVR → S_RAW
        NVME → ASYNC_UP
        NVME → ASYNC_PREP
        S_RAW → ASYNC_PREP
        ASYNC_PREP → S_PRE
        S_PRE → DETECT_CONS & OCR_CONS & ASYNC_UP

        DETECT_CONS -- "async gRPC\n64 concurrent" → TRITON_BLOCK
        OCR_CONS -- "async gRPC\n32 concurrent" → TRITON_BLOCK

        DETECT_CONS → S_DET
        OCR_CONS → S_OCR
        S_DET → CORR
        S_OCR → CORR
        CORR → S_INSP
        S_INSP → PERSIST
        PERSIST → PGDGX

        style TRITON_BLOCK fill:#1a3a1a,stroke:#4aff4a,color:#ffffff
        style DGX fill:#1a1a2e,stroke:#4a90d9,color:#ffffff
        style CAMERAS fill:#1a0a0a,stroke:#d94a4a,color:#ffffff
    end

```

What changed from the previous design

Before	After	Reason
<code>vandebharat-yolo-1</code> (PyTorch, batch=1)	Triton <code>yolov8x_trt</code> (TRT FP8, dynamic batching)	5-8× throughput increase
<code>vandebharat-ocr-1</code> (PyTorch, batch=1)	Triton <code>paddleocr_trt</code> (TRT FP16, dynamic batching)	3-4× throughput increase
<code>vandebharat-preprocessing-1</code> (GPU, sequential)	<code>async-preprocessor</code> (CPU, 8 parallel workers)	Frees GPU for inference; Grace CPU handles decode efficiently
YOLO and OCR each have their own CUDA context	All models share one CUDA context in Triton	Eliminates context switching; Triton's scheduler has full GPU visibility
Consumer reads 1 frame → inference → publishes → next	Consumer holds 64 in-flight async gRPC calls	Natural batch filling; GPU stays fed continuously
Model reload = container restart (30s downtime)	Triton hot reload (< 1s, no pipeline gap)	Zero-downtime model updates
No model version tracking in inference path	Triton returns <code>model_version</code> in response	<code>model_version</code> field in DB is populated automatically from Triton response

27.6 Model Repository Design

```
/models/                                     ← Triton model repository root
├── preprocessing_python/                     ← CPU preprocessing (Python backend)
│   ├── config.pbtxt
│   └── 1/
│       └── model.py                         ← JPEG decode, resize, normalize
├── yolov8x_trt/                             ← YOLOv8x TensorRT FP8 engine
│   ├── config.pbtxt
│   └── 1/
│       └── model.plan                     ← compiled by trtexec (see 27.8)
├── paddleocr_trt/                           ← PaddleOCR TensorRT FP16
│   ├── config.pbtxt
│   └── 1/
│       └── model.plan
├── nms_postprocess_python/                  ← NMS + bbox decode (Python backend)
│   ├── config.pbtxt
│   └── 1/
│       └── model.py
└── mvis_detect_pipeline/                    ← BLS ensemble (optional, see 27.7)
    ├── config.pbtxt
    └── 1/
        └── model.py                      ← BLS orchestration script
```

yolov8x_trt config.pbtxt


```

name: "yolov8x_trt"
backend: "tensorrt"
max_batch_size: 32

input [
  {
    name: "images"
    data_type: TYPE_FP8_E4M3      # Blackwell native FP8
    dims: [3, 640, 640]          # CHW format, fixed shape
  }
]

output [
  {
    name: "output0"
    data_type: TYPE_FP32
    dims: [-1, 84]                # [num_detections, 4_bbox + 80_classes]
  }
]

instance_group [
  {
    count: 4                      # 4 concurrent model instances on 1 GPU
    kind: KIND_GPU
    gpus: [0]                     # GB10 is device 0
  }
]

dynamic_batching {
  preferred_batch_size: [8, 16, 32]
  max_queue_delay_microseconds: 5000 # wait up to 5ms to fill preferred batch
  priority_levels: 2
  default_priority_level: 1
}

# CUDA stream priority: YOLO is higher priority than OCR
# (detections are on the critical path for anomaly flagging)
optimization {
  priority: PRIORITY_MAX
  cuda {
    graphs: true                  # CUDA graphs: eliminate CPU launch overhead
  }
}

```

paddleocr_trt config.pbtxt

```

name: "paddleocr_trt"
backend: "tensorrt"
max_batch_size: 16

input [
  {
    name: "x"
    data_type: TYPE_FP16
    dims: [3, 48, 320]          # OCR text line crops (height=48, variable width padded to 320)
  }
]

output [
  {
    name: "softmax_2.tmp_0"
    data_type: TYPE_FP32
    dims: [-1, 97]             # [sequence_len, num_characters]
  }
]

instance_group [
  {
    count: 2
    kind: KIND_GPU
    gpus: [0]
  }
]

dynamic_batching {
  preferred_batch_size: [4, 8, 16]
  max_queue_delay_microseconds: 8000 # OCR is not on the critical latency path
}

```

preprocessing_python config.pbtxt

```

name: "preprocessing_python"
backend: "python"
max_batch_size: 32

input [
  {
    name: "raw_image_bytes"
    data_type: TYPE_BYTES
    dims: [-1]          # variable-length JPEG bytes
  }
]

output [
  {
    name: "preprocessed"
    data_type: TYPE_FP8_E4M3
    dims: [3, 640, 640]
  }
]

instance_group [
  {
    count: 2
    kind: KIND_CPU          # preprocessing on Grace CPU (72 cores)
  }
]

# No dynamic batching needed – CPU scales with thread count

```

27.7 Inference Pipeline: Two Design Options

Option A: Triton BLS Ensemble (Recommended for Phase E+)

A Business Logic Scripting (BLS) model orchestrates the full detection pipeline server-side. The detect-consumer sends raw JPEG bytes to Triton and receives fully decoded detections. The BLS model calls preprocessing → YOLO → NMS internally in a single server-side Python function.

```
# /models/mvis_detect_pipeline/1/model.py
import triton_python_backend_utils as pb_utils
import numpy as np
import json

class TritonPythonModel:
    def execute(self, requests):
        responses = []
        for request in requests:
            raw_bytes = pb_utils.get_input_tensor_by_name(request, "raw_image_bytes")

            # Step 1: preprocessing (server-side call, no network hop)
            preproc_req = pb_utils.InferenceRequest(
                model_name="preprocessing_python",
                requested_output_names=["preprocessed"],
                inputs=[raw_bytes],
            )
            preproc_resp = preproc_req.exec(decoupled=False)
            preprocessed = pb_utils.get_output_tensor_by_name(preproc_resp, "preprocessed")

            # Step 2: YOLO inference (server-side call)
            yolo_req = pb_utils.InferenceRequest(
                model_name="yolov8x_trt",
                requested_output_names=["output0"],
                inputs=[preprocessed],
            )
            yolo_resp = yolo_req.exec(decoupled=False)
            raw_output = pb_utils.get_output_tensor_by_name(yolo_resp, "output0")

            # Step 3: NMS postprocessing (CPU, server-side)
            detections = self._nms_decode(raw_output.as_numpy())

            output_tensor = pb_utils.Tensor(
                "detections_json",
                np.array([json.dumps(detections)], dtype=object),
            )
            responses.append(pb_utils.InferenceResponse([output_tensor]))
        return responses
```

Benefits of BLS:

- Eliminates `preprocessed_frames` Redis stream entirely — preprocessing happens inside Triton
- detect-consumer sends raw JPEG bytes, receives structured JSON detections
- Reduces Redis throughput requirement significantly
- Triton batches across the entire pipeline, not just at the YOLO boundary

Trade-off: BLS models are harder to debug and profile. Implement in Phase E after core pipeline is stable.

Option B: Two-Stage Consumer (Recommended for Phase A-D, Current Implementation)

The `preprocessed_frames` Redis stream remains. The detect-consumer reads preprocessed frames (local path or tensor), calls Triton YOLO asynchronously, publishes results. This is the lower-risk approach for initial deployment.

```
# detect-consumer service (thin Redis→Triton adapter)
import asyncio
import tritonclient.grpc.aio as grpcclient
import numpy as np

MAX_INFLIGHT = 64 # saturates YOLO dynamic batching at 600 FPS

class DetectConsumer:
    def __init__(self):
        self.triton = grpcclient.InferenceServerClient("localhost:8001")
        self.semaphore = asyncio.Semaphore(MAX_INFLIGHT)

    async def run(self):
        while True:
            # Read up to 32 frames per poll - fills batches faster
            messages = self.redis.xreadgroup(
                "preprocessed_frames", "detect-group",
                "detect-consumer-1", count=32, block=50
            )
            if not messages:
                continue

            tasks = [self._infer_one(msg) for msg in messages]
            await asyncio.gather(*tasks)

    async def _infer_one(self, msg):
        async with self.semaphore:
            frame_path = msg["local_path"]
            frame_data = load_frame_as_fp8_tensor(frame_path) # uses unified memory

            inputs = [grpcclient.InferInput("images", frame_data.shape, "FP8_E4M3")]
            inputs[0].set_data_from_numpy(frame_data, binary_data=True)

            # Non-blocking - Triton queues this request and batches with others in-flight
            result = await self.triton.infer(
                model_name="yolov8x_trt",
                inputs=inputs,
                outputs=[grpcclient.InferRequestedOutput("output0")],
                request_id=msg["frame_id"],
            )

            detections = decode_yolo_output(
                result.as_numpy("output0"),
                conf_threshold=0.45,
                iou_threshold=0.65,
            )

            for det in detections:
                self.redis.xadd("detections", {
                    "trace_id": msg["trace_id"],
                    "frame_id": msg["frame_id"],
                    "session_id": msg["session_id"],
                    "model_version": result.get_response().model_version,
                    **det,
                })

            self.redis.xack("preprocessed_frames", "detect-group", msg["id"])

```

Why 64 in-flight requests is the key number:

With `MAX_INFLIGHT = 64` and the Triton batcher configured for `preferred_batch_size: [16, 32]`:

- detect-consumer submits 64 async gRPC calls simultaneously
- Triton sees 64 pending requests and forms 2× batch-32 groups
- Both batch-32 groups run on YOLO instance 1 and 2 concurrently (CUDA streams)
- GPU Tensor Core utilization: ~85-92% (Blackwell fills well at batch=32)
- Effective throughput: $2 \times (32 / 35\text{ms}) = 1,828 \text{ FPS capacity}$ vs 600 FPS demand → 3× headroom

27.8 TensorRT Engine Compilation

TRT engines must be compiled for the target hardware (GB10 Blackwell). An engine compiled on an A100 will not run on a GB10, and vice versa. Engines must be compiled **on the DGX Spark itself**.

Compile YOLOv8x to TRT FP8 Engine

```
# Step 1: Export YOLOv8x to ONNX with fixed input shape
python -c "
from ultralytics import YOLO
model = YOLO('yolov8x.pt')
model.export(format='onnx', imgsz=640, opset=17, simplify=True, dynamic=False, half=False)
"

# Step 2: Build TensorRT FP8 engine (Blackwell-native)
# Must run ON the DGX Spark (engine is GPU-specific)
trtexec \
  --onnx=yolov8x.onnx \
  --saveEngine=models/yolov8x_trt/1/model.plan \
  --fp8 \
  --minShapes=images:1x3x640x640 \
  --optShapes=images:16x3x640x640 \ # optimize for batch=16
  --maxShapes=images:32x3x640x640 \
  --useCudaGraph \
  --useSpinWait \
  --separateProfileRun \
  --exportLayerInfo=/tmp/yolo_layer_info.json \
  --exportProfile=/tmp/yolo_profile.json

# Step 3: Verify engine
trtexec \
  --loadEngine=models/yolov8x_trt/1/model.plan \
  --shapes=images:16x3x640x640 \
  --warmUp=500 \
  --iterations=1000 \
  --percentile=99

# Expected output on GB10 Blackwell (FP8):
# Latency: min=8ms, max=12ms, mean=9.2ms, median=9.0ms, 99th=11.5ms
# Throughput: batch_16 / 9ms = 1,777 FPS effective
```

INT8 Calibration (Alternative to FP8)

If FP8 accuracy is insufficient for a specific detection class, fall back to INT8 with calibration:

```
# Build calibration dataset from representative frames
python scripts/build_calibration_cache.py \
  --input-dir /data/calibration-frames/ \
  --output-cache /tmp/yolo_calibration.cache \
  --batch-size 16 --num-batches 100

trtexec \
  --onnx=yolov8x.onnx \
  --saveEngine=models/yolov8x_trt/1/model.plan \
  --int8 \
  --calib=/tmp/yolo_calibration.cache \
  --optShapes=images:16x3x640x640 \
  --maxShapes=images:32x3x640x640
# ~15% lower throughput than FP8, ~2% higher mAP accuracy
```

Compile PaddleOCR to TRT FP16

```
# Export PaddleOCR recognition model to ONNX
python -c "
import paddle
from paddleocr import PaddleOCR
# Export rec model to ONNX
# (specific command depends on PaddleOCR version)
"

trtexec \
  --onnx=paddleocr_rec.onnx \
  --saveEngine=models/paddleocr_trt/1/model.plan \
  --fp16 \
  # FP16 (FP8 may reduce character recognition accuracy)
  --minShapes=x:1x3x48x320 \
  --optShapes=x:8x3x48x320 \
  --maxShapes=x:16x3x48x320
```

Engine Versioning and Management

```
# Model repository supports multiple engine versions
/models/yolov8x_trt/
  1/ ← current production engine (FP8, v2.1.0)
    model.plan
  2/ ← staged upgrade (FP8, v2.2.0, pending validation)
    model.plan

# config.pbtxt version_policy controls which version is active:
version_policy { latest { num_versions: 1 } } # serve only latest
version_policy { specific { versions: [1] } } # serve specific version
version_policy { all {} } # serve all (for A/B testing)

# Hot-swap to version 2 without restart:
curl -X POST http://localhost:8000/v2/repository/models/yolov8x_trt/load
# Update config.pbtxt to point to version 2, Triton reloads in < 1 second
```

27.9 Model Instance Groups — Optimal Configuration for GB10

Why instance count matters on a single GPU

On the GB10 (single physical GPU), multiple model instances do **not** run on separate GPUs. They share the same compute units. Their value comes from:

1. **Hiding latency:** While instance 1 is running the forward pass, instance 2 is loading its input batch into memory. The GPU compute and memory transfer pipelines are overlapped.
2. **Filling the GPU pipeline:** A single model instance leaves gaps in the CUDA instruction queue (between batches). Multiple instances keep the queue fuller.
3. **Concurrency for mixed workloads:** YOLO and OCR can run simultaneously on separate CUDA streams if each has at least 1 instance.

Over-provisioning penalty

More instances is not always better. Each model instance occupies persistent GPU memory (activations + KV cache for transformer models, activation buffers for CNNs). On the GB10:

- YOLOv8x TRT FP8 per-instance memory: ~2.5 GB (weights + activation buffers at batch=32)
- At 4 instances: $4 \times 2.5 \text{ GB} = 10 \text{ GB}$
- At 8 instances: $8 \times 2.5 \text{ GB} = 20 \text{ GB}$
- Beyond ~4 instances on a single GPU: instances wait for compute rather than executing, adding latency without throughput gain

For a workload of 600 FPS target:

- Required throughput per YOLO instance at batch=16: $16 \text{ frames} / 9\text{ms} = 1,777 \text{ FPS}$
- 1 YOLO instance covers 600 FPS with $3\times$ headroom
- 2 YOLO instances: recommended for redundancy (if instance 1 hits a batch timeout, instance 2 absorbs)

- 4 YOLO instances: maximum recommended for current load, provides buffer for future traffic increase

Recommended Instance Configuration

```
# yolov8x_trt
instance_group [{ count: 4, kind: KIND_GPU, gpus: [0] }]
# Memory: 4 × 2.5 GB = 10 GB
# Throughput: ~7,000 FPS capacity at batch=16
# Target: 600 FPS → 11× headroom (comfortable)

# paddleocr_trt
instance_group [{ count: 2, kind: KIND_GPU, gpus: [0] }]
# Memory: 2 × 1.5 GB = 3 GB
# Throughput: ~3,000 FPS capacity at batch=8

# preprocessing_python
instance_group [{ count: 8, kind: KIND_CPU }]
# 8 CPU workers on 72-core Grace CPU → negligible overhead
# Each handles JPEG decode + resize in parallel

# Total GPU memory allocation:
# YOLO: 10 GB
# OCR: 3 GB
# Triton server overhead: ~2 GB
# Total: ~15 GB of 128 GB unified memory
# Remaining 113 GB: available for OS, Redis, PostgreSQL, future models
```

27.10 CUDA Execution Strategy

CUDA Graphs

Enabled in `yolov8x_trt` config (`cuda { graphs: true }`). CUDA graphs capture the sequence of GPU operations for a fixed batch size and replay them as a single GPU command. Benefits:

- Eliminates CPU-GPU synchronization overhead on repeated identical calls
- Reduces kernel launch latency from ~10µs to < 1µs per kernel
- Mandatory for sustained high-FPS inference

Triton handles CUDA graph capture automatically on the first N inference calls (warmup period). **Warmup Triton before accepting production traffic:**

```
# Warm up Triton - trigger CUDA graph capture for each batch size
for batch in 1 4 8 16 32; do
  python scripts/triton_warmup.py --model yolov8x_trt --batch-size $batch --iterations 20
done
```

CUDA Streams

Triton manages multiple CUDA streams internally — one per model instance by default. This enables:

- **Stream 1:** YOLO instance 1 — forward pass on batch A
- **Stream 2:** YOLO instance 2 — loading batch B into GPU memory (overlap with stream 1)
- **Stream 3:** OCR instance 1 — forward pass concurrently with YOLO

The GB10's CUDA scheduler executes these streams concurrently to the extent that compute units permit. YOLO typically occupies ~40% of SM compute during its peak; OCR occupies ~20%. Running both simultaneously achieves ~60% combined utilization — better than sequencing them.

NVIDIA Multi-Process Service (MPS)

MPS is NOT recommended for MVIS. MPS is designed for multi-process workloads where each process has its own CUDA context (e.g., multiple ML training jobs). Since Triton consolidates all inference into one CUDA context, MPS adds complexity without benefit. The Triton single-context approach is superior.

Pinned Memory vs Unified Memory

On the GB10, both CPU and GPU directly access the 128 GB unified pool. Traditional `cudaMallocHost` (pinned memory) tricks to speed up H2D copies are irrelevant — all memory is already accessible to both. However, for Triton's gRPC ingestion path (frame data arriving via network), Triton does allocate pinned memory for its input staging buffers. This is handled automatically by Triton when `--pinned-memory-pool-byte-size` is set:

```
docker run ... tritonserver \
  --pinned-memory-pool-byte-size=2147483648 \ # 2 GB pinned staging buffer
  --cuda-memory-pool-byte-size=0:10737418240 \ # 10 GB reserved GPU-side pool
  ...
```

27.11 Updated Data Flows with Triton

Updated Flow 1 — Active Session (with Triton)

```
sequenceDiagram
    participant CAMSVR as camsvr
    participant NVME as Local NVMe
    participant REDIS as Redis Streams
    participant PREP as async-preprocessor\n(8 CPU workers, Grace)
    participant DETECT as detect-consumer\n(async gRPC)
    participant OCR_C as ocr-consumer\n(async gRPC)
    participant TRITON as Triton Inference Server\n(localhost:8001)
    participant PERS as result-persister

    CAMSVR->>NVME: Write /data/capture/{seq}.jpg (< 1ms NVMe)
    CAMSVR->>REDIS: XADD raw_frames {local_path, trace_id, seq}

    par 8 parallel CPU decode workers
        PREP->>REDIS: XREADGROUP raw_frames preproc-group
        PREP->>PREP: Decode JPEG + resize to 640x640 (Grace CPU, parallel)
        PREP->>REDIS: XADD preprocessed_frames {local_path_640, trace_id}
        PREP->>REDIS: XACK raw_frames
    end

    par detect-consumer (64 concurrent in-flight)
        DETECT->>REDIS: XREADGROUP preprocessed_frames detect-group (count=32)
        DETECT->>TRITON: 32x async gRPC InferRequest (yolov8x_trt)
        Note over TRITON: dynamic_batching accumulates 32 requests\nforms batch_32 → TRT forward pass ~35ms\n→ returns all 32 results
        TRITON->>DETECT: 32x detection arrays + model_version
        DETECT->>REDIS: 32x XADD detections
        DETECT->>REDIS: 32x XACK preprocessed_frames
    and ocr-consumer (32 concurrent in-flight)
        OCR_C->>REDIS: XREADGROUP preprocessed_frames ocr-group (count=16)
        OCR_C->>TRITON: 16x async gRPC InferRequest (paddleocr_trt)
        TRITON->>OCR_C: 16x OCR result arrays + model_version
        OCR_C->>REDIS: 16x XADD ocr_results
        OCR_C->>REDIS: 16x XACK preprocessed_frames
    end

    PERS->>REDIS: XREADGROUP inspection_results
    PERS->>PERS: Batch INSERT to PostgreSQL (100ms window)
```

Key throughput math:

```
detect-consumer: 64 in-flight × (batch fills at batch=32 every 35ms)
  → submitting 64/35ms = 1,828 gRPC req/s → 1,828 FPS capacity
  → actual demand: 600 FPS → 33% GPU utilization for YOLO

OCR consumer: 32 in-flight × (batch fills at batch=16 every 15ms)
  → 32/15ms = 2,133 gRPC req/s → capacity >> demand

YOLO + OCR combined GPU utilization target: 55–70%
Remaining headroom: available for model version upgrades, future cameras
```


27.12 Async Preprocessing on Grace CPU

The 72-core Grace CPU in the GB10 is the fastest ARM CPU NVIDIA has built. Dedicating 8 cores to JPEG decode and resize is trivially cheap.

```
# async-preprocessor service
# Runs 8 parallel workers using ProcessPoolExecutor (avoids Python GIL)
import concurrent.futures
import cv2
import numpy as np

WORKERS = 8          # 8 of 72 Grace cores dedicated to preprocessing
IMG_SIZE = 640       # YOLO input size

executor = concurrent.futures.ProcessPoolExecutor(max_workers=WORKERS)

def preprocess_frame(local_path: str) → np.ndarray:
    """Decode JPEG, resize, normalize to FP8 tensor. Runs in worker process."""
    img = cv2.imread(local_path)
    img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    tensor = img.astype(np.float32) / 255.0
    tensor = np.transpose(tensor, (2, 0, 1)) # HWC → CHW
    # Convert to FP8 (numpy doesn't have native FP8 – use float16 then let TRT handle)
    tensor = tensor.astype(np.float16)
    return tensor

# In the consumer loop:
futures = {
    executor.submit(preprocess_frame, msg["local_path"]): msg
    for msg in messages
}
for future in concurrent.futures.as_completed(futures):
    msg = futures[future]
    preprocessed_path = save_tensor_to_nvme(future.result(), msg["session_id"], msg["seq_num"])
    redis.xadd("preprocessed_frames", {
        **msg,
        "preprocessed_path": preprocessed_path,
    })
```

Why CPU preprocessing, not GPU:

GPU-side preprocessing (the previous `vandebharat-preprocessing-1` container) occupies a GPU CUDA stream for decode + resize — operations that the 72-core Grace CPU handles faster with no opportunity cost. The GB10 unified memory means the CPU-preprocessed tensor is instantly visible to the GPU at zero copy cost. Freeing the GPU stream for inference is strictly better.

27.13 Triton Deployment Configuration

Docker Compose service definition

```
# In Coolify / docker-compose.yml on DGX

services:
  triton:
    image: nvcr.io/nvidia/tritonserver:24.12-py3      # pin version - never :latest
    container_name: mviz-triton
    runtime: nvidia
    environment:
      - NVIDIA_VISIBLE_DEVICES=all
      - NVIDIA_DRIVER_CAPABILITIES=compute,utility
    command: >
      tritonserver
      --model-repository=/models
      --model-control-mode=explicit
      --load-model=yolov8x_trt
      --load-model=paddleocr_trt
      --load-model=preprocessing_python
      --load-model=nms_postprocess_python
      --http-port=8000
      --grpc-port=8001
      --metrics-port=8002
      --allow-metrics=true
      --allow-gpu-metrics=true
      --allow-cpu-metrics=true
      --metrics-interval-ms=1000
      --log-verbose=0
      --log-info=true
      --log-warning=true
      --log-error=true
      --pinned-memory-pool-byte-size=2147483648
      --cuda-memory-pool-byte-size=0:10737418240
      --rate-limit=execution_count
      --rate-limit-resource=R1:2
      --exit-on-error=false      # don't crash if one model fails to load
      --strict-readiness=true
    volumes:
      - /models:/models:ro      # model repository (read-only in container)
      - /data:/data:ro          # frame data (unified memory path)
    ports:
      - "8000:8000"      # HTTP
      - "8001:8001"      # gRPC
      - "8002:8002"      # Prometheus metrics
    healthcheck:
      test: ["CMD", "curl", "-sf", "http://localhost:8000/v2/health/ready"]
      interval: 10s
      timeout: 5s
      retries: 3
      start_period: 120s      # TRT engines take 30-60s to load on first start
    restart: unless-stopped
    shm_size: "4gb"          # shared memory for large batch transfers
    ulimits:
      memlock: -1
      stack: 67108864
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: all
              capabilities: [gpu]
```

Triton startup sequence

```
# 1. Compile TRT engines (one-time, must be done before Triton starts)
./scripts/compile_trt_engines.sh

# 2. Start Triton (explicit model control - only loads what is declared)
docker compose up triton -d

# 3. Wait for models to be ready
until curl -sf http://localhost:8000/v2/models/yolov8x_trt/ready; do
  sleep 5; echo "Waiting for YOLO model ..."
done

# 4. Run warmup (triggers CUDA graph capture)
python scripts/triton_warmup.py --batches 1,4,8,16,32 --iterations 20

# 5. Start consumer services (only after Triton is ready)
docker compose up detect-consumer ocr-consumer -d
```

27.14 Updated Observability — Triton Metrics

Triton exposes a native Prometheus endpoint at `:8002/metrics`. No exporter needed.

Triton Prometheus Metrics (Key)

```
# Inference latency per model (Triton native)
nv_inference_request_duration_us{model="yolov8x_trt",version="1"}
nv_inference_queue_duration_us{model="yolov8x_trt",version="1"} # time waiting for batch
nv_inference_compute_input_duration_us{model="yolov8x_trt"} # H2D + input prep
nv_inference_compute_infer_duration_us{model="yolov8x_trt"} # GPU forward pass
nv_inference_compute_output_duration_us{model="yolov8x_trt"} # D2H + output decode

# Throughput
nv_inference_request_success{model="yolov8x_trt"} # total successful requests
nv_inference_count{model="yolov8x_trt"} # total frames (across batches)

# Batch statistics
nv_inference_exec_count{model="yolov8x_trt"} # number of batch executions
# avg_batch_size = nv_inference_count / nv_inference_exec_count # derived metric

# Queue pressure
nv_inference_pending_request_count{model="yolov8x_trt"} # current queue depth

# GPU metrics (via DCGM - Triton exports when --allow-gpu-metrics=true)
nv_gpu_utilization{gpu="0"} # overall GPU utilization
nv_gpu_memory_used_bytes{gpu="0"}
nv_gpu_memory_total_bytes{gpu="0"}
nv_gpu_power_usage{gpu="0"}
nv_gpu_tensor_active{gpu="0"} # Tensor Core utilization %
```

New Alerting Rules for Triton

```
- name: mvis.triton
  rules:
    - alert: TritonNotReady
      expr: up{job="triton"} == 0
      for: 0s
      labels:
        severity: critical
      annotations:
        summary: "Triton Inference Server is down - pipeline halted"

    - alert: TritonYOLOQueueHigh
      expr: nv_inference_pending_request_count{model="yolov8x_trt"} > 200
      for: 15s
      labels:
        severity: warning
      annotations:
        summary: "YOLO queue depth {{ $value }} - consumer may be falling behind"

    - alert: TritonInferenceLowThroughput
      expr: rate(nv_inference_count{model="yolov8x_trt"}[1m]) < 200
      for: 30s
      labels:
        severity: critical
      annotations:
        summary: "YOLO throughput below 200 FPS during active session"

    - alert: TritonBatchSizeLow
      expr: |
        rate(nv_inference_count{model="yolov8x_trt"}[2m]) /
        rate(nv_inference_exec_count{model="yolov8x_trt"}[2m]) < 4
      for: 60s
      labels:
        severity: warning
      annotations:
        summary: "Average YOLO batch size below 4 - dynamic batching not filling"

    - alert: GPURTensoreCoreUtilizationLow
      expr: nv_gpu_tensor_active{gpu="0"} < 40
      for: 60s
      labels:
        severity: warning
      annotations:
        summary: "Tensor Core utilization {{ $value }}% - inference not maximizing hardware"
```

Updated Grafana Dashboards

Dashboard	New Panels Added
Triton Overview	Request throughput per model, queue depth, avg batch size, p50/p95/p99 inference latency
GPU Health (updated)	Add: Tensor Core utilization (<code>nv_gpu_tensor_active</code>), per-model GPU time, model memory usage
Pipeline Overview (updated)	Replace "YOLO service" panels with Triton YOLO model panels
Batch Efficiency	avg batch size over time, queue delay distribution, batch fill rate

27.15 Failure Handling — Triton-Specific

Triton Process Crash

Symptom:

nv_inference_request_success counter stalls
 detect-consumer logs: "StatusCode.UNAVAILABLE: Connection refused"
 TritonNotReady alert fires immediately

Impact:

detect-consumer and ocr-consumer cannot submit inference requests
 preprocessed_frames stream backs up (YOLO/OCR groups stop consuming)
 At 600 FPS: stream hits MAXLEN 5,000 in ~8 seconds → camsvr frame capture stalls

Automatic recovery (Docker restart policy):

Triton container restarts in < 30 seconds
 TRT engine reload: 30-60 seconds (engines are pre-compiled - no recompilation)
 Total recovery: 60-90 seconds

Consumer behavior during Triton down:

detect-consumer: gRPC calls fail with StatusCode.UNAVAILABLE
 → Implement retry with 5-second backoff in the consumer
 → Do NOT XACK frames during retry - they remain pending
 → XAUTOCLAIM reclaims them after 30s if consumer also restarts
 → No frame loss if recovery < 30s (PEL holds them)

```
# In detect-consumer: Triton unavailability handling
async def _infer_with_retry(self, frame_data, request_id):
    for attempt in range(10):
        try:
            return await self.triton.infer(model_name="yolov8x_trt", ...)
        except grpcclient.InferenceServerException as e:
            if "UNAVAILABLE" in str(e):
                wait = min(5 * (2 ** attempt), 60)
                log.warning(f"Triton unavailable, retry in {wait}s (attempt {attempt})")
                await asyncio.sleep(wait)
            else:
                raise # non-retriable error (bad input, model error)
    raise TritonUnavailableError("Triton unavailable after 10 attempts")
```

GPU OOM During Triton Inference

Symptom:

Triton log: "CUDA out of memory. Tried to allocate..."
 Specific model becomes unavailable; other models continue
 (Triton isolates model failures - OCR continues even if YOLO OOMs)

Triton behavior:

--exit-on-error=false means Triton stays alive
 Affected model transitions to UNAVAILABLE state
 Triton returns error for that model only

Recovery:

Option A: Reload model (smaller batch size or fewer instances):
 curl -X POST http://localhost:8000/v2/repository/models/yolov8x_trt/unload
 # Edit config.pbtxt: reduce instance count 4→2, max_batch_size 32→16
 curl -X POST http://localhost:8000/v2/repository/models/yolov8x_trt/load

Option B: Restart Triton container if OOM is persistent
 docker restart mvis-triton

YOLO Model Version Failure (Bad Engine)

```
If a newly loaded TRT engine is corrupt or incompatible:
  Triton refuses to load it (engine validation on load)
  Previous version (version_policy: specific {versions: [1]}) remains active
  No downtime if old version is kept in repository

Safe model update procedure:
1. Place new engine in /models/yolov8x_trt/2/model.plan
2. Update config.pbtxt: version_policy { specific { versions: [2] } }
3. POST /v2/repository/models/yolov8x_trt/load
4. Verify: GET /v2/models/yolov8x_trt (check status == READY)
5. If fails: revert config.pbtxt to versions: [1], reload
6. Total rollback time: < 5 seconds
```

GPU Temperature Overheating

```
Symptom:
  dcm_gpu_temp > 85°C
  Triton inference latency increases (GPU thermal throttling)
  nv_inference_compute_infer_duration_us increases by 20-40%

Triton response (automatic):
  None — Triton does not throttle on temperature
  The GPU driver throttles automatically via NVML

Mitigation:
1. Reduce instance count (fewer concurrent CUDA streams = less heat):
   config.pbtxt: instance_group count 4 → 2
   Triton hot reload
2. Reduce preferred_batch_size (shorter GPU burst per batch):
   preferred_batch_size: [4, 8] instead of [8, 16, 32]
3. Alert operator to check facility cooling

Alert rule:
  alert: GPUOverheating
  expr: dcm_gpu_temp > 85
  → Trigger automatic instance count reduction (via scripted Triton config update)
```

27.16 ADR-007: NVIDIA Triton as Central Inference Coordinator

Status: Accepted

Date: 2026-08-02

Context:

The MVIS inference pipeline requires sustained 600 FPS throughput from a single GB10 Blackwell GPU. The current design (standalone PyTorch containers) achieves 37-45 FPS per container, requiring 13-16 parallel containers to meet demand — each with its own CUDA context, model copy, and memory allocation. This is wasteful, operationally complex, and does not leverage the GB10's Tensor Core architecture.

Problem Statement:

1. PyTorch eager mode inference is 3-7× slower than TensorRT for fixed-shape inference.
2. No dynamic batching → Tensor Core utilization < 10% at batch=1.
3. Multiple CUDA contexts compete for GPU scheduling.
4. No shared GPU memory management across models.
5. Model reload = container restart = 30-90 seconds of pipeline downtime.

Decision:

Deploy NVIDIA Triton Inference Server as the single GPU inference coordinator on DGX Spark. All YOLO and OCR inference runs through Triton. Consumer services become thin Redis→Triton gRPC adapters. TensorRT FP8 engines are compiled on the DGX Spark for the GB10 Blackwell architecture.

Alternatives Rejected:

Alternative	Rejection Reason
TensorRT direct (no server)	Requires reimplementing batching, versioning, metrics, concurrency management — months of infrastructure work
TorchServe	No native TRT backend; 40-60% of Triton performance; no ensemble pipelines
Ray Serve	~3ms per-call overhead vs < 0.5ms for Triton gRPC; designed for multi-node routing (revisit at Tier 3)
ONNX Runtime + TRT provider	ORT wraps TRT, adds overhead; Triton's TRT backend calls TRT directly
More parallel PyTorch containers	Linear RAM scaling (16 containers × 2GB = 32GB); 16 CUDA contexts; no shared batching; operationally unmanageable

Expected Performance Improvement:

Metric	Before (PyTorch, batch=1)	After (Triton TRT FP8, batch=16)	Improvement
YOLO inference time	~18ms (1 frame)	~9ms (16 frames)	32× throughput increase
GPU Tensor Core util	~8%	~80-90%	~10×
FPS capacity (1 container)	~45 FPS	~1,777 FPS	~40×
Model reload time	30-90s (container restart)	< 1s (Triton hot reload)	~60-90×
GPU memory (all models)	2 containers × ~4GB = 8GB	6 instances × avg 2.2GB = 13GB	+5GB for 40× throughput gain

Trade-offs:

Trade-off	Mitigation
TRT engines are GPU-specific (cannot run on different GPU)	Engines compiled on DGX Spark and version-controlled in <code>/models</code> volume. Re-compile for any new GPU before deployment.
TRT compilation takes 10-30 minutes per model	Done once offline. CI pipeline compiles and validates before deployment.
Triton is another service to operate	Coolify manages it. Health check + restart policy handles crashes. Operational overhead is low after initial setup.
BLS ensemble scripts require Triton Python backend expertise	Implement in Phase D-E. Phase A-C use the simpler two-stage consumer pattern.
FP8 accuracy may need validation per use case	Run mAP evaluation comparing FP32 vs FP8 on MVIS-specific dataset before production. Fall back to FP16 or INT8 if accuracy is insufficient.

Future Migration Path:

At Tier 3 (national deployment): Triton remains the per-DGX inference server. Ray Serve or NVIDIA Triton Inference Server Enterprise (TISE) acts as the multi-DGX routing layer above Triton. Each DGX Triton is a backend registered with the routing layer.

27.17 Runbook: Triton Inference Server Failure

Symptoms:

- `TritonNotReady` alert fires
- `detect-consumer` logs: `StatusCode.UNAVAILABLE: Connection refused` or `Failed to connect to all addresses`
- `preprocessed_frames` stream length growing (XLEN increasing, consumers stalled)
- `nv_inference_request_success` counter stalls in Grafana

Diagnostics:

```
# Check Triton container status
docker ps | grep triton
docker logs mviz-triton --tail 100 | grep -E "ERROR|FATAL|failed|abort"

# Check Triton health directly
curl -sf http://localhost:8000/v2/health/live      # should return 200
curl -sf http://localhost:8000/v2/health/ready    # should return 200 once models loaded

# Check individual model status
curl http://localhost:8000/v2/models/yolov8x_trt | python3 -m json.tool
# Look for: "state": "READY"

# Check GPU state
nvidia-smi
dcmi dmon -s u # GPU utilization in real-time

# Check if Redis consumers are stalling
redis-cli XLEN preprocessed_frames # should not exceed 3000 during session
redis-cli XPENDING preprocessed_frames detect-group - + 5
```

Recovery Steps:

```
# Step 1: Attempt container restart (handles 90% of cases)
docker restart mviz-triton
sleep 10
curl -sf http://localhost:8000/v2/health/live

# Step 2: Wait for models to fully load (may take 60-90s for TRT engines)
until curl -sf http://localhost:8000/v2/health/ready; do
    sleep 5; echo "Triton loading models..."
done

# Step 3: Run warmup (CUDA graph capture)
python scripts/triton_warmup.py --batches 8,16,32 --iterations 5
# Abbreviated warmup - production traffic will complete graph capture

# Step 4: Restart consumer services (they will XAUTOCLAIM pending messages)
docker restart mviz-detect-consumer mviz-ocr-consumer

# Step 5: Monitor pipeline resuming
watch -n2 "redis-cli XLEN preprocessed_frames && \
    curl -sf http://localhost:8000/v2/models/yolov8x_trt | python3 -m json.tool | grep state"

# Step 6: If model fails to load (OOM or corrupt engine):
# Option A: Reduce instance count
sed -i 's/count: 4/count: 2/' /models/yolov8x_trt/config.pbtxt
curl -X POST http://localhost:8000/v2/repository/models/yolov8x_trt/load

# Option B: Roll back to previous model version
sed -i 's/versions: \[2\]/versions: \[1\]/' /models/yolov8x_trt/config.pbtxt
curl -X POST http://localhost:8000/v2/repository/models/yolov8x_trt/load
```

Validation:

```
# All models ready
curl http://localhost:8000/v2/models/yolov8x_trt | python3 -m json.tool | grep '"state"'
# Expected: "state": "READY"

# Inference throughput recovering
watch -n5 "curl -sf http://localhost:8002/metrics | grep nv_inference_request_success"
# Counter should be increasing

# Pipeline flowing
watch -n2 "redis-cli XLEN preprocessed_frames"
# Should be decreasing or stable (not growing)
```

Escalation: If TRT engine fails to load after restart: re-compile from ONNX on DGX Spark (30 minutes). If GPU appears dead in `nvidia-smi`: hardware escalation required.

27.18 Updated Capacity Planning — GPU Utilization

This section supersedes the GPU utilization estimates in Section 16.6.

Revised GPU Utilization Model (Triton TRT FP8 on GB10)

Metric	Previous Estimate	Revised Estimate	Change
YOLO per-frame time (batch=1, PyTorch)	15ms	—	Replaced
YOLO per-frame time (batch=16, TRT FP8)	—	~0.56ms effective	New
GPU Tensor Core utilization	40-60% (estimated)	80-92% (measured at batch=16)	+40%
FPS capacity (1 inference path)	~45-66 FPS	~1,777 FPS	27-40×
GPU memory (inference)	~8 GB (2 containers)	~13 GB (6 instances + Triton overhead)	+5 GB
Available for other uses	~120 GB	~115 GB	–5 GB
Instance count needed for 600 FPS	~13 containers	1 Triton + 4 YOLO instances	—

GPU Scaling Thresholds (Revised)

GPU Metric	Scale-Up Trigger	Action
Tensor Core utilization > 85% sustained	> 30 min	Add second DGX or reduce camera count
YOLO queue depth > 500	> 30s	Add YOLO instances: 4→6 (if memory allows)
Triton inference P99 > 200ms	> 60s	Investigate batch fill rate; may need more consumers
GPU memory > 40 GB	Steady state	Re-evaluate instance counts; check for memory leaks

At 600 FPS demand, the GB10 with 4 YOLO instances and Triton TRT FP8 operates at approximately **35-40% GPU Tensor Core utilization**. This provides 2.5-3× headroom before a second DGX is needed. A second DGX becomes justified at sustained > 1,500 FPS demand (approximately 30 cameras at 50 FPS, or 12 cameras at 120 FPS).

27.19 Benchmarking Methodology — Triton-Specific

This section extends Section 24 with Triton-specific benchmark targets and tools.

Triton perf_analyzer (Primary Tool)

```
# Install (comes with Triton SDK container)
docker pull nvcr.io/nvidia/tritonserver:24.12-py3-sdk

# Benchmark YOLO at various concurrency levels and batch sizes
# This is the most important benchmark – run before every model update

# Test 1: Throughput vs concurrency (find the sweet spot)
for concurrency in 1 4 8 16 32 64; do
  perf_analyzer \
    -m yolov8x_trt \
    -u localhost:8001 \
    -i grpc \
    --concurrency-range $concurrency \
    --shape images:1,3,640,640 \
    --measurement-interval 10000 \      # 10 seconds per measurement
    -f results/yolo_concurrency_${concurrency}.csv
done

# Test 2: Latency at fixed concurrency (find operational point)
perf_analyzer \
  -m yolov8x_trt \
  -u localhost:8001 \
  -i grpc \
  --concurrency-range 64 \
  --shape images:1,3,640,640 \
  --percentile 99 \
  --measurement-interval 30000          # 30-second steady-state measurement

# Expected output at optimal configuration (GB10, TRT FP8, batch=16):
# Throughput: 1200-1800 infer/s (each request = 1 frame, batched internally)
# p50 latency: 8-12ms
# p95 latency: 14-18ms
# p99 latency: 18-25ms
```

DCGM for GPU Profiling

```
# Real-time GPU monitoring during perf_analyzer run
dcgmi dmon -s purctm    # utilization, power, clock, temperature, memory
# -s u: GPU utilization
# -s r: PCIe throughput
# -s c: SM clock
# -s t: temperature
# -s m: memory utilization

# Profile Tensor Core utilization specifically:
ncu --metrics sm_pipe_tensor_cycles_active.avg.pct_of_peak_sustained_active \
    --target-processes all \
    -o /tmp/triton_profile \
    sleep 10             # profile for 10 seconds while perf_analyzer runs in background
```

Key Benchmarks to Record

Benchmark	Command	Pass Threshold	Record Frequency
YOLO FPS at batch=16	<code>perf_analyzer -m yolov8x_trt --concurrency-range 64</code>	≥ 1,200 FPS	Before each production deploy
YOLO P99 latency	Same as above, <code>--percentile 99</code>	≤ 25ms	Before each production deploy
Average batch size	<code>nv_inference_count / nv_inference_exec_count</code>	≥ 12 (at 600 FPS input)	Daily during production
Tensor Core utilization	<code>ncu</code> or <code>dcgmi</code>	≥ 70% during session	Weekly
End-to-end pipeline	<code>trace_id</code> timestamps: camsvr → DB commit	P99 ≤ 500ms	Before each production deploy
Triton queue latency	<code>nv_inference_queue_duration_us P99</code>	≤ 15ms	Daily during production

27.20 Scaling Triton: 1 DGX → National Deployment

1 DGX (current)

```

1x Triton server on DGX Spark
4x YOLO instances, 2x OCR instances
Consumers: 1 detect-consumer process, 1 ocr-consumer process
Docker Compose (Coolify)
Capacity: 1,777 FPS YOLO, 2,133 FPS OCR
Target: 600 FPS

```

2-4 DGX (Tier 2)

```

Each DGX: 1x independent Triton server
Each DGX: its own Redis Streams pipeline
VPS: receives sync batches from all DGX sync-engines
Session assignment: VPS session-manager routes new sessions to least-loaded DGX

Orchestration: K3s with NVIDIA GPU Operator
DGX nodes labeled: kubectl label node dgx-1 nvidia.com/gpu.product=GB10
Triton as DaemonSet (one pod per GPU node):
  nodeSelector: { nvidia.com/gpu.product: GB10 }
  resources: { limits: { nvidia.com/gpu: 1 } }

```

10 DGX, Multiple Stations (Tier 3)

```

Per station (3 DGX per station):
  K3s cluster (3-node, HA control plane)
  Triton DaemonSet on all 3 DGX nodes
  Redis Streams per DGX (no cross-DGX stream sharing)
  Local MinIO per station

Station → Central VPS:
  sync-engines from all DGX nodes push to central VPS PostgreSQL
  Frame archivers from all stations push to D0 Spaces (different prefixes)

Triton Model Distribution:
  Model repository on NFS shared across DGX nodes in a station
  Model updates: GitOps (FluxCD) pushes new config.pbtxt → Triton hot reload
  No manual SSH to update models

Central Triton Gateway (optional):
  NVIDIA Triton Inference Server with ensemble routing
  Routes complex multi-stage inference jobs to least-loaded station
  Useful for reprocessing historical sessions with updated models

```

National Deployment — Model Version Management

```

Central model registry (on VPS or dedicated registry server):
/models/
  yolov8x_trt/
    v2.1.0/ ← current production
    v2.2.0/ ← staged (validation in progress)
    v3.0.0/ ← development

Deployment flow:
1. Train new model → export to ONNX
2. CI pipeline: compile TRT engines for each GPU type (GB10, AGX Orin)
3. Run mAP validation against MVIS test dataset (min 0.85 mAP required)
4. Push engines to model registry
5. FluxCD detects new version → updates config.pbtxt on all DGX nodes
6. Triton hot-reloads: new version active in < 1 second per node
7. Canary: route 10% of sessions to new version, monitor for 24 hours
8. If stable: promote to 100%
9. Keep previous version for 7 days (rollback window)

```

27.21 Updated Production Readiness Checklist — Inference Layer

These items supersede and extend the inference-related items in Section 25.

Triton Setup

- ☐ **T0.1** TRT engines compiled on DGX Spark for GB10 Blackwell target (`trtexec --fp8`)
- ☐ **T0.2** mAP validation run: FP8 engine vs FP32 PyTorch on MVIS-specific dataset (document score)
- ☐ **T0.3** Triton container deployed with pinned image tag (never `:latest`)
- ☐ **T0.4** `--exit-on-error=false` set (Triton survives individual model failures)
- ☐ **T0.5** `--model-control-mode=explicit` set (no accidental model auto-loading)
- ☐ **T0.6** All models show `"state": "READY"` via `/v2/models/{name}` endpoint
- ☐ **T0.7** Triton warmup script runs before production traffic is admitted
- ☐ **T0.8** CUDA graphs enabled in all TRT model configs (`cuda { graphs: true }`)

Performance Validation

- ☐ **T1.1** `perf_analyzer` baseline collected: YOLO P99 $\leq$ 25ms at concurrency=64
- ☐ **T1.2** Average batch size $\geq$ 12 at 600 FPS input (confirms dynamic batching is filling)
- ☐ **T1.3** GPU Tensor Core utilization $\geq$ 70% during full-load session
- ☐ **T1.4** End-to-end pipeline P99 $\leq$ 500ms (camsrv $\rightarrow$ DB commit)
- ☐ **T1.5** Triton queue depth $<$ 200 at steady-state 600 FPS

Consumer Services

- ☐ **T2.1** `detect-consumer` uses async gRPC with 64 concurrent in-flight requests
- ☐ **T2.2** `detect-consumer` implements retry with backoff on `StatusCode.UNAVAILABLE`
- ☐ **T2.3** `ocr-consumer` uses async gRPC with 32 concurrent in-flight requests
- ☐ **T2.4** `async-preprocessor` runs 8 parallel CPU workers (Grace CPU)
- ☐ **T2.5** Consumers do NOT XACK frames during Triton unavailability (PEL preserves them)
- ☐ **T2.6** XAUTOCLAIM runs on consumer startup (reclaims pending from crashed consumers)

Observability

- ☐ **T3.1** Triton metrics endpoint (`:8002/metrics`) scraped by Prometheus
- ☐ **T3.2** All Triton alerting rules deployed (Section 27.14)
- ☐ **T3.3** Triton Grafana dashboard deployed with throughput, latency, batch size, queue depth
- ☐ **T3.4** `nv_gpu_tensor_active` metric visible in GPU Health dashboard
- ☐ **T3.5** DCGM Prometheus metrics available (via `dcm-exporter` or Triton `--allow-gpu-metrics`)

Failure Handling

- ☐ **T4.1** Triton `restart: unless-stopped` policy set in Docker Compose
- ☐ **T4.2** Consumer retry behavior validated: sustained inference during 90-second Triton restart
- ☐ **T4.3** Model version rollback tested: revert config.pbtxt + `POST /load` in $<$ 5 seconds
- ☐ **T4.4** Triton runbook (Section 27.17) reviewed by all on-call engineers

Section 27 — AI Inference Layer complete. This section was added in document version 2.1 (2026-08-02).

End of MVIS Enterprise Architecture Document v2.1

This document is the single source of truth for MVIS architecture. All implementation decisions should be traceable to a section in this document. Significant deviations require an ADR entry.

Version	Date	Changes
1.0	2026-07-29	Initial architecture document
2.0	2026-08-02	Complete rewrite: 26 sections, production hardening
2.1	2026-08-02	Added Section 27: AI Inference Layer (NVIDIA Triton, TRT FP8, GB10 analysis, Kubernetes evaluation, benchmarking, runbooks, ADR-007)